

INSTITUT TEKNOLOGI DEL

UTILIZATION OF FREE CLOUD STORAGE FOR RELIABLE BACKUP SYSTEM

PROPOSAL TUGAS AKHIR

13323011 Idvend Agus Sefwyn Manurung
13323014 Tarorom Joseph Abel Gurning
13323032 Yehezkiel Sairotama Rumapea

FAKULTAS VOKASI
PROGRAM STUDI D3 TEKNOLOGI KOMPUTER
INSTITUT TEKNOLOGI
2025

INSTITUT TEKNOLOGI DEL

UTILIZATION OF FREE CLOUD STORAGE FOR RELIABLE BACKUP SYSTEM

PROPOSAL TUGAS AKHIR

Diajukan sebagai salah satu syarat untuk memperoleh gelar Ahli Madya Teknik (A.Md.T)

13323011 Idvend Agus Sefwyn Manurung

13323014 Tarorom Joseph Abel Gurning

13323032 Yehezkiel Sairotama Rumapea

FAKULTAS VOKASI

PROGRAM STUDI D3 TEKNOLOGI KOMPUTER

INSTITUT TEKNOLOGI

2025

DAFTAR ISI

DAFTAR ISI 3
DAFTAR TABEL 5
DAFTAR GAMBAR 6
BAB I PENDAHULUAN 7
1.1 Latar Belakang 7
1.2 Rumusan Masalah 9
1.3 Tujuan 9
1.4 Batasan Penelitian 9
1.5 Hasil Yang Diharapkan 10
1.6 Sistematika Penyajian 10
BAB II TINJAUAN PUSTAKA 12
2.1 Landasan Teori 12
2.1.1 Data Backup 12
2.1.2 Cloud Storage 13
2.1.3 Restore Data 14
2.1.4 Rclone 15
2.1.5 Round Robin Algorithm 17
2.1.6 Cron Job 17
2.1.7 Shell Script 18
2.1.8 Go-Routines (Golang) 18
2.2 Related Work 19
2.2.1 Analysis of Rclone Implementation for File Sharing Security on OneDrive Cloud Storage 19
2.2.2 Strategies for Data Backup and Disaster Recovery in Google Cloud Platform 20
2.2.3 A forensic analysis of rclone and rclone's prospects for digital forensic investigations of cloud storage 21
BAB III ANALISIS DAN DESAIN 22
3.1 Analisis 22
3.1.1 Analisis Masalah 22
3.1.2 Analisis Pemecah Masalah 23
3.2 Analisis Kebutuhan Sistem 26
3.2.1 Analisis kebutuhan Perangkat Keras(Hardware) 26
3.2.2 Analisis kebutuhan Perangkat Lunak(software) 26
3.2.3 Konfigurasi user 27
3.3 System Desain 28
3.3.1 Detailed System Architecture Diagram 30
3.3.2 Desain Database 34
3.3.3 Flowchart 35
3.4 Desain Antarmuka 41

3.4.1 Desain Tampilan Login Page 42
3.4.2 Desain Tampilan Dashboard 43
3.4.3 Desain Tampilan Isi Remote Gdrive 44
3.4.4 Desain Tampilan Halaman Job 45
3.4.5 Desain Tampilan Halaman scheduled Jobs 46
3.4.6 Desain Configuration form 47
BAB IV HASIL DAN PEMABAHASAN 50
4.1 Implementasi 50
4.1.1 Application Tier 50
4.1.2 Presentation Layer 60
4.1.3 Data Layer 67
4.2 Instalasi system 68
4.2.1 Instalasi dan Konfigurasi Sistem 68
4.2.2 Langkah-Langkah Instalasi 69
4.2.3 Konfigurasi Sistem 71
4.3 Pengujian 73
4.3.1 Pengujian Autentikasi 73
4.3.2 Pengujian Job Management 74
4.3.3 Pengujian Operasi Backup 76
4.3.4 Pengujian Operasi restore 76
4.3.5 Pengujian Eksekusi Script 77
4.3.6 Pengujian Operation Mode 78
4.3.7 Pengujian Retensi Data 79
4.3.8 Pengujian Penanganan Error 80
4.3.9 Pengujian Monitoring 81
4.3.10 Pengujian Instalasi 82
4.4 Pembahasan 85
4.4.1 Implementasi Rclone untuk Otomasi Backup 85
4.4.2 Penerapan Arsitektur Layered 85
4.4.3 Manajemen efisiensi penyimpanan dengan algoritma rotasi 86
4.4.4 Parsing Data Sistem 86
4.4.5 Kinerja Sistem Monitoring Terpusat 87
4.4.6 Otomasi Instalasi dengan Shell Script 87
BAB V KESIMPULAN DAN SARAN 89
5.1 Kesimpulan 89
5.2 Saran 90

DAFTAR TABEL

Tabel 3. 1 Hardware 26
Tabel 3. 2 Software 27
Tabel 3. 3 Konfigurasi sistem 28
Tabel 4. 1 Software Requirements 68
Tabel 4. 2 System Requirements 69
Tabel 4. 3 Pengujian Autentikasi 74
Tabel 4. 4 Pengujian Job Management 75
Tabel 4. 5 Pengujian Operasi Backup 76
Tabel 4. 6 Pengujian Operasi restore 77
Tabel 4. 7 Pengujian Eksekusi Script 77
Tabel 4. 8 Pengujian Operation Mode 78
Tabel 4. 9 Pengujian Retensi Data 79
Tabel 4. 10 Pengujian Penanganan Error 80
Tabel 4. 11 Pengujian Monitoring 81
Tabel 4. 12 Pengujian Instalasi 82

DAFTAR GAMBAR

Gambar 3. 1 High-Level system Architecture 28
Gambar 3. 2 Detailed System Architecture Diagram 30
Gambar 3. 3 Retention Flow 36
Gambar 3. 4 Backup Flow 38
Gambar 3. 5 Restore Flow 40
Gambar 3. 6 Desain Login 42
Gambar 3. 7 Desain Tampilan Dashboard 43
Gambar 3. 8 GDrive Remote 44
Gambar 3. 9 Tampilan halaman jobs 45
Gambar 3. 10 Halaman Scheduled Jobs 46
Gambar 3. 11 Backup Config Form 47
Gambar 3. 12 Restore Config Form 49
Gambar 4. 1 Controller 51
Gambar 4. 2 services 51
Gambar 4. 3 repository 51
Gambar 4. 4 Storage Chart 62
Gambar 4. 5 Storage Detail 63
Gambar 4. 6 Live Log 64
Gambar 4. 7 Views 66
Gambar 4. 8 Reuseable Component 66
Gambar 4. 9 Database 67
Gambar 4. 10 Databaase 67
Gambar 4. 11 Rclone CLI 67

BAB I
PENDAHULUAN

Bab ini akan menjelaskan latar belakang masalah untuk memahami alasan dilaksanakannya tugas akhir ini. Tujuan dari tugas akhir akan dipaparkan, dan pertanyaan pertanyaan yang akan dijawab dalam proses pengerjaan akan dirumuskan. Batasan-batasan yang menjadi acuan juga akan dijelaskan, serta hasil yang diharapkan akan dijabarkan. Selain itu, sistematika penyajian tugas akhir akan diuraikan agar pembaca dapat memahami setiap bagian yang membentuk keseluruhan tugas ini.

1.1 Latar Belakang

Salah satu harta paling penting didalam dunia IT adalah data. Data tidak hanya sekadar informasi yang disimpan, tetapi data juga berguna untuk dianalisis atau dikembangkan menjadi sesuatu yang lebih bermakna[1]. Kekhawatiran utama baik bagi organisasi dan individu adalah keamanan data dan penyimpanan dengan biaya yang lebih rendah. Pengguna saat ini menghasilkan lebih banyak file daripada yang dapat disimpan langsung di perangkat mereka[2]. Fitur penyimpanan cloud yang disorot adalah mobilitasnya, dan pengguna dapat mengakses data mereka dengan bantuan koneksi internet yang baik. Mereka dapat mengakses apa pun dari mana saja[3].

Penyimpanan cloud disebut sebagai strategi untuk meningkatkan kemampuan yang sudah ada serta menambahkan fungsi baru tanpa harus berinvestasi pada infrastruktur baru, dalam lingkungan organisasi saat ini, layanan data yang dijalankan oleh penyedia cloud menghadapi banyak tantangan dalam menjaga keandalan layanan data.Fleksibilitas data menjadi syarat penting bagi perusahaan untuk menjaga informasi sekaligus mendukung pertumbuhan kedepannya[4]. Lebih jauh lagi, kontribusi pengguna individu maupun organisasi kecil terhadap perkembangan ekosistem digital juga menegaskan pentingnya pengelolaan data yang baik. Banyak di antaranya masih menghadapi kendala dalam pengelolaan data, salah satunya karena kurangnya sistem backup yang memadai.

Hal ini sesuai dengan temuan Prabowo (2018) yang menunjukkan bahwa keterbatasan sumber daya seringkali membuat sistem backup tidak menjadi prioritas[5].Banyak aplikasi modern, termasuk aplikasi penyimpanan data, memperoleh manfaat dari penerapan arsitektur *Multi-Cloud*. Konsep ini memungkinkan data disimpan dan dikelola pada lebih dari satu penyedia layanan cloud, sehingga memberikan berbagai keuntungan dalam penyimpanan maupun pencadangan data[6]. Meskipun demikian, pendekatan ini menimbulkan kompleksitas, khususnya dalam memastikan sinkronisasi data yang mulus

dan efisien di berbagai platform cloud[7].Dalam konteks penelitian ini, perlindungan data diwujudkan melalui penyimpanan salinan cadangan pada layanan cloud sehingga risiko kehilangan akibat kerusakan perangkat lokal, kesalahan pengguna dapat diminimalisasi.

Dari perspektif pengguna, permasalahan utama dalam pengelolaan data adalah bagaimana memaksimalkan ketersediaan data dengan biaya serendah mungkin. Biaya ini umumnya meliputi biaya penyimpanan dan biaya jaringan, yang pada layanan berbayar cukup signifikan. Dalam konsep *multi-cloud*, tantangan tersebut biasanya dijawab dengan strategi penempatan data yang optimal agar biaya tetap rendah sekaligus ketersediaan data tetap terjaga[8].Namun, dalam penelitian ini pendekatannya lebih sederhana, yaitu dengan memanfaatkan layanan cloud gratis seperti Google Drive sebagai media utama.

Akan tetapi, pendekatan ini masih memiliki kekurangan yakni keamanan data karena Penyimpanan data yang terpusat pada satu layanan cloud memiliki risiko yang cukup tinggi, antara lain kebocoran data, serangan siber, maupun kegagalan sistem yang dapat menyebabkan hilangnya data penting[9]. Adaptasi teknologi penyimpanan cloud telah merevolusi berbagai manajemen data bagi organisasi, Meskipun demikian, perubahan ini juga membawa tantangan keamanan yang signifikan. Kompleksitas arsitektur cloud yang *multi tenant* semakin memperbesar risiko kebocoran data sehingga diperlukan solusi yang tidak hanya fokus pada penyimpanan, tetapi juga pada aspek keamanan dan perlindungan data[10].

Berdasarkan Penelitian penelitian sebelumnya, sistem backup berbasis cloud sudah banyak dikembangkan dengan berbagai pendekatan. Meskipun demikian Sebagian besar penelitian tersebut cenderung berfokus pada layanan penyimpanan cloud yang masih berbayar, skala sistemnya yang besar atau konteksnya mengarah pada industry atau instansi yang besar sehingga kurang relevan untuk kebutuhan individu dan organisasis kecil. Selain itu keterbatasan pada sinkronisasi atau dump untuk database, kurangnya antarmuka pengguna (UI) untuk mempermudah pengguna dalam proses backup masih belum banyak dibahas secara mendalam.Karena itu,proyek ini mencoba untuk mengisi kekurangan tersebut dengan berfokus pada pemanfaatan free cloud storage seperti Google drive sebagai media penyimpanan utama. Proyek ini juga menargetkan perancangan sistem backup yang mudah digunakan, andal dalam pemulihan data serta hemat biaya. Dengan demikian, proyek ini diharapkan dapat menjadi alternatif Solusi backup yang lebih praktis bagi pengguna yang tidak memiliki sumber daya yang besar.

1.2 Rumusan Masalah

Adapun rumusan masalah dalam pengerjaan Tugas Akhir ini adalah:

1. Bagaimana merancang dan membangun sistem backup berbasis cloud yang aman, mudah digunakan, dan mampu mengelola data secara terpusat melalui antarmuka berbasis web?
2. Bagaimana memanfaatkan layanan cloud storage gratis seperti Google Drive secara optimal untuk proses backup, restore, dan sinkronisasi data?
3. Bagaimana memastikan sistem backup mampu melakukan pencadangan data secara otomatis?
4. Bagaimana merancang sistem manajemen backup terpusat yang mengotomatisasi proses pencadangan ke *cloud storage* secara andal?

1.3 Tujuan

Proyek ini bertujuan untuk membangun sebuah sistem manajemen backup berbasis cloud yang dapat digunakan secara gratis, mudah dioperasikan, dan mampu bekerja secara otomatis tanpa ketergantungan pada layanan berbayar. Sistem dirancang untuk menyediakan solusi pencadangan dan pemulihan data yang aman dengan memanfaatkan Google Drive sebagai media penyimpanan utama. Melalui integrasi Rclone, cron job, serta dukungan prescripts dan postscripts, proyek ini bertujuan untuk menciptakan proses backup yang terjadwal, dan andal sehingga pengguna tidak perlu melakukan pengaturan atau sinkronisasi secara manual. Selain itu, proyek ini juga bertujuan untuk menghadirkan antarmuka web yang sederhana dan intuitif agar pengguna dapat dengan mudah mengelola konfigurasi backup, melakukan restore, serta memantau kapasitas penyimpanan cloud. Secara keseluruhan, proyek ini ditujukan untuk menghasilkan sistem backup praktis dan hemat biaya yang dapat membantu individu maupun organisasi kecil dalam menjaga keamanan, integritas, dan ketersediaan data mereka.

1.4 Batasan Penelitian

Adapun hal-hal yang menjadi batasan dalam pengerjaan Tugas Akhir ini adalah:

1. Platform cloud storage dibatasi pada Google Drive dengan kapasitas gratis, tanpa membahas integrasi dengan layanan cloud berbayar atau penyimpanan cloud lainnya.
2. Lingkungan sistem operasi yang digunakan adalah linux(ubuntu), sehingga implementasi pada OS lain(Widows/MacOS) tidak dibahas lebih lanjut.

3. Ukuran data yang dapat di backup menyesuaikan dengan kapasitas penyimpanan Google Drive,sehingga tidak dirancang untuk kebutuhan skala besar (enterprise).
4. Meskipun interaksi pengguna difokuskan melalui antarmuka grafis (UI) berbasis web, Pengaitan Gdrive ke sistem masih harus memakai Rclone CLI.
5. Segala hal tentang perubahan Drive yang ingin di remote hanya bisa dilakukan secara manual melalui terminal, karena ada batasan pada tools itu sendiri
6. File yang bisa ditransfer oleh sistem hanya file yang bersifat tidak aktif/tidak sedang berjalan real time.

1.5 Hasil Yang Diharapkan

Dari penelitian dan pengembangan proyek ini, diharapkan dihasilkan sebuah sistem backup berbasis cloud gratis yang mampu memberikan solusi pencadangan data yang aman, dan mudah digunakan oleh individu maupun organisasi dengan keterbatasan sumber daya. Sistem ini diharapkan mampu menjalankan proses backup secara otomatis dan terjadwal menggunakan mekanisme cron job, sehingga pencadangan dapat berlangsung konsisten tanpa bergantung pada intervensi pengguna. Selain itu, antarmuka berbasis web yang dikembangkan diharapkan dapat mempermudah pengguna dalam mengatur konfigurasi backup, menjalankan proses restore, memantau status penyimpanan, serta melihat riwayat aktivitas sistem. Sistem juga diharapkan mampu menyediakan proses pemulihan data (restore) yang cepat, selektif, dan andal, baik untuk file maupun database, sehingga pengguna dapat mengembalikan data penting dengan aman ketika terjadi kesalahan, kerusakan, atau kehilangan data. Integrasi prescripts dan postscripts diharapkan dapat memastikan setiap proses backup dan restore berjalan dengan valid, terkontrol, serta menjaga integritas data. Lebih jauh lagi, kemampuan monitoring kapasitas cloud storage dan status koneksi diharapkan dapat membantu pengguna mengantisipasi keterbatasan ruang penyimpanan serta mendeteksi masalah sinkronisasi sejak dini. Dengan demikian, sistem ini mampu menjadi solusi alternatif yang praktis, hemat biaya, dan relevan untuk kebutuhan backup data di lingkungan komputasi modern.

1.6 Sistematika Penyajian

Sistematika penyajian dalam laporan tugas akhir ini disusun untuk memberikan alur pembahasan yang runtut, jelas, dan mudah dipahami. Adapun susunan laporan ini adalah sebagai berikut:

1. **BAB I PENDAHULUAN -** Bab ini menyajikan gambaran umum mengenai penelitian yang dilakukan. Isinya meliputi latar belakang penelitian, rumusan masalah, tujuan penelitian, batasan penelitian, hasil yang diharapkan, serta sistematika penyajian laporan. Bab ini memberikan fondasi awal bagi pembaca untuk memahami konteks dari sistem yang dikembangkan.
2. **BAB II TINJAUAN PUSTAKA** - Bab ini berisi landasan teori dan konsep-konsep terkait yang mendukung penelitian. Pembahasan mencakup teori tentang backup data, cloud storage, sinkronisasi file, proses restore, Shell Script, Cron Job, prescripts dan postscripts, serta pemanfaatan Rclone. Selain itu, bab ini memuat penelitian terdahulu sebagai perbandingan dan referensi untuk memperkuat dasar teori dalam pengembangan sistem.
3. **BAB III ANALISIS DAN DESIGN -** Bab ini berfokus pada analisis dan perancangan solusi. Isinya meliputi analisis masalah, analisis pemecahan masalah, analisis kebutuhan perangkat lunak dan perangkat keras, perancangan arsitektur sistem menggunakan High Level Architecture (HLA), diagram blok sistem, perancangan antarmuka dashboard, flowchart proses backup dan restore, serta desain basis data menggunakan ERD. Bab ini menjadi acuan dalam proses implementasi sistem.
4. **BAB IV HASIL DAN PEMBAHASAN -** Bab ini menyajikan hasil implementasi sistem berdasarkan rancangan yang telah dibuat. Pembahasan mencakup tampilan antarmuka, cara kerja sistem, proses konfigurasi backup dan restore, hasil pengujian sinkronisasi dengan Rclone, eksekusi prescripts dan postscripts, serta analisis terhadap performa sistem yang dibangun. Bab ini juga membahas bagaimana sistem memenuhi tujuan penelitian dan menjawab rumusan masalah.
5. **BAB V KESIMPULAN DAN SARAN -** Bab ini berisi kesimpulan yang diperoleh dari proses penelitian dan implementasi sistem. Kesimpulan dirumuskan berdasarkan hasil analisis pada bab sebelumnya dan ketercapaian tujuan penelitian. Selain itu, bab ini juga memuat saran yang ditujukan untuk pengembangan sistem lebih lanjut, baik dari segi fitur, performa, maupun penerapan di lingkungan yang lebih luas.

BAB II
TINJAUAN PUSTAKA

Pada bab ini akan dijelaskan mengenai tujuan pustaka sebagai dasar teori yang akan digunakan sebagai landasan utama dalam perancangan dan pembangunan produk yang digunakan dalam pembuatan Tugas Akhir ini.

2.1 Landasan Teori

Bagian ini membahas konsep konsep dasar yang digunakan dalam penelitian ini,mencakup definisi data, backup, cloud storage, sinkronisasi, serta proses restore. Teori teori ini penting sebagai dasar dalam perancangan dan implementasi sistem backup berbasis cloud.

2.1.1 Data Backup

Backup adalah proses pembuatan salinan data dengan tujuan melindungi informasi penting dari kehilangan, baik secara keseluruhan maupun sebagian, serta memungkinkan pemulihan pekerjaan yang terganggu akibat kegagalan sistem atau interupsi. Dalam konteks komputasi awan, backup tidak hanya mencakup penyimpanan data akhir, tetapi juga checkpoint atau intermediate data yang dihasilkan selama proses eksekusi pekerjaan, sehingga pemulihan dapat dilakukan dari titik terakhir tanpa harus mengulang seluruh pekerjaan[11]. Dalam praktiknya, proses backup melibatkan berbagai kebijakan dan tahapan, tergantung pada jenis data dan sistem yang digunakan. Backup tradisional memungkinkan pemulihan data langsung dari salinan, namun sering kali memerlukan pemulihan seluruh sistem, yang memakan waktu lama terutama untuk database berukuran besar. Proses ini biasanya melibatkan pengaturan kebijakan backup, seperti backup berbasis file yang menyalin data sistem operasi dan perangkat lunak, serta backup khusus database yang menyalin database itu sendiri. Jika backup tidak dilakukan dengan benar, pemulihan sistem dapat memakan waktu tambahan dan memerlukan pengumpulan informasi dari dokumen asli, sehingga mengakibatkan hilangnya sebagian data penting dan penundaan layanan. Oleh karena itu, alur backup harus dirancang dengan cermat, mencakup pembuatan salinan data, pengaturan kebijakan, dan strategi pemulihan yang efektif untuk menjamin kontinuitas operasional[12].

Untuk projek ini, metode backup yang digunakan adalah metode full backup. Full Backup (pencadangan penuh) didefinisikan sebagai metode pencadangan yang paling

sederhana dan sangat diperlukan untuk menjamin keamanan data. Metode yang paling mendasar yang disebut juga total backup adalah pencadangan fisik di mana seluruh berkas data, log, dan kontrol sistem disalin ke lokasi lain setelah database dimatikan (*off-line*), menjadikannya tidak tersedia bagi pengguna. Keuntungan utama dari total backup ini adalah kemampuannya untuk mengamankan semua citra (*images*) dari database sehingga sistem dapat kembali ke keadaan awal jika terjadi kerusakan media penyimpanan. Namun, kerugian signifikan dari total backup adalah perlunya database di-shutdown, dan karena menyalin seluruh berkas, proses ini memakan waktu lebih lama dan biaya yang lebih besar, terutama seiring bertambahnya ukuran data. Untuk mengatasi keterbatasan ini, dikembangkanlah skema pencadangan lain, tetapi *Full Backup* tetap menjadi operasi dasar yang harus dilakukan secara berkala, di mana ia akan mengekspor semua berkas dan memperbarui atribut arsip sehingga sistem kembali ke keadaan awal sebelum skema incremental atau cumulative dilakukan [13].

2.1.2 Cloud Storage

Cloud storage adalah bagian dari infrastruktur berbasis cloud yang digunakan untuk penyimpanan, pengelolaan, dan pemrosesan data. Dengan berkembangnya kebutuhan perusahaan akan data yang terus meningkat setiap hari, metode penyimpanan tradisional seperti tape drive atau media lokal dianggap semakin usang, sehingga bisnis beralih ke solusi berbasis cloud. Keuntungan utama cloud storage meliputi backup otomatis, pemulihan data lebih cepat, dan replikasi data di berbagai lokasi, yang mendukung kontinuitas bisnis serta mengurangi risiko kehilangan data akibat kegagalan sistem, serangan siber, atau kesalahan manusia. Penggunaan cloud storage ini menjadi bagian penting dari strategi backup dan disaster recovery modern, memungkinkan perusahaan untuk memanfaatkan model cloud, hybrid, dan multi-cloud secara fleksibel dan skalabel[14]. Cloud storage merupakan layanan penyimpanan berbasis cloud yang memungkinkan data disimpan secara online dan diakses kapan saja dari mana saja. Meski memberikan fleksibilitas dan efisiensi yang tinggi, penggunaan cloud juga menghadirkan risiko keamanan, karena data dari berbagai pengguna ditempatkan dalam infrastruktur bersama dan kontrol atas data berpindah ke penyedia layanan. Hal ini menimbulkan tantangan terkait kerahasiaan, integritas, dan ketersediaan data, sehingga diperlukan mekanisme perlindungan tambahan seperti enkripsi sebelum data diunggah ke cloud[15].

Google Drive adalah layanan cloud storage yang dikembangkan oleh Google dan diluncurkan pada April 2012. Google Drive menyediakan penyimpanan gratis sebesar 15

GB untuk setiap akun Google dan dapat ditingkatkan melalui paket berlangganan Google One. Sebagai platform cloud storage, Google Drive memiliki beberapa fitur utama yang mendukung produktivitas pengguna. Fitur penyimpanan dan sinkronisasi memungkinkan pengguna untuk menyimpan berbagai jenis file yang akan otomatis tersinkronisasi di semua perangkat yang terhubung. Google Drive juga mendukung kolaborasi real-time dimana multiple pengguna dapat mengakses dan mengedit dokumen secara bersamaan dengan perubahan yang tersimpan otomatis. Platform ini terintegrasi penuh dengan ekosistem Google seperti Google Docs, Sheets, Slides, Gmail, dan aplikasi Google lainnya, sehingga memudahkan workflow pengguna.

Dari segi sharing dan permission management, Google Drive memungkinkan pengguna untuk membagikan file dengan kontrol akses yang detail seperti view, comment, dan edit. Yang paling penting untuk pengembangan aplikasi adalah tersedianya Google Drive API yang menyediakan REST API untuk memungkinkan developer mengintegrasikan fungsionalitas Google Drive ke dalam aplikasi pihak ketiga. Google Drive API menggunakan protokol HTTP/HTTPS dengan format data JSON untuk komunikasi antara aplikasi client dan server Google Drive. API ini mendukung operasi *CRUD (Create, Read, Update, Delete)* pada file dan folder, serta fitur advanced seperti search, permission management, dan file sharing [16].

2.1.3 Restore Data

Restore data dalam konteks cloud storage merujuk pada proses pemulihan data dari backup untuk memastikan data tetap tersedia, lengkap, dan terlindungi dari kehilangan akibat kegagalan sistem, kesalahan manusia, atau kejadian tak terduga. Solusi pemulihan berbasis cloud memungkinkan organisasi untuk mengambil kembali data secara cepat dan efektif, sekaligus mengurangi risiko kehilangan data dan waktu henti. Otomatisasi backup secara berkala menjadi lapisan proteksi tambahan yang mendukung kontinuitas operasional dan integritas data. Dengan kemampuan skalabilitas dan kemudahan pengelolaan, cloud-based backup dan recovery memastikan data yang penting tetap aman, konsisten, dan selalu tersedia bagi perusahaan dalam lingkungan kerja yang sangat bergantung pada data[17]. Selain backup berbasis cloud, pemulihan data pada proyek ini menggunakan metode yang memastikan data tetap lengkap, konsisten, dan operasional sistem tetap berjalan lancar. Point-in-time recovery dapat diterapkan untuk mengembalikan file ke kondisi tertentu sebelum terjadi kehilangan atau kerusakan, tanpa memengaruhi data terbaru yang tersimpan. Layanan Google Drive menyediakan fitur pemulihan bawaan yang mendukung

restore file secara cepat dan efisien. Dengan otomatisasi backup dan penggunaan fitur restore, proyek ini memungkinkan data yang dikembalikan lebih lengkap, konsisten, dan siap digunakan untuk analisis atau pengambilan keputusan tanpa memerlukan server tambahan[18].

2.1.4 Rclone

Rclone merupakan perangkat lunak berbasis command-line yang berfungsi untuk mengelola dan melakukan sinkronisasi file pada berbagai layanan cloud storage. Rclone mendukung sejumlah fitur penting seperti enkripsi, sinkronisasi, dan pencadangan data, sehingga menjadi alat yang kuat untuk meningkatkan keamanan sekaligus efisiensi dalam manajemen file di cloud. Melalui kemampuannya dalam melakukan enkripsi, Rclone memastikan bahwa data yang dikirim maupun disimpan tetap terlindungi dari akses yang tidak sah, sekaligus mampu melakukan sinkronisasi data secara otomatis. Dengan kombinasi fitur tersebut, Rclone tidak hanya berperan sebagai media transfer file, tetapi juga sebagai solusi yang dapat membantu menjaga integritas serta kerahasiaan data dalam lingkungan cloud storage [19].

Arsitektur Rclone dirancang secara modular dengan empat layer utama yang bekerja terkoordinasi. Layer paling atas adalah User Interface Layer yang menerima input pengguna melalui command-line interface, Remote Control API, atau web-based GUI. Di bawahnya terdapat Rclone Core Engine sebagai komponen inti yang terdiri dari Command Parser untuk menguraikan perintah, Operation Manager untuk mengelola eksekusi operasi file, Config Manager untuk menangani konfigurasi, dan Token Manager untuk mengelola autentikasi dengan cloud storage provider.

Layer ketiga adalah Backend Drivers Layer yang berisi driver spesifik untuk setiap cloud storage provider seperti Google Drive Driver yang menggunakan Google Drive API v3 atau Dropbox Driver yang menggunakan Dropbox API v2. Setiap driver mengimplementasikan interface standar Rclone Core namun disesuaikan dengan API masing-masing provider. Layer terbawah adalah Cloud Storage Providers yang diakses oleh Backend Drivers melalui REST API dengan protokol HTTPS. Arsitektur modular ini memberikan keuntungan berupa separation of concerns yang jelas, kemudahan penambahan provider baru tanpa mengubah core engine, dan konsistensi interface untuk berbagai provider yang berbeda.

Rclone berkomunikasi dengan Google Drive melalui Google Drive API v3 menggunakan arsitektur REST dengan protokol HTTPS dan format data JSON. Proses

komunikasi dimulai ketika pengguna menjalankan perintah Rclone. Core Engine melakukan parsing perintah, load konfigurasi dan token, kemudian memeriksa validitas access token. Jika token expired, Rclone otomatis melakukan refresh dengan mengirim HTTP POST request ke Google Token Endpoint beserta refresh_token, client_id, client_secret, dan grant_type. Google mengembalikan access token baru yang disimpan ke file konfigurasi tanpa memerlukan interaksi pengguna.

Setelah token valid, Rclone membuat HTTP request sesuai operasi yang diminta. Untuk list files, Rclone mengirim HTTP GET ke endpoint files dengan header Authorization berisi access token dalam format Bearer dan query parameters seperti pageSize untuk membatasi jumlah items per halaman, fields untuk optimasi bandwidth, dan q untuk filtering. Request dikirim melalui HTTPS ke Google Drive API Server yang kemudian memvalidasi token, memeriksa permission, dan query data dari database. Google mengembalikan response JSON berisi array of objects dengan field id, name, mimeType, size, modifiedTime, dan parents. Jika hasil melebihi pageSize, response menyertakan nextPageToken untuk pagination otomatis hingga semua data terambil.

Untuk upload file, proses melibatkan dua tahap: pertama, membuat metadata file dengan HTTP POST ke upload endpoint menggunakan metode resumable upload dan mendapat upload URL unik; kedua, upload konten file dengan HTTP PUT ke upload URL tersebut. File besar dibagi menjadi chunks untuk memungkinkan resumable upload jika terjadi kegagalan di tengah proses. Download file dilakukan dengan HTTP GET ke file endpoint dengan parameter alt media untuk mendapatkan konten file, dengan dukungan range requests untuk file besar yang memungkinkan download partial dan resume.

Rclone mengimplementasikan error handling untuk berbagai HTTP status code: 401 Unauthorized memicu token refresh otomatis dan retry request, 403 Forbidden yang menandakan rate limit ditangani dengan exponential backoff, 404 Not Found dikembalikan ke pengguna, dan 5xx server error di-retry beberapa kali dengan delay yang meningkat. Google Drive API memberlakukan quota limit untuk mencegah abuse dan memastikan fair usage. Untuk meningkatkan efisiensi, Rclone menggunakan request batching untuk menggabungkan multiple operasi kecil menjadi satu request, concurrent requests untuk mengirim multiple requests secara parallel dan memaksimalkan bandwidth, serta caching metadata file yang sering diakses untuk mengurangi query ke server.

2.1.5 Round Robin Algorithm

Algoritma Round Robin secara konsep merupakan metode penjadwalan yang mengedepankan prinsip keadilan (fairness) dengan membagi waktu eksekusi ke dalam slot-slot kecil yang disebut sebagai time slice atau quantum. Dalam mekanisme ini, setiap tugas atau proses di dalam antrean diberikan kesempatan untuk berjalan sesuai durasi kuantum tersebut secara bergantian. Apabila suatu proses belum selesai hingga jatah waktunya habis, sistem akan menghentikan proses tersebut secara sementara (preemptive) dan memindahkannya ke urutan paling belakang antrean agar proses lain mendapatkan giliran. Hal ini bertujuan untuk mencegah terjadinya dominasi satu proses panjang yang dapat menyebabkan proses lainnya tertunda dalam waktu lama, sehingga responsivitas sistem tetap terjaga.

Dalam implementasinya pada proyek G-Backup, konsep ini diadopsi menjadi logika utama pada Manajer Antrean Job Backup untuk mengatur jalannya berbagai instruksi backup yang telah dijadwalkan oleh pengguna. Alih-alih menjalankan backup satu per satu secara linear yang berisiko menyebabkan kemacetan jika terdapat file berukuran besar, sistem menggunakan prinsip Round Robin untuk memastikan bahwa setiap job (seperti backup database, folder, atau skrip) mendapatkan slot eksekusi secara teratur. Dengan menerapkan pengembangan berupa Intelligent Round Robin, sistem Anda menjadi lebih cerdas karena mampu menentukan kuantum waktu secara dinamis dengan mempertimbangkan burst time atau perkiraan beban transfer data ke Google Drive. Hubungan ini memastikan bahwa penggunaan sumber daya server dan bandwidth melalui Rclone tetap optimal, meminimalisir waktu tunggu (waiting time) pada dashboard, dan mempercepat total waktu penyelesaian (turnaround time) seluruh rangkaian tugas backup secara keseluruhan[20].

2.1.6 Cron Job

Cron Job adalah mekanisme penjadwalan tugas yang berasal dari sistem operasi berbasis Unix dan berfungsi sebagai alat penting dalam mengotomatisasi pekerjaan berulang (*repetitive tasks*) dengan intervensi manual yang minimal. Dalam konteks manajemen data *cloud* modern, *Cron Job* menyediakan cara yang sederhana namun kuat untuk mengeksekusi tugas-tugas terjadwal pada waktu atau interval yang telah ditentukan secara otomatis. Fungsi utama *Cron Job* adalah menggantikan metode manual yang dinilai tidak efisien dan rawan kesalahan—seperti transfer data dan pembaruan *database*—yang

dapat menyebabkan penundaan, inkonsistensi data, dan peningkatan biaya operasional pada aplikasi *cloud*. Meskipun efektif untuk otomatisasi dasar, *Cron Job* yang tradisional memiliki keterbatasan, yaitu kurangnya integrasi dengan teknik *scripting* canggih dan platform manajemen *cloud* modern[21]. Oleh karena itu penggunaan *cron job* pada proyek ini merupakan suatu keharusan dimana pada bagian scheduling sebuah job sistem memakai cron job, meskipun merupakan bagian dari library bahasa pemograman yang digunakan, bukan cron job bawaan linux .

2.1.7 Shell Script

Shell Script pada dasarnya adalah skrip yang ditulis untuk juru bahasa baris perintah *(command-line interpreter)* atau yang disebut "shell" (seperti Bash, Zsh, atau Tcsh), yang memiliki fungsi utama untuk menginterpretasikan dan mengeksekusi urutan perintah. Awalnya dikembangkan pada awal tahun 1970-an untuk mempermudah eksekusi perintah bagi administrator di lingkungan UNIX, shell scripting telah berkembang menjadi teknologi inti yang sangat kuat. Fungsinya mencakup manajemen lingkungan server, mengotomatisasi workflow yang kompleks, dan memfasilitasi manipulasi data yang cepat. Saat ini, shell script digunakan secara luas untuk mengotomatisasi tugas-tugas penting seperti pemrosesan batch, pencadangan data, instalasi perangkat lunak, dan pemantauan sistem, menjadikannya tulang punggung dalam alur kerja DevOps dan *Continuous Integration/Continuous Deployment (CI/CD)*. Sementara itu, Bash *(Bourne Again Shell),* yang dirilis pada tahun 1989 dan menjadi shell default untuk sistem Linux, merupakan salah satu jenis shell yang paling umum digunakan, menggabungkan fitur-fitur dari Bourne dan Korn shells dengan fungsionalitas skrip yang diperluas[22]. Shell script pada proyek ini merupakan hal yang vital sebab penggunaan shell script adalah salah satu fitur utama yang disediakan untuk mempermudah user admin mengelola file sebelum dibackup memakai rclone.

2.1.8 Go-Routines (Golang)

Goroutines merupakan mekanisme lightweight concurrency dalam bahasa pemrograman Go yang memungkinkan sebuah fungsi berjalan secara bersamaan (concurrent) dengan fungsi lainnya tanpa membebani sumber daya sistem secara berlebihan. Secara teknis, Goroutines disebut sebagai utas ringan karena dikelola sepenuhnya oleh Go Runtime scheduler dan bukan oleh sistem operasi, sehingga hanya membutuhkan memori awal sebesar 2KB, jauh lebih efisien dibandingkan thread tradisional

yang memerlukan 1-2MB. Keunggulan utamanya terletak pada kemampuan untuk menjalankan ribuan hingga jutaan tugas secara simultan, di mana Go secara otomatis mendistribusikan beban kerja tersebut ke utas sistem operasi yang tersedia, sehingga menciptakan lingkungan pemrograman paralel yang sangat skalabel dengan overhead memori dan context switching yang sangat rendah.

Dalam implementasinya pada proyek G-Backup, konsep Goroutines menjadi fondasi utama yang memungkinkan sistem menjalankan operasi backup secara asinkron dan non-blocking. Melalui penggunaan keyword go sebelum pemanggilan fungsi backup, sistem dapat memicu proses transfer data ke Google Drive melalui Rclone sebagai tugas latar belakang, sehingga backend tetap sepenuhnya responsif dalam melayani permintaan dari dashboard tanpa harus menunggu proses unggah yang lama selesai. Hubungan ini sangat krusial bagi proyek Anda, karena memungkinkan penanganan banyak tugas backup sekaligus secara paralel tanpa menghabiskan RAM server, yang menjamin sistem tetap stabil meskipun dijalankan pada headless server dengan spesifikasi terbatas. Selain itu, sinkronisasi antar Goroutines memastikan pencatatan log dan pembaruan status pada antarmuka pengguna tetap akurat dan real-time, sehingga menciptakan pengalaman pengguna yang mulus meskipun sistem sedang menangani beban kerja transfer data yang berat di balik layar.[23]

2.2 Related Work

Pada sub bab ini akan dijelaskan mengenai jurnal yang terkait dengan sistem yang akan dibangun. Berikut jurnal yang terkait dengan sistem yang akan dibangun.

2.2.1 Analysis of Rclone Implementation for File Sharing Security on OneDrive Cloud Storage

Penelitian ini menguji efektivitas Rclone sebagai alat untuk meningkatkan keamanan berbagi berkas di layanan penyimpanan awan OneDrive. Fokus utama dari studi ini adalah pada dua fitur kunci: pencadangan berkas *(file backup)* dan enkripsi berkas *(file encryption)*. Dengan menggunakan metodologi *Network Development Life Cycle (NDLC),* peneliti melakukan analisis, perancangan, dan prototipe simulasi dalam lingkungan virtual, yang melibatkan server Ubuntu dan klien Windows, untuk menguji skenario pencadangan dan enkripsi data.

Hasil pengujian menunjukkan bahwa Rclone berhasil dan efektif dalam kedua fungsi tersebut. Analisis waktu pencadangan menunjukkan bahwa durasi proses dipengaruhi oleh ukuran berkas; misalnya, rata-rata waktu yang dibutuhkan untuk mencadangkan berkas .docx adalah 2,033 detik, sementara rata-rata waktu untuk berkas .pdf adalah 4,666 detik. Selain itu, fitur enkripsi juga terbukti berhasil, dan enkripsi dapat terpenuhi baik dari sisi server maupun klien. Secara keseluruhan, waktu rata-rata yang dibutuhkan untuk mengenkripsi empat jenis berkas dalam tiga kali percobaan adalah 37 detik. Penelitian ini menyimpulkan bahwa Rclone adalah solusi yang sangat efektif untuk kebutuhan keamanan berbagi berkas di penyimpanan awan, karena tidak hanya meningkatkan perlindungan data tetapi juga menawarkan fleksibilitas dan efisiensi dalam manajemen berkas[23]. Berbeda dengan penelitian tersebut yang menekankan pada pengujian performa backup dan enkripsi dalam konteks OneDrive, proyek ini difokuskan pada implementasi Rclone untuk integrasi backup, restore, sinkronisasi, serta monitoring kapasitas penyimpanan di Google Drive, lengkap dengan antarmuka dashboard yang dirancang agar lebih ramah bagi pengguna awam.

2.2.2 Strategies for Data Backup and Disaster Recovery in Google Cloud Platform

Penelitian ini berfokus pada pengembangan paket RClone dalam bahasa R untuk analisis genetika populasi. Paket ini digunakan untuk mengidentifikasi multilocus genotypes (MLGs) dan multilocus lineages (MLLs), menghitung indeks keragaman serta kemerataan genotipe, hingga menganalisis distribusi spasial organisme yang berkembang biak secara klonal. Studi ini banyak dimanfaatkan pada konteks ekologi dan evolusi, terutama karena teknologi *next-generation sequencing* (NGS) menghasilkan data dalam jumlah besar yang perlu dikelola secara efisien. Dari sisi metodologi, penelitian ini memiliki kemiripan dengan proyek kami karena sama-sama menekankan pentingnya pengelolaan data berukuran besar dengan cara yang lebih efisien, meskipun ruang lingkupnya berbeda[24]. Perbedaan mendasar terletak pada konteks penerapan: penelitian tersebut memanfaatkan RClone untuk analisis genetika populasi, sedangkan proyek kami memanfaatkan Rclone sebagai command-line tool untuk backup, sinkronisasi, restore, serta enkripsi data berbasis Google Drive. Selain itu, proyek kami menambahkan aspek praktis berupa dashboard antarmuka yang memudahkan pengguna dalam mengelola data digital mereka, bukan sekadar fokus pada analisis ilmiah.

2.2.3 A forensic analysis of rclone and rclone's prospects for digital forensic investigations of cloud storage

Penelitian ini membahas analisis forensik digital terhadap penggunaan Rclone sebagai perangkat command-line untuk sinkronisasi, pencadangan, dan transfer file pada layanan cloud storage. Studi ini menemukan bahwa Rclone meninggalkan berbagai artefak digital, seperti file konfigurasi, credential cache, dan log penggunaan, yang dapat dijadikan bukti penting dalam investigasi forensik. Eksperimen dilakukan lintas platform pada Windows, Linux, dan macOS untuk memetakan jejak forensik yang dihasilkan. Hasil penelitian menunjukkan bahwa meskipun Rclone efektif dalam melakukan sinkronisasi dan backup, potensi risiko keamanan tetap ada, terutama karena penyimpanan credential secara plaintext. Oleh karena itu, penulis menekankan pentingnya pengembangan metode investigasi berbasis artefak Rclone untuk mendukung penyelidikan kejahatan berbasis cloud storage. Berbeda dengan penelitian tersebut yang berfokus pada perspektif forensik dan keamanan, proyek ini lebih menekankan pemanfaatan Rclone sebagai solusi pencadangan, sinkronisasi, dan enkripsi berbasis Google Drive untuk mendukung kebutuhan individu maupun organisasi kecil secara efisien dan aman[25]. Berbeda dengan penelitian tersebut yang berfokus pada perspektif forensik dan keamanan, proyek ini lebih menekankan pemanfaatan Rclone sebagai solusi pencadangan, sinkronisasi, dan enkripsi berbasis Google Drive untuk mendukung kebutuhan individu maupun organisasi kecil secara efisien dan aman, serta dilengkapi dengan sistem validasi, pemantauan kapasitas penyimpanan, dan dashboard yang memudahkan pengguna awam dalam melakukan backup maupun restore.

BAB III
ANALISIS DAN DESAIN

3.1 Analisis

Berdasarkan landasan teori yang telah dipaparkan pada BAB II mengenai cloud storage, arsitektur Rclone, mekanisme komunikasi API dengan Google Drive, algoritma Round-Robin, Shell Script dan yang lainnya, maka pada bab ini akan dilakukan analisis dan perancangan sistem yang mengimplementasikan konsep-konsep tersebut ke dalam aplikasi yang akan dikembangkan. Pemahaman terhadap teori-teori pada bab sebelumnya menjadi pondasi dalam merancang arsitektur dan fungsionalitas sistem.Pada sub bab ini akan dibahas mengenai analisis permasalahan yang menjadi latar belakang pengembangan sistem, solusi yang diusulkan untuk mengatasi permasalahan tersebut, serta analisis kebutuhan sistem yang meliputi kebutuhan perangkat keras dan perangkat lunak.

3.1.1 Analisis Masalah

Pengelolaan data pada layanan cloud storage, masalah yang paling sering muncul adalah kehilangan data akibat kerusakan perangkat, kesalahan pengguna, serangan siber, maupun kegagalan sistem. Meskipun layanan cloud seperti Google Drive dan OneDrive telah menyediakan penyimpanan terdistribusi, pengguna masih menghadapi keterbatasan. Salah satu keterbatasan utama adalah tidak tersedianya fitur *automated backup*, khususnya untuk jenis data yang berbeda, seperti file dokumen biasa dan database.

Sebagian besar pengguna masih mengandalkan backup manual, yang berisiko menimbulkan inkonsistensi data, hingga kehilangan data apabila pengguna lupa melakukan backup secara rutin. Selain itu, proses restore sering kali membutuhkan waktu lama serta tidak selalu menjamin integritas data sepenuhnya. Aspek lain yang juga menjadi perhatian adalah keterbatasan kapasitas penyimpanan gratis pada cloud storage (misalnya 15GB pada Google Drive) yang dapat memperlambat proses backup maupun restore.

Dari sisi fleksibilitas operasional, interaksi langsung melalui terminal dengan baris perintah yang panjang dan kompleks, seperti pada konfigurasi dan eksekusi Rclone, menjadi tantangan tersendiri bagi administrator sistem. Setiap tahapan proses mulai dari pembuatan remote, pengaturan direktori sumber dan tujuan, hingga pelaksanaan sinkronisasi memerlukan sintaks shell yang berbeda dan berpotensi menimbulkan kesalahan apabila dilakukan secara manual. Selain itu, integrasi scripts untuk menyiapkan

serta menutup proses backup sering kali membutuhkan pemanggilan perintah tambahan yang memperpanjang alur kerja. Kondisi ini menurunkan efisiensi operasional dan meningkatkan risiko kesalahan konfigurasi. Oleh karena itu, diperlukan mekanisme otomasi berbasis antarmuka administratif yang mampu mengeksekusi perintah perintah dalam penjadwalan backup, termasuk fitur *prescripts* dan *postscripts*, sehingga administrator dapat mengelola proses backup maupun restore tanpa harus berinteraksi langsung dengan command-line setiap kali melakukan operasi rutin. Hal ini semakin diperburuk dengan kompleksitas penggunaan tools command-line seperti Rclone, yang memang menawarkan fitur backup otomatis, namun membutuhkan konfigurasi manual (penulisan perintah yang panjang) sehingga menyulitkan user admin. Kesalahan kecil dalam konfigurasi Rclone dapat menyebabkan data tidak terbackup dengan benar, atau bahkan terhapus karena salah penggunaan perintah.

Dengan demikian, terdapat gap antara kebutuhan user admin akan solusi backup yang mudah, fleksibel, aman, dan terjadwal otomatis, dengan ketersediaan tools yang ada saat ini. Oleh karena itu, diperlukan sebuah sistem yang mampu mengintegrasikan kemampuan Rclone dengan antarmuka grafis, dukungan backup otomatis untuk folder maupun file, serta mekanisme restore. Selain itu untuk mencegah penumpukan file backup yang berlebihan di cloud, sistem menggunakan algoritma round robin, sehingga hanya menyimpan sejumlah versi backup terakhir. Versi paling lama akan dihapus otomatis ketika ada backup baru.

3.1.2 Analisis Pemecah Masalah

Dalam pengelolaan data pada layanan cloud storage, sering kali pengguna mengalami kesulitan ketika harus melakukan proses backup dan restore secara manual menggunakan perintah terminal yang kompleks. Kondisi ini menjadi tantangan tersendiri bagi individu maupun organisasi yang membutuhkan sistem pencadangan data yang mudah digunakan namun tetap fleksibel dan dapat diandalkan. Oleh karena itu, penelitian ini bertujuan untuk merancang sebuah sistem backup berbasis cloud yang mampu menyederhanakan proses tersebut tanpa mengurangi fleksibilitas alat bantu seperti Rclone.

Langkah utama dalam pemecahan masalah ini adalah dengan menghadirkan antarmuka berbasis web yang memungkinkan pengguna untuk mengelola proses backup dan restore melalui dashboard. Melalui dashboard ini, pengguna dapat melakukan berbagai pengaturan seperti menentukan target backup (file, folder lokal), mengatur lokasi penyimpanan di cloud (Google Drive), memilih mode backup (manual atau otomatis), serta

mengatur waktu pelaksanaannya. Dalam konteks sistem ini, seluruh konfigurasi tersebut dikelola melalui konsep Job yang diposisikan sebagai solusi operasional yang mendefinisikan strategi teknis secara terukur guna mengatasi hambatan redundansi data secara manual. Sebagai unit kerja pusat, Job mengintegrasikan variabel-variabel kritis seperti pemetaan jalur data, otentikasi penyimpanan awan, dan algoritma retensi ke dalam satu alur kerja sistematis yang menjawab tantangan efisiensi sumber daya dan kepastian ketersediaan data. Semua konfigurasi Job ini tersimpan di database lokal, sehingga sistem dapat memanggilnya kembali tanpa perlu melakukan konfigurasi ulang secara manual melalui terminal. Pendekatan ini memanfaatkan konsep cloud storage sebagai layanan penyimpanan berbasis internet yang telah dijelaskan pada subbab 2.1.2, dimana data disimpan pada server Google Drive dan dapat diakses dari berbagai perangkat selama terhubung dengan internet.

Sistem dirancang agar mampu menjalankan proses backup secara otomatis berdasarkan jadwal yang ditentukan pengguna. Penjadwalan dilakukan menggunakan cron sebagai pemicu tetap yang mengimplementasikan konsep cron job pada subbab 2.1.6, sedangkan eksekusi proses dilakukan oleh job runner yang membaca konfigurasi dari database. Dengan pendekatan ini, perubahan konfigurasi yang dilakukan melalui dashboard akan langsung diterapkan pada eksekusi berikutnya tanpa memerlukan pengaturan ulang cron di sistem operasi. Untuk mengatur urutan eksekusi job yang berjalan secara paralel, sistem menggunakan Goroutine yang merupakan fitur bahasa pemograman golang yang telah dipaparkan pada subbab 2.1.8.

Sebagai pengganti mekanisme enkripsi yang biasa digunakan pada sistem backup konvensional, sistem ini mengimplementasikan fitur prescripts dan postscripts sebagai bagian dari alur proses otomatisasi. Prescripts dan postscripts ini diimplementasikan dalam bentuk shell script sesuai konsep pada subbab 2.1.7, yang dijalankan sebelum dan sesudah proses utama untuk mempersiapkan lingkungan kerja, melakukan validasi data, pembersihan sementara terhadap file yang tidak diperlukan atau enkripsi sekalipun tergantung kemauan dari pengguna. Setelah proses utama selesai, postscripts dijalankan untuk melakukan verifikasi hasil backup, memperbarui catatan log, serta menghapus file sementara agar ruang penyimpanan tetap efisien. Penerapan kedua mekanisme ini memastikan setiap tahapan backup berjalan secara konsisten dan dapat dikontrol dengan baik, sejalan dengan prinsip data backup yang telah dijelaskan pada subbab 2.1.1 mengenai pentingnya menjaga konsistensi dan integritas data. Perlu ditegaskan bahwa sistem hanya

melakukan backup pada file/folder yang sudah selesai di proses oleh script, sehingga tidak terjadi kesalahpahaman dimana sistem rclone bisa mengakses file atau melakukan aksi aksi tertentu secara otomatis.

Untuk komunikasi dengan Google Drive, sistem memanfaatkan Rclone sebagai core engine sesuai pembahasan pada subbab 2.1.4. Rclone akan menangani seluruh proses komunikasi API dengan Google Drive termasuk token management, serta operasi upload dan download file. Arsitektur modular Rclone yang terdiri dari User Interface Layer, Core Engine, dan Backend Drivers Layer memungkinkan sistem untuk berkomunikasi dengan Google Drive.

Proses restore juga dirancang dengan pendekatan yang sama, mengimplementasikan konsep restore data pada subbab 2.1.3. Pengguna cukup memilih file atau folder yang telah tersimpan di Google Drive melalui dashboard. Sistem kemudian mengunduh file ke server lokal menggunakan Rclone, menempatkannya ke lokasi tujuan, dan terakhir hasil pemulihan sesuai dengan konfigurasi serta tercatat di sistem log. Mekanisme ini memastikan data yang hilang atau rusak dapat dipulihkan kembali ke kondisi semula dengan cepat dan akurat.

Seluruh aktivitas backup dan restore dicatat secara otomatis pada tabel log yang berisi informasi status proses, waktu eksekusi, hasil keberhasilan atau kegagalan, serta besar data yang di backup. Selain itu, sistem juga dilengkapi dengan fitur monitoring yang berfungsi memantau status koneksi dan kapasitas penyimpanan pada Google Drive. Apabila koneksi terputus selama proses berlangsung, sistem memiliki mekanisme retry otomatis untuk melanjutkan proses tanpa perlu dilakukan pengulangan manual, mengadopsi error handling mechanism dari Rclone yang telah dijelaskan sebelumnya.

Dengan demikian, rancangan sistem yang diusulkan dalam penelitian ini menjadi solusi terintegrasi yang menggabungkan fleksibilitas Rclone dengan kemudahan penggunaan antarmuka berbasis web. Sistem ini mampu mengotomatisasi proses backup menggunakan mekanisme cron job, shell script untuk prescripts dan postscripts, Round Robin Algorithm untuk penjadwalan job, serta melakukan restore file/folder dari cloud source, dan menjaga konsistensi, integritas, dan ketersediaan data. Pendekatan ini tidak hanya meningkatkan efisiensi dan keandalan sistem backup, tetapi juga memberikan kemudahan bagi pengguna dalam mengelola data mereka di lingkungan cloud secara aman.

3.2 Analisis Kebutuhan Sistem

Untuk mewujudkan solusi yang efektif dan efisien dalam pengembangan serta implementasi sistem, diperlukan dukungan perangkat keras dan perangkat lunak yang sesuai dengan kebutuhan setiap tahapan proyek - mulai dari proses pengembangan, pengujian, hingga penyajian dokumentasi berbasis web.

3.2.1 Analisis kebutuhan Perangkat Keras(Hardware)

Perangkat keras adalah komponen-komponen penting untuk kegiatan analisis dan implementasi yang dibutuhkan dalam pengembangan sistem yang akan dibuat untuk pengerjaan Tugas Akhir. Spesifikasi perangkat keras yang diperlukan untuk mendukung implementasi pada sistem yang akan dikembangkan mencakup hal-hal berikut.

Tabel 3. 1 Hardware

Komponen	Spesifikasi Minimal	Keterangan
Processor	Dual Core 2.0 GHz atau lebih	Menjalankan UI dan proses sinkronisasi secara lancar
RAM	4 GB (disarankan 8 GB)	Mendukung multitasking dan eksekusi UI + proses Rclone
Penyimpanan (Storage)	20 GB kosong	Menyimpan sistem operasi, log, dan file sementara sebelum diunggah
Koneksi Internet	Minimal 1 Mbps	Dibutuhkan untuk proses upload/download ke Google Drive secara stabil
Perangkat Input/Output	Keyboard, Mouse, Layar Monitor	Untuk interaksi dengan UI
Platform	PC/Laptop berbasis Linux/Windows	Lingkungan utama untuk pengembangan dan penggunaan UI

3.2.2 Analisis kebutuhan Perangkat Lunak(software)

Analisis kebutuhan perangkat lunak bertujuan untuk mengidentifikasi komponen software yang diperlukan agar sistem dapat berjalan sesuai rancangan. Perangkat lunak yang dipilih harus mampu mendukung proses pengembangan, integrasi dengan Rclone, penyimpanan konfigurasi, serta pengelolaan antarmuka pengguna. Perangkat lunak tambahan seperti Git untuk version control dan modul logging digunakan untuk memudahkan kolaborasi serta memastikan setiap proses backup maupun restore dapat dipantau secara efektif.

Tabel 3. 2 Software

Komponen	Tools yang Digunakan	Keterangan
Sistem Operasi	Kali Linux / Ubuntu (Linux)	Lingkungan utama pengembangan dan eksekusi UI
System engine	Rclone	Upload, download, sinkronisasi file/folder, cek kapasitas & status koneksi
Bahasa Pemrograman	Golang	Pembuatan logika sistem + integrasi shell command Rclone
Framework GUI	Vue.js	Untuk membangun antarmuka grafis pengguna
Database Lokal	MariaDB	Menyimpan konfigurasi user, log, dan jadwal backup
Scheduler	Cron (Library Golang)	Menjalankan backup otomatis sesuai jadwal
Monitoring Storage	Rclone about	Mengecek total kapasitas & sisa penyimpanan cloud
Monitoring Konektivitas	Rclone lsd/ls	Mengecek status koneksi cloud (terhubung atau tidak)
Prescripts & Postscripts	Rclone script module	Menjalankan prescripts sebelum proses utama dan postscripts setelah proses selesai untuk menjaga konsistensi data
Version Control	Git + GitHub	Dokumentasi pengembangan dan kolaborasi kode
Logging & Monitoring	Golang logging module	Mencatat status proses backup, restore, dan error koneksi

3.2.3 Konfigurasi user

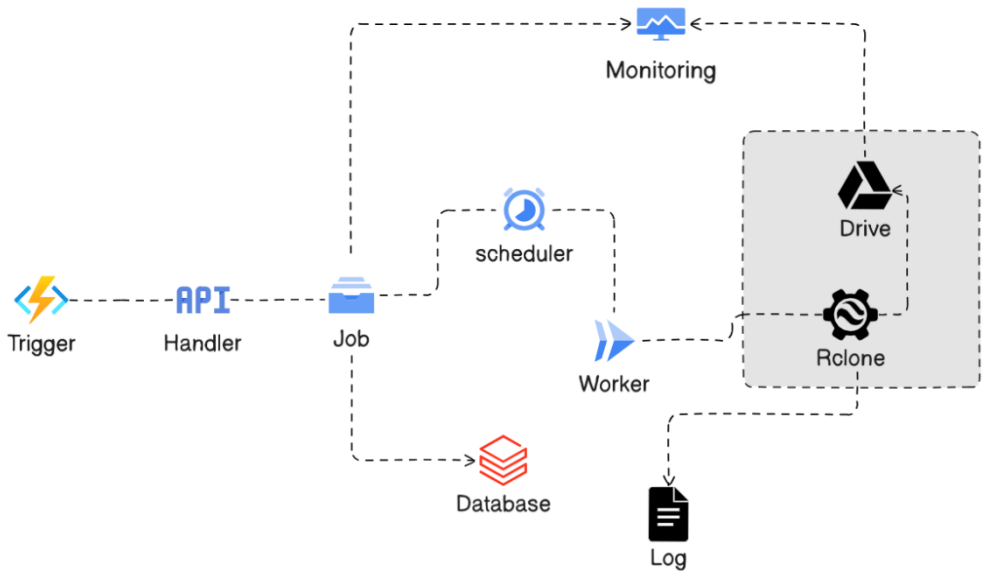
Analisis kebutuhan perangkat lunak bertujuan untuk mengidentifikasi komponen software yang diperlukan agar sistem dapat berjalan sesuai rancangan. Perangkat lunak yang dipilih harus mampu mendukung proses pengembangan, integrasi dengan Rclone, penyimpanan konfigurasi, serta pengelolaan antarmuka pengguna. Komponen utama meliputi sistem operasi Linux (Ubuntu) sebagai lingkungan pengembangan, Rclone sebagai engine utama, bahasa pemrograman serta framework pendukung untuk membangun antarmuka grafis berbasis web, database MySQL/MariaDB untuk penyimpanan konfigurasi dan log, serta cron job sebagai scheduler untuk menjalankan backup otomatis.

Tabel 3. 3 Konfigurasi sistem

Kategori	Field Konfigurasi	Deskripsi
Backup	Job Name	Nama unik job backup
	Path Lokal (jika File/Folder)	Lokasi file/folder yang akan dibackup
	Drive tujuan	Nama remote Rclone (contoh: gdrive:)
	Cloud Path	Folder tujuan di Google Drive
	Operation Mode	Mode operasi backup (Sync dan Copy)
	Schedule Time	Waktu/interval backup (cron) dalam jam/hari
	Prescripts	Skrip atau perintah yang dijalankan **sebelum** proses backup untuk mempersiapkan data atau lingkungan
	Postscripts	Skrip atau perintah yang dijalankan **setelah** proses backup untuk verifikasi atau pembersihan sementara
Restore	File/Folder target	File/DB dari daftar file/folder di Gdrive
	Drive Sumber	Drive lokasi file yang ingin di restore
	Folder lokal	Lokasi file/folder yang ingin di restore

3.3 System Desain

3.3.1 High-Level system Architecture



Gambar 3. 1 High-Level system Architecture

Arsitektur sistem backup berbasis cloud ini dirancang dengan pendekatan modular yang terdiri dari beberapa komponen utama yang saling terintegrasi untuk menjalankan proses backup otomatis ke Google Drive. Sistem dimulai dari komponen Trigger yang berfungsi sebagai pemicu inisiasi proses backup, baik berupa aksi manual dari user melalui dashboard web maupun trigger otomatis berdasarkan jadwal yang telah dikonfigurasi

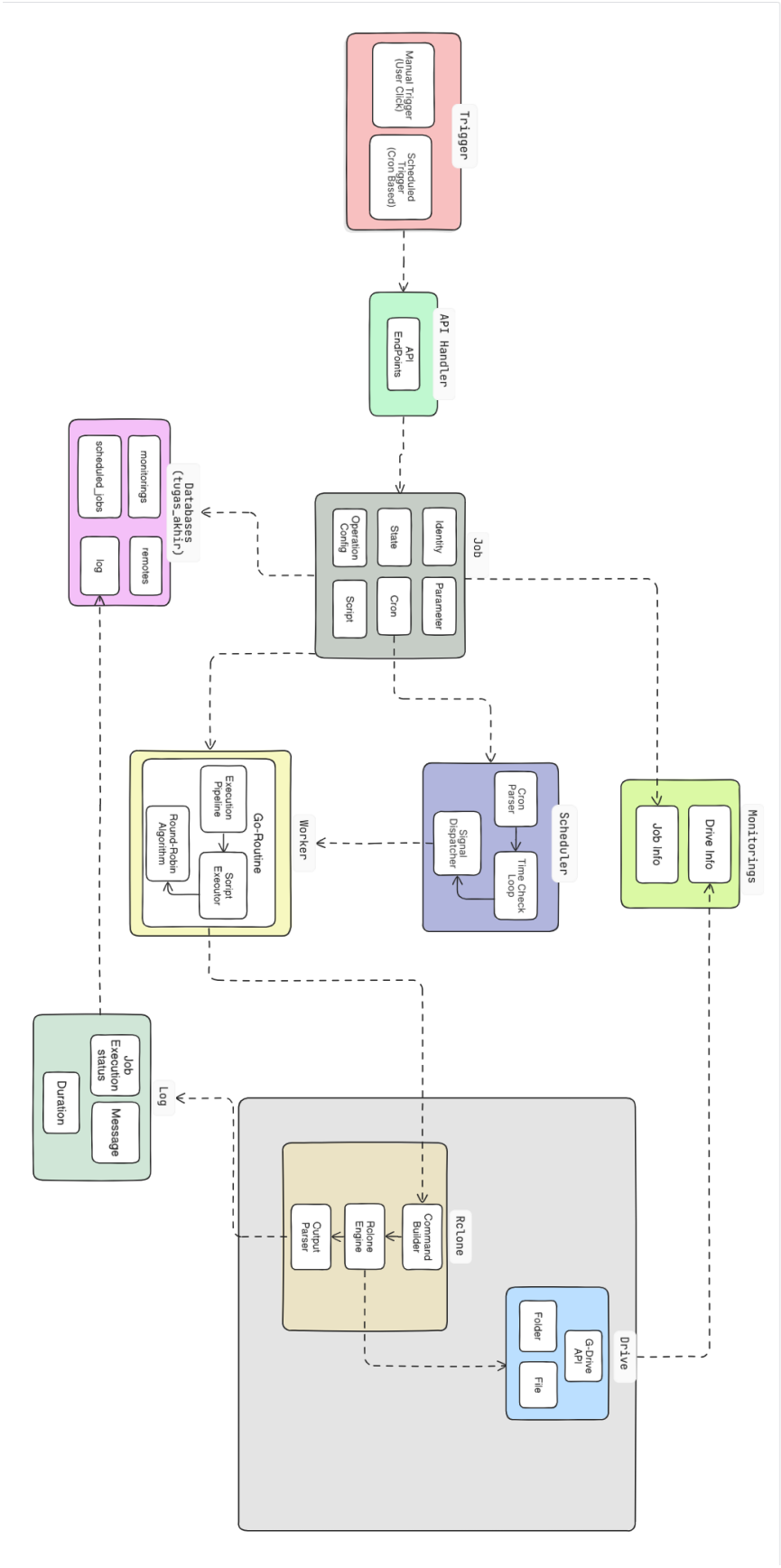
sebelumnya. Trigger berkomunikasi dengan API yang berfungsi sebagai interface layer untuk menerima request dan meneruskannya ke komponen Job. API bertanggung jawab dalam menangani validasi input dan mengatur komunikasi antar komponen dalam sistem.

Komponen Job menyimpan konfigurasi lengkap tentang task backup yang mencakup informasi source data, destination, mode backup, dan waktu eksekusi. Job berkomunikasi dengan Database untuk melakukan operasi penyimpanan dan pengambilan konfigurasi, serta berkomunikasi dengan Scheduler untuk jadwal eksekusi sesuai dengan waktu yang telah ditentukan. Database berfungsi sebagai repository yang menyimpan semua konfigurasi job, penjadwalan, dan metadata sistem yang diperlukan untuk menjalankan backup. Scheduler bertugas mengatur kapan job harus dijalankan dengan membaca informasi dari Database dan memicu Runner pada waktu yang tepat menggunakan mekanisme cron job.

Runner adalah komponen eksekutor yang menjalankan proses backup dengan membaca konfigurasi dari Job dan mengeksekusi Rclone untuk melakukan transfer data ke Google Drive. Rclone berfungsi sebagai engine yang menangani seluruh komunikasi dengan Google Drive, termasuk token, dan operasi upload/download file. Data hasil backup disimpan di Google Drive yang menyediakan layanan penyimpanan cloud yang bisa diakses dari berbagai perangkat. Setiap kali Runner menjalankan proses backup, hasilnya akan dicatat dalam Log yang menyimpan informasi lengkap tentang status eksekusi job, waktu mulai dan selesai, hasil eksekusi, serta error yang terjadi selama proses berlangsung.

Seluruh sistem diawasi oleh komponen Monitoring yang memantau status dan performa dari semua komponen secara. Monitoring mengawasi aktivitas Scheduler dalam penjadwalan job, kinerja Runner dalam menjalankan eksekusi, proses komunikasi Rclone dengan Google Drive, serta koneksi dan kapasitas penyimpanan di Drive. Dengan sistem monitoring, administrator dapat mendeteksi masalah dan menerima alert jika terjadi kegagalan dalam sistem. Alur kerja sistem secara keseluruhan dimulai dari Trigger yang mengirim request ke API, kemudian API berkomunikasi dengan Job untuk menyimpan atau mengupdate konfigurasi di Database, Scheduler membaca konfigurasi dan memicu Runner pada waktu yang ditentukan, Runner mengeksekusi Rclone untuk backup ke Google Drive dan mencatat hasilnya ke Log, sementara Monitoring mengawasi seluruh proses untuk memastikan sistem berjalan dengan baik.

3.3.2 Detailed System Architecture Diagram



Gambar 3. 2 Detailed System Architecture Diagram

Untuk memberikan pemahaman yang lebih detail mengenai implementasi sistem, arsitektur dirinci dalam diagram blok yang menunjukkan komponen-komponen utama beserta subkomponen di dalamnya. Diagram diatas menggambarkan arsitektur sistem backup dan restore berbasis cloud yang dirancang dengan pendekatan modular, dimana setiap komponen memiliki peran spesifik yang saling terintegrasi untuk membentuk alur kerja yang sistematis. Adapun penjelasan detail komponen adalah sebagai berikut.

Trigger merupakan komponen pemicu yang menginisiasi seluruh proses backup dalam sistem. Komponen ini terdiri dari dua jenis mekanisme pemicu yaitu Manual Trigger yang memungkinkan pengguna menjalankan backup secara langsung melalui dashboard dengan mengklik tombol atau interface yang disediakan, dan Scheduled Trigger yang bekerja secara otomatis berdasarkan jadwal yang telah dikonfigurasi menggunakan mekanisme cron job. Kedua jenis trigger ini memberikan fleksibilitas bagi pengguna untuk melakukan backup baik sesuai kebutuhan mendesak maupun secara rutin terjadwal tanpa intervensi manual. Trigger berkomunikasi dengan API Handler untuk mengirimkan request inisiasi proses backup ke sistem.

API Handler berfungsi sebagai gateway yang menerima semua request dari komponen Trigger dan meneruskannya ke komponen Job. Komponen ini menyediakan API Endpoints yang merupakan kumpulan endpoint RESTful untuk berbagai operasi sistem seperti create job, update job configuration, trigger manual backup, query job status, dan logs. API Handler juga bertugas melakukan validasi input untuk memastikan data yang masuk sesuai format dan business rules yang ditetapkan, serta menangani authentication dan authorization untuk memastikan hanya user dengan privilese yang tepat dapat melakukan operasi tertentu.

Job merupakan komponen central yang menyimpan dan mengelola seluruh konfigurasi backup dalam sistem. Job terdiri dari enam subkomponen dengan fungsi spesifik: Identity menyimpan informasi identitas unik seperti job ID dan job name; Parameter berisi konfigurasi teknis seperti source path, destination path, backup mode, compression settings, dan encryption preferences; State mencatat status terkini job apakah running, completed, atau failed; Cron menyimpan ekspresi cron untuk penjadwalan otomatis; Operation Config menentukan jenis operasi Rclone seperti sync atau copy dan Script berisi path untuk prescript dan postscript yang dijalankan sebelum dan sesudah backup utama. Job berkomunikasi dengan Database untuk persistensi data dan dengan

Scheduler untuk registrasi scheduled execution, menjadikannya pusat koordinasi seluruh backup task dalam sistem.

Scheduler bertanggung jawab terhadap pengelolaan eksekusi berbasis waktu agar setiap job dapat dijalankan sesuai dengan jadwal yang telah ditentukan. Komponen ini terdiri atas beberapa subkomponen utama. Antrian waktu berfungsi mengelola daftar job berdasarkan waktu eksekusi menggunakan mekanisme prioritas, sehingga job dengan waktu eksekusi terdekat akan diproses terlebih dahulu.Parser cron bertugas menginterpretasikan ekspresi penjadwalan dari konfigurasi job dan menghitung waktu eksekusi berikutnya. Selanjutnya, pengirim sinyal berfungsi mengirimkan notifikasi eksekusi kepada Worker ketika waktu yang ditentukan telah tercapai. Scheduler bekerja secara berkelanjutan dengan melakukan pemeriksaan berkala terhadap antrian waktu serta menangani kondisi khusus, seperti eksekusi yang terlewat akibat sistem tidak aktif, sesuai dengan kebijakan yang telah ditentukan.

Worker merupakan komponen eksekusi yang menjalankan proses pencadangan secara aktual. Komponen ini mengimplementasikan model konkurensi menggunakan mekanisme goroutine sehingga memungkinkan beberapa job dijalankan secara paralel dengan efisiensi sumber daya yang tinggi. Worker terdiri atas beberapa subkomponen, antara lain alur eksekusi yang mengatur urutan pelaksanaan skrip pra-pencadangan, proses pencadangan utama, dan skrip pasca-pencadangan dengan penanganan kesalahan pada setiap tahapan.

Selain itu, terdapat subkomponen eksekutor skrip yang bertugas menjalankan skrip sistem, menangkap keluaran eksekusi, serta memantau kode keluar proses. Worker juga mengimplementasikan algoritma *round-robin* untuk mengelola distribusi file agar jumlah file dalam suatu direktori tetap terkendali. Worker berkomunikasi dengan komponen Rclone untuk proses transfer data dan dengan komponen Log untuk pencatatan hasil eksekusi.

Rclone berfungsi sebagai mesin inti yang menangani komunikasi dengan layanan penyimpanan Google Drive dan berperan sebagai lapisan abstraksi untuk operasi penyimpanan berbasis awan. Komponen ini terdiri atas pembangun perintah yang menyusun perintah eksekusi beserta parameter yang diperlukan, seperti jalur sumber, tujuan penyimpanan, dan mode transfer data.Mesin Rclone menjalankan perintah tersebut, menangani autentikasi serta pembaruan token secara otomatis, melakukan proses transfer data, dan menerapkan mekanisme pengulangan apabila terjadi kegagalan. Keluaran dari

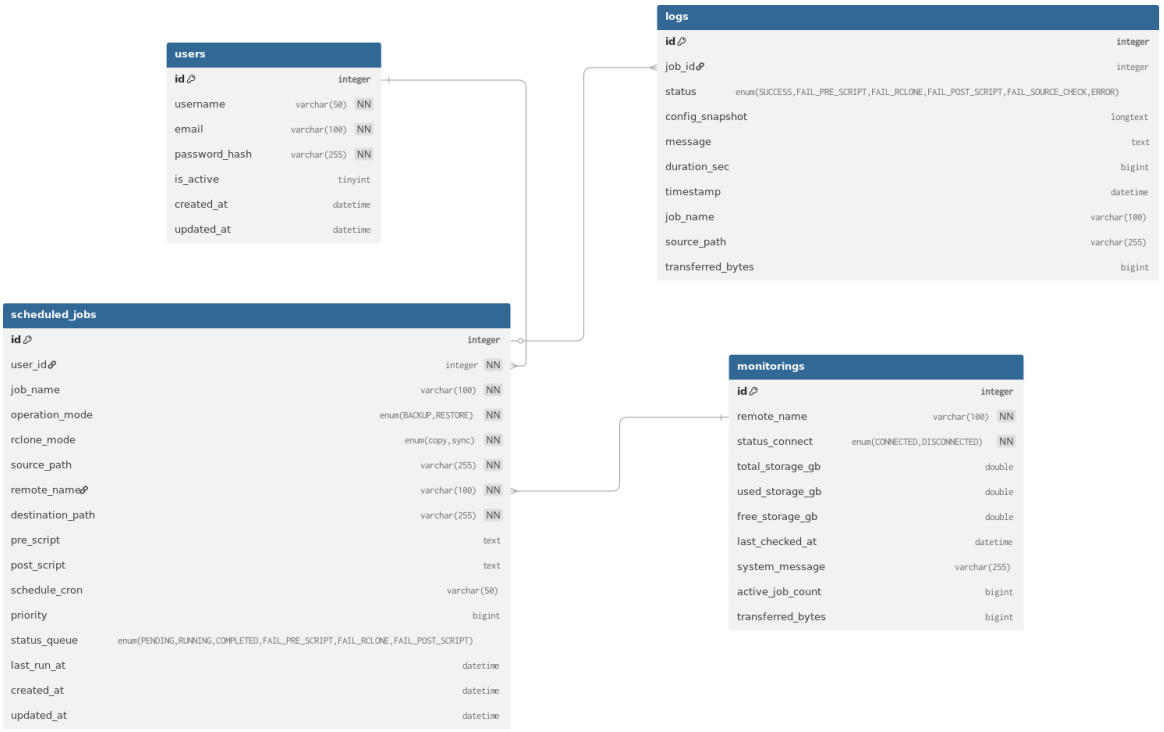
proses ini kemudian dianalisis oleh pengurai keluaran untuk mengekstraksi informasi kemajuan, pesan kesalahan, peringatan, serta status akhir proses. Informasi tersebut selanjutnya diteruskan ke komponen Log dan Monitoring.

Monitoring berperan sebagai lapisan observabilitas sistem yang menyediakan pemantauan kondisi sistem secara waktu nyata. Komponen ini mencakup pemantauan informasi penyimpanan, seperti status koneksi ke Google Drive, kapasitas penyimpanan yang tersedia, penggunaan kuota, serta latensi koneksi. Selain itu, Monitoring juga memantau status seluruh job pencadangan dalam sistem, termasuk job yang sedang berjalan, menunggu antrian, telah selesai, atau mengalami kegagalan.

Log merupakan komponen pencatatan yang menyimpan seluruh aktivitas sistem dalam format tertata untuk keperluan audit dan penelusuran kesalahan. Informasi yang dicatat mencakup status eksekusi job sebagai indikator keberhasilan atau kegagalan, pesan detail yang berisi informasi operasional, peringatan, maupun kesalahan, serta durasi eksekusi yang digunakan untuk analisis kinerja system.

Google Drive berfungsi sebagai media penyimpanan eksternal yang menyediakan infrastruktur penyimpanan yang aman, andal, dan dapat diskalakan. Penyimpanan data diorganisasikan dalam bentuk direktori untuk memudahkan pengelolaan dan penerapan kebijakan retensi data. Data hasil pencadangan disimpan dalam bentuk berkas yang dapat berupa berkas individu, berkas terkompresi, maupun berkas terenkripsi, dengan setiap berkas dilengkapi metadata untuk mendukung proses manajemen dan pemulihan data.

3.3.3 Desain Database



Perancangan basis data untuk sistem ini menggunakan model relasional yang terdiri dari empat entitas utama yang saling terintegrasi, yaitu tabel users, scheduled_jobs, logs, dan monitorings. Struktur ini dirancang untuk memfasilitasi manajemen tugas pencadangan, pemantauan infrastruktur penyimpanan, serta pencatatan riwayat eksekusi.

Entitas pertama, users, berfungsi sebagai pusat manajemen otentikasi dan otorisasi. Tabel ini menyimpan kredensial administrator yang terdiri dari *username*, alamat *email*, dan *password* yang telah di-*hash* demi keamanan. Setiap pengguna dalam tabel ini memiliki relasi *one-to-many* terhadap tabel scheduled_jobs, yang berarti satu administrator dapat membuat dan mengelola banyak konfigurasi pekerjaan sekaligus.

Tabel scheduled_jobs bertindak sebagai komponen inti dari sistem yang menyimpan seluruh konfigurasi teknis untuk tugas *backup* maupun *restore*. Tabel ini mencakup atribut detail seperti jalur sumber data (*source_path*), jalur tujuan (*destination_path*), dan nama penyimpanan awan (*remote_name*). Fleksibilitas sistem tercermin pada kolom operation_mode yang menentukan jenis tugas (*BACKUP* atau *RESTORE*), rclone_mode untuk metode transfer, serta schedule_cron yang menyimpan ekspresi waktu untuk penjadwalan otomatis. Selain itu, tabel ini mendukung fitur otomasi lanjutan melalui kolom pre_script dan post_script untuk eksekusi perintah *shell* kustom,

serta memiliki status antrian (*status_queue*) untuk memantau siklus hidup eksekusi pekerjaan.

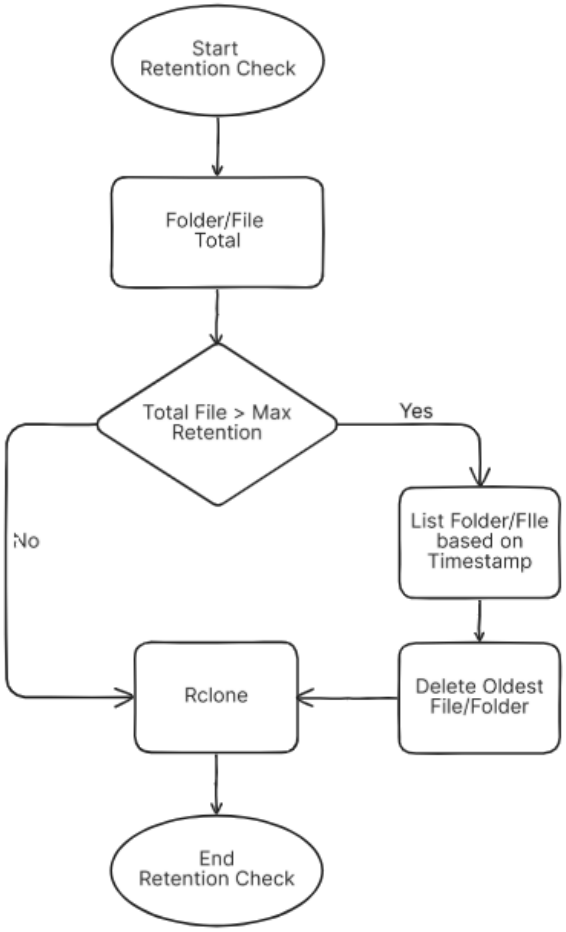
Untuk keperluan audit dan pemantauan kinerja, sistem menggunakan tabel logs yang merekam riwayat eksekusi dari setiap pekerjaan. Tabel ini memiliki relasi dengan scheduled_jobs melalui *Foreign Key* job_id. Setiap baris data dalam tabel ini menyimpan informasi kritis pasca-eksekusi, termasuk status akhir (seperti *SUCCESS* atau berbagai jenis *FAIL*), pesan kesalahan atau output sistem, durasi eksekusi (*duration_sec*), serta total data yang ditransfer (*transferred_bytes*). Fitur penting lainnya adalah kolom config_snapshot yang menyimpan salinan konfigurasi saat pekerjaan dijalankan dalam format JSON, memungkinkan penelusuran konfigurasi historis jika terjadi perubahan pada pekerjaan induk.

Terakhir, tabel monitorings didedikasikan untuk memantau kesehatan dan kapasitas penyimpanan *remote storage* (seperti Google Drive). Tabel ini menyimpan metrik *real-time* yang diambil dari Rclone, meliputi status konektivitas (*status_connect*), total kapasitas penyimpanan, serta ruang yang terpakai dan tersedia (*used/free storage*). Tabel ini terhubung secara logis dengan tabel scheduled_jobs melalui atribut remote_name, memastikan bahwa setiap pekerjaan pencadangan diarahkan ke penyimpanan yang status dan kapasitasnya terpantau dengan baik.

3.3.4 Flowchart

Flowchart digunakan untuk menggambarkan alur proses yang terjadi di dalam sistem supaya lebih mudah dipahami. Melalui flowchart, setiap tahapan mulai dari input pengguna, validasi sistem, hingga proses backup maupun restore dapat divisualisasikan dalam bentuk diagram alir. Dengan adanya representasi ini, alur kerja sistem menjadi lebih jelas, sehingga memudahkan dalam tahap implementasi maupun evaluasi. Flowchart pada sistem ini dibagi menjadi tiga bagian utama, yaitu flow konfigurasi, flow backup, dan flow restore.

3.3.3.1 Retention Flow



Gambar 3. 3 Retention Flow

Flowchart ini menggambarkan alur proses retention check yang berfungsi untuk mengelola dan membersihkan file atau folder backup lama yang melebihi batas maksimal retention, menjaga kapasitas penyimpanan tetap efisien dan mencegah penumpukan data backup yang tidak diperlukan.

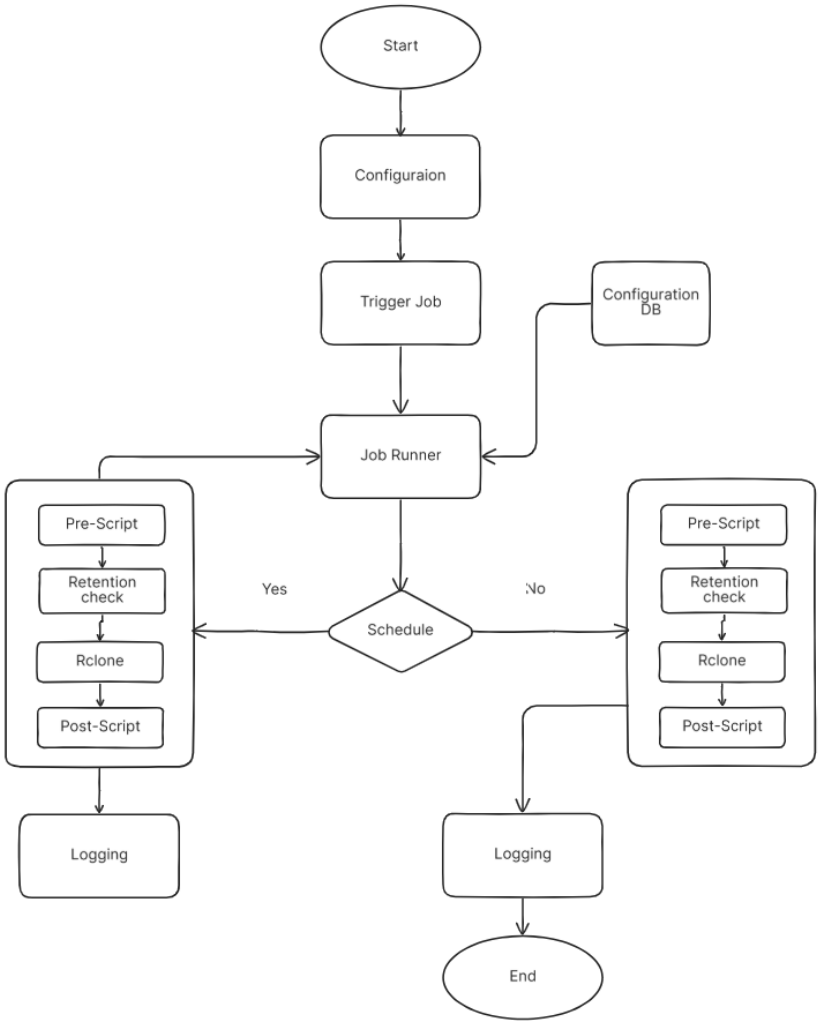
Proses dimulai dari Start Retention Check dan melanjutkan ke tahap Folder/File Total untuk menghitung total jumlah folder atau file backup yang tersimpan di Google Drive. Perhitungan ini mencakup semua folder atau file dalam path backup yang dikonfigurasi, memberikan informasi lengkap tentang jumlah backup yang ada saat ini. Setelah mendapatkan jumlah total, sistem melakukan decision point pada Total File > Max Retention untuk memeriksa apakah jumlah backup melebihi batas maksimal yang ditetapkan dalam konfigurasi, baik berupa jumlah file tertentu maupun berdasarkan periode waktu tertentu.

Jika kondisi bernilai No, artinya jumlah backup masih dalam batas yang diizinkan, sistem langsung melanjutkan ke proses Rclone tanpa melakukan penghapusan. Ini menunjukkan kapasitas penyimpanan masih mencukupi dan tidak perlu cleanup terhadap backup lama. Sebaliknya, jika kondisi bernilai Yes, sistem perlu melakukan cleanup karena jumlah backup telah melebihi batas. Sistem melanjutkan ke List Folder/File based on Timestamp untuk mengambil daftar lengkap semua backup beserta timestamp yang menunjukkan kapan setiap backup dibuat. Daftar ini diurutkan dari yang paling lama ke yang paling baru untuk menentukan backup mana yang harus dihapus terlebih dahulu.

Setelah mendapat daftar terurut, sistem melanjutkan ke Delete Oldest File/Folder untuk menghapus backup paling lama menggunakan Rclone command delete atau purge. Penghapusan dilakukan secara berurutan sampai total backup kembali dalam batas max retention. Misalnya jika max retention 10 backup dan saat ini ada 15, sistem akan menghapus 5 backup terlama sehingga tersisa 10 backup terbaru. Setelah cleanup selesai, kedua alur (No dan Yes) bertemu di tahap Rclone dimana engine siap menjalankan operasi backup atau restore dengan ruang penyimpanan yang sudah optimal.

Proses berakhir pada End Retention Check yang menandakan mekanisme retention check selesai dijalankan. Sistem telah memastikan jumlah backup berada dalam batas yang diizinkan dan siap melanjutkan operasi berikutnya. Flowchart ini menunjukkan retention check sebagai proses otomatis yang berjalan sebelum setiap operasi backup atau restore untuk menjaga kebersihan dan efisiensi penyimpanan, mengimplementasikan prinsip data lifecycle management dimana data lama yang tidak diperlukan dihapus secara sistematis untuk memberi ruang bagi data baru yang lebih relevan.

3.3.3.2 Backup Flow



Gambar 3. 4 Backup Flow

Flowchart ini menggambarkan alur lengkap proses backup yang dapat dijalankan baik secara terjadwal (scheduled) maupun manual, dengan mekanisme Pre-Script, Retention Check, dan Post-Script untuk memastikan backup berjalan optimal dan terkelola dengan baik.

Proses dimulai dari Start dan melanjutkan ke tahap Configuration dimana sistem membaca konfigurasi job yang berisi informasi lengkap seperti source path, destination path, backup mode, schedule settings, dan script configurations. Setelah konfigurasi dimuat, sistem melanjutkan ke Trigger Job yang berfungsi menginisiasi eksekusi job backup, baik dipicu oleh user secara manual melalui dashboard maupun oleh scheduler secara otomatis

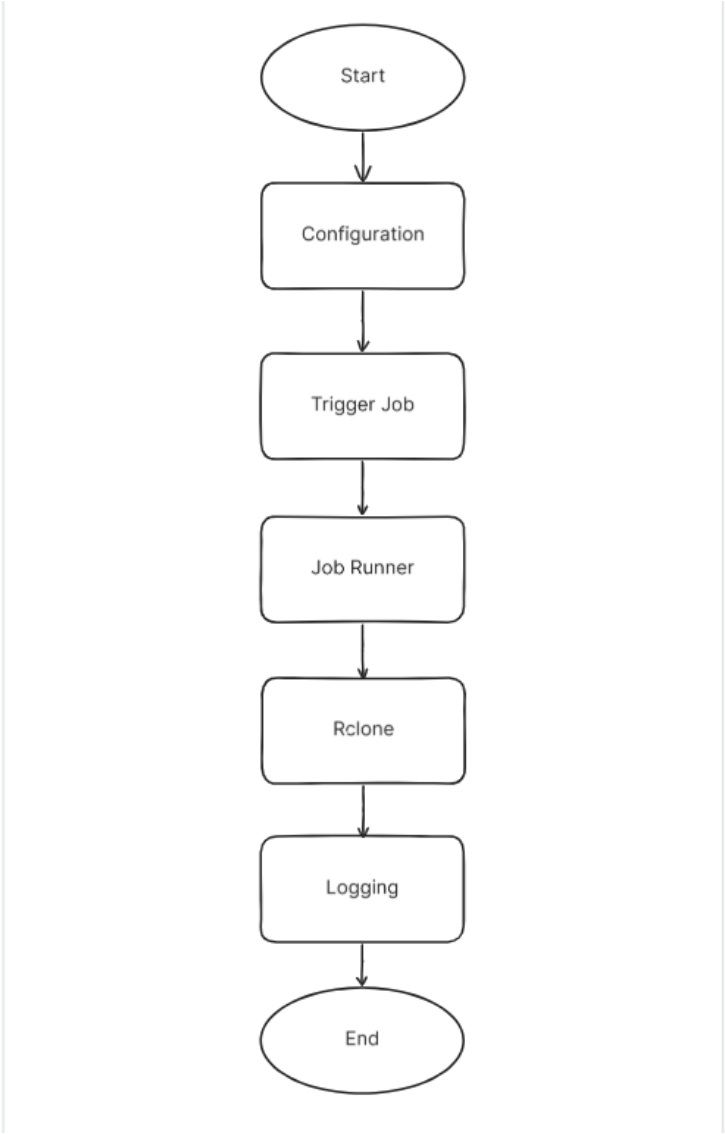
berdasarkan waktu yang dikonfigurasi. Trigger Job berkomunikasi dengan Configuration DB untuk mengambil atau memvalidasi konfigurasi yang tersimpan di database, memastikan semua parameter job sudah benar sebelum eksekusi dimulai.

Setelah trigger berhasil, alur berlanjut ke Job Runner yang bertugas mengeksekusi job sesuai konfigurasi. Job Runner juga tetap terhubung dengan Configuration DB untuk membaca parameter eksekusi yang diperlukan selama proses berjalan. Dari Job Runner, sistem melakukan decision point pada Schedule untuk memeriksa apakah job ini merupakan scheduled job atau manual job. Pemeriksaan ini penting karena menentukan execution pipeline yang akan dijalankan.

Jika kondisi Yes (scheduled job), sistem menjalankan execution pipeline lengkap di sisi kiri yang dimulai dari Pre-Script untuk menjalankan shell script persiapan seperti database dump, file compression, atau cleanup temporary files. Setelah Pre-Script selesai, sistem melanjutkan ke Retention Check yang memeriksa apakah jumlah backup di Google Drive melebihi batas maksimal retention dan menghapus backup terlama jika diperlukan untuk menjaga efisiensi storage. Kemudian proses berlanjut ke Rclone untuk menjalankan operasi backup aktual dengan upload data ke Google Drive menggunakan Rclone engine. Setelah Rclone selesai, sistem menjalankan Post-Script untuk melakukan verifikasi hasil backup, cleanup files, atau send notifications. Pipeline ini berakhir di Logging yang mencatat semua detail eksekusi termasuk status, duration, files processed, dan error messages jika ada.

Jika kondisi No (manual job), sistem menjalankan execution pipeline di sisi kanan yang memiliki tahapan identik yaitu Pre-Script, Retention Check, Rclone, dan Post-Script, namun berjalan sebagai response terhadap manual trigger. Pipeline manual ini juga berakhir di Logging untuk recording hasil eksekusi. Kedua logging path dari scheduled dan manual job kemudian merge dan melanjutkan ke End yang menandakan proses backup telah selesai. Flowchart ini menunjukkan bahwa sistem backup dirancang dengan fleksibilitas tinggi yang mendukung dua mode eksekusi dengan pipeline, serta mengintegrasikan mekanisme Pre-Script dan Post-Script untuk customization, Retention Check untuk storage management logging untuk audit trail dan troubleshooting.

3.3.3.4 Restore Flow



Gambar 3. 5 Restore Flow

Flowchart ini menggambarkan alur proses restore yang lebih sederhana dan straightforward dibandingkan proses backup, dimana data yang tersimpan di Google Drive akan dipulihkan kembali ke lokasi tujuan yang ditentukan.

Proses dimulai dari Start dan melanjutkan ke tahap Configuration dimana sistem membaca konfigurasi restore job yang berisi informasi seperti file atau folder yang akan di-restore dari Google Drive, destination path di lokal atau server dimana file akan dipulihkan,. Konfigurasi ini penting untuk memastikan sistem mengetahui dengan tepat apa yang harus di-restore dan kemana file akan ditempatkan setelah proses download selesai.

Setelah konfigurasi dimuat, sistem melanjutkan ke Trigger Job yang menginisiasi eksekusi restore job. Berbeda dengan backup yang dapat berjalan secara scheduled atau manual, restore biasanya hanya dipicu secara manual oleh user ketika dibutuhkan untuk recovery data. User memilih file atau folder yang ingin di-restore melalui dashboard, kemudian Trigger Job akan memulai proses restore berdasarkan pilihan tersebut.

Alur berlanjut ke Job Runner yang bertugas mengeksekusi restore job sesuai konfigurasi. Job Runner akan mengkoordinasikan seluruh tahapan restore untuk memastikan proses berjalan dengan benar dari awal hingga akhir. Dari Job Runner, sistem melanjutkan ke Rclone yang merupakan core engine untuk melakukan operasi download file dari Google Drive ke destination yang ditentukan. Rclone akan menggunakan credentials yang tersimpan untuk autentikasi dengan Google Drive, kemudian melakukan download operation dengan support untuk resumable transfer jika file berukuran besar atau koneksi terputus di tengah proses.

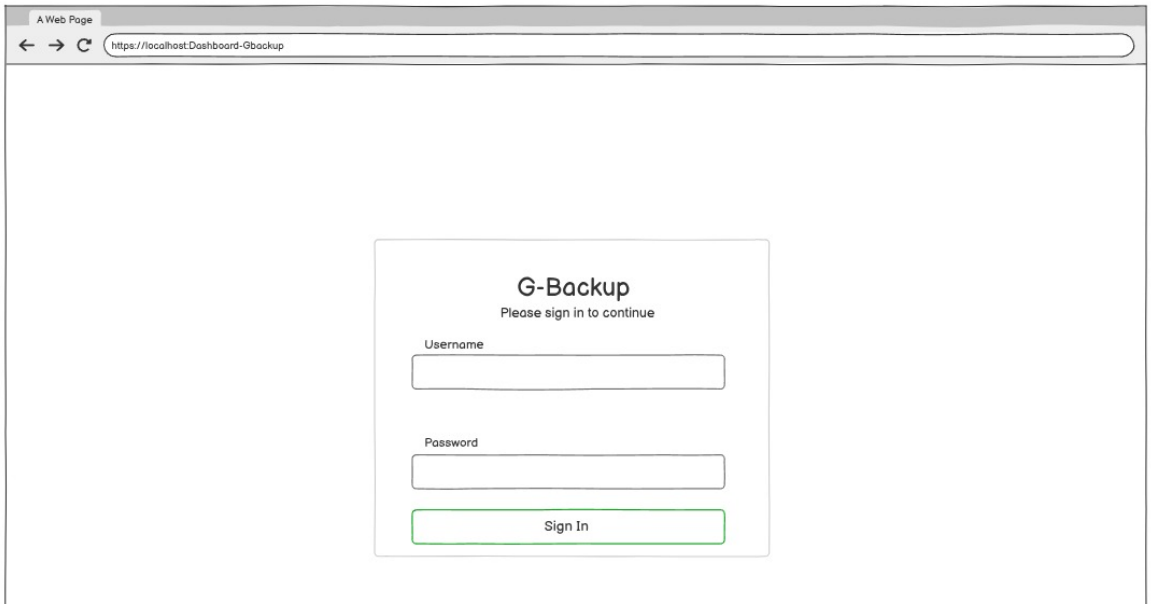
Setelah Rclone selesai mendownload dan menempatkan file ke destination path, sistem melanjutkan ke Logging yang mencatat hasil restore termasuk informasi seperti file yang di-restore, size, destination path, timestamp, duration, status (success atau failed), dan error messages jika terjadi kegagalan. Logging ini penting untuk audit trail dan memastikan setiap operasi restore tercatat dengan baik. Proses berakhir pada End yang menandakan restore job telah selesai dan data telah berhasil dipulihkan ke lokasi tujuan.

Flowchart restore ini menunjukkan proses yang lebih linear dan tidak memerlukan Pre-Script, Post-Script, atau Retention Check seperti pada backup, karena restore bersifat on-demand recovery yang fokus pada kecepatan dan kesederhanaan dalam mengembalikan data yang dibutuhkan user.

3.4 Desain Antarmuka

Tahap desain merupakan langkah penting dalam merancang sistem backup berbasis cloud agar sesuai dengan kebutuhan pengguna sekaligus mampu mengatasi permasalahan yang telah dianalisis sebelumnya. Desain ini bertujuan untuk memberikan gambaran visual mengenai bagaimana sistem akan digunakan, bagaimana informasi ditampilkan, serta bagaimana pengguna dapat melakukan konfigurasi, backup, maupun restore data secara lebih mudah. Dengan adanya rancangan desain, diharapkan implementasi sistem dapat lebih terarah, konsisten, dan sesuai dengan tujuan penelitian.

3.4.1 Desain Tampilan Login Page

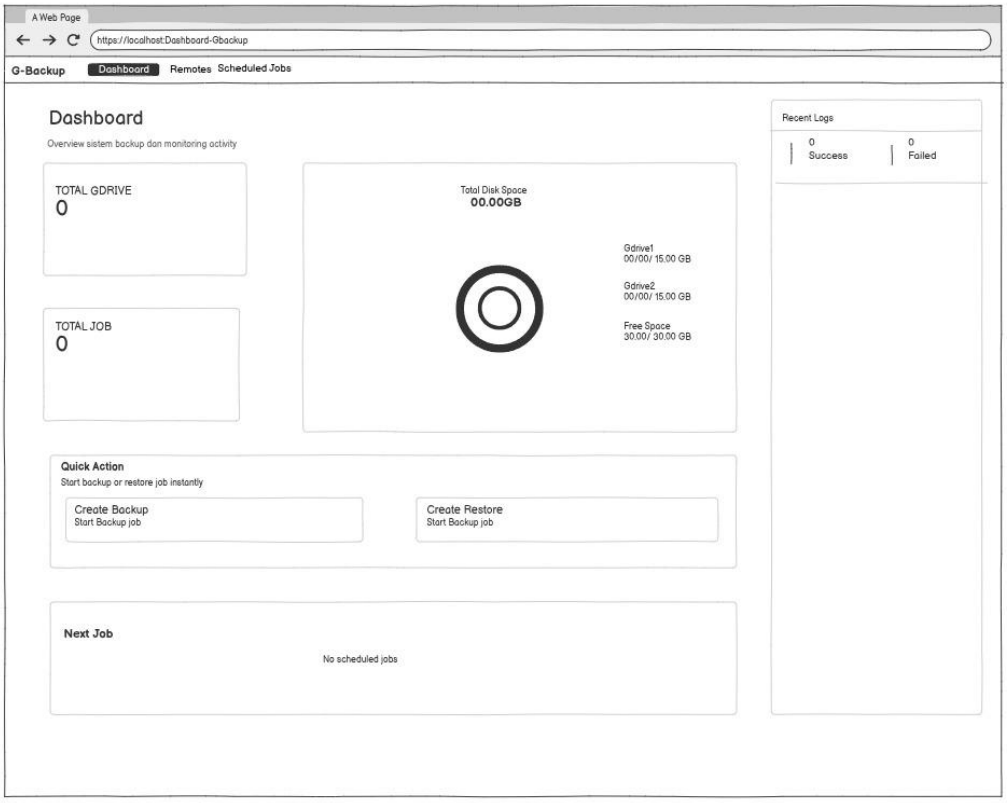


Gambar 3. 6 Desain Login

Halaman login merupakan gerbang utama bagi pengguna untuk dapat mengakses sistem G-Backup. Antarmuka ini dirancang dengan prinsip kesederhanaan dan fokus pada fungsi autentikasi, sehingga pengguna dapat masuk dengan cepat tanpa distraksi visual. Pada bagian tengah halaman, ditampilkan sebuah kartu (login card) dengan judul "G-Backup" yang berfungsi sebagai identitas sistem. Tepat di bawahnya terdapat teks tambahan "Please sign in to continue" yang memberikan konteks bahwa autentikasi diperlukan sebelum pengguna dapat menggunakan fitur dashboard.

Formulir login terdiri dari dua elemen input: Username dan Password, yang disusun secara vertikal untuk memudahkan pengguna dalam melakukan pengisian data secara berurutan. Di bagian bawah formulir disediakan tombol "Sign In", yang berfungsi untuk memproses autentikasi ke server backend. Setelah menekan tombol ini, sistem akan melakukan validasi kredensial dan mengeluarkan token otentikasi jika valid. Tata letak elemen dibuat simetris dan terpusat untuk memastikan kenyamanan visual serta meningkatkan fokus pengguna.

3.4.2 Desain Tampilan Dashboard

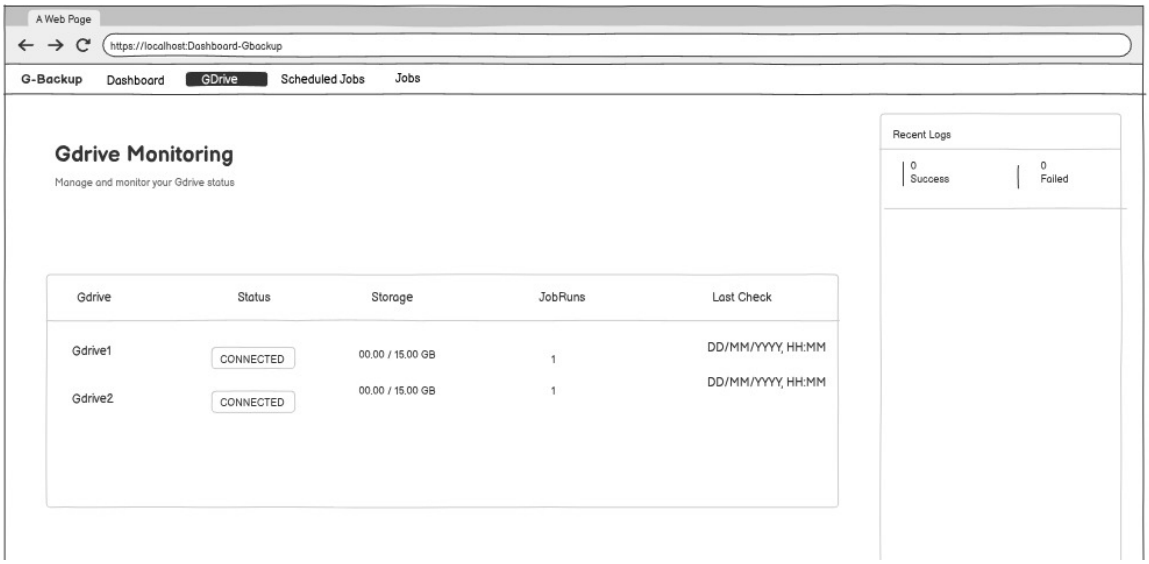


Gambar 3. 7 Desain Tampilan Dashboard

Dashboard merupakan halaman utama yang ditampilkan setelah pengguna berhasil login. Halaman ini berfungsi sebagai pusat pemantauan sistem backup, dengan menampilkan ringkasan kondisi penyimpanan, jumlah job yang terdaftar, serta riwayat proses terbaru.Pada bagian atas terdapat menu navigasi seperti Dashboard, Remotes, dan Scheduled Jobs sebagai akses cepat ke fungsi utama sistem. Dua kartu informasi, yaitu Total GDrive dan Total Job, memberikan gambaran singkat mengenai jumlah remote serta jumlah pekerjaan backup/restore yang telah dikonfigurasi.Bagian tengah menampilkan visualisasi penggunaan ruang penyimpanan Google Drive dalam bentuk diagram lingkaran. Informasi ini mencakup kapasitas total, penggunaan masing-masing akun GDrive, serta ruang kosong yang tersedia, sehingga pengguna dapat memantau kondisi storage secara mudah.Di bawahnya terdapat area Quick Action, yang menyediakan tombol Create Backup dan Create Restore untuk memulai pembuatan job secara cepat. Panel Next Job kemudian menampilkan jadwal eksekusi job otomatis berikutnya yang telah terdaftar di sistem.Pada sisi kanan, panel Recent Logs menampilkan jumlah proses yang berhasil maupun gagal, sehingga pengguna dapat memantau status eksekusi terbaru secara langsung.Secara keseluruhan, dashboard dirancang untuk menyajikan informasi inti secara ringkas dan

memungkinkan pengguna mengambil tindakan dengan cepat, sekaligus memantau seluruh aktivitas backup dan restore dalam satu tampilan.

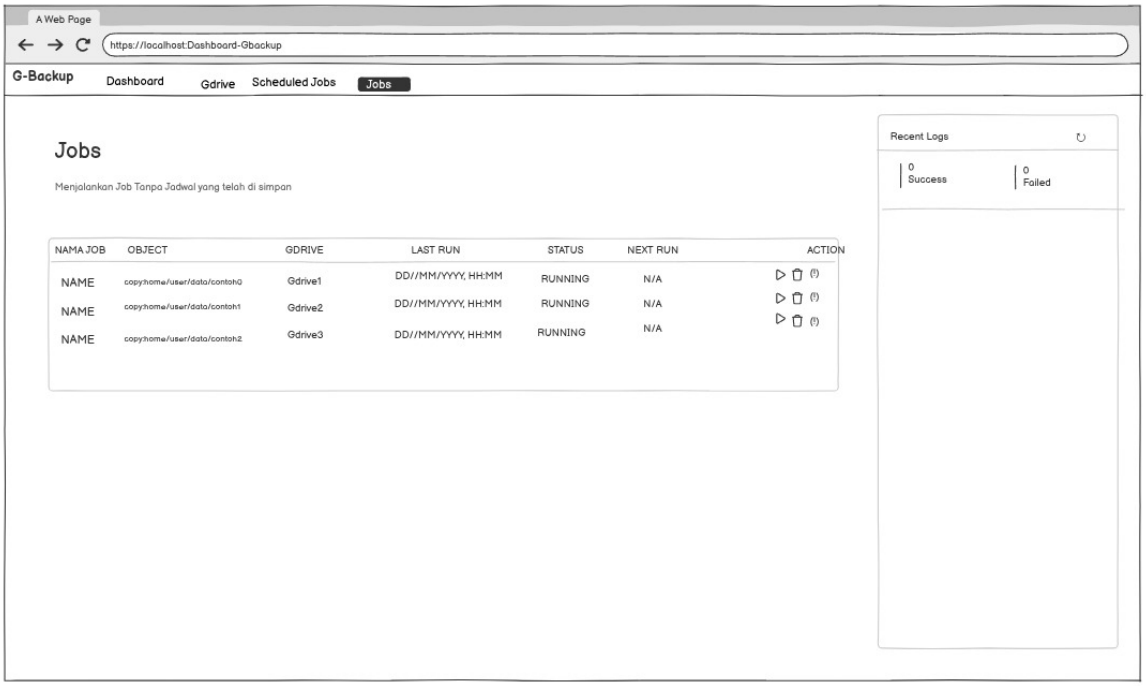
3.4.3 Desain Tampilan Isi Remote Gdrive



Gambar 3. 8 GDrive Remote

Halaman Remote Monitoring berfungsi sebagai pusat kendali untuk memantau kesehatan dan kapasitas *Remote* Google Drive (GDrive) yang terdaftar di sistem G-Backup. Tujuan utamanya adalah untuk memberikan informasi proaktif kepada administrator mengenai *status* koneksi dan penggunaan *storage* eksternal. Tabel utama menampilkan nama-nama *Remote* (contohnya, Gdrive1) beserta metrik kritis yang ditarik oleh *Monitoring Service* Golang. Metrik tersebut mencakup Status koneksi (*Connected* atau *Disconnected*), Storage yang menunjukkan penggunaan kapasitas secara visual dan persentase (misalnya, 7.17 of 15 Gb (47%) Used), jumlah Job Runs yang bergantung pada *Remote* tersebut (misalnya, 4 Job), dan *timestamp* Last Checked untuk menunjukkan seberapa baru data tersebut (misalnya, 07:19 PM).

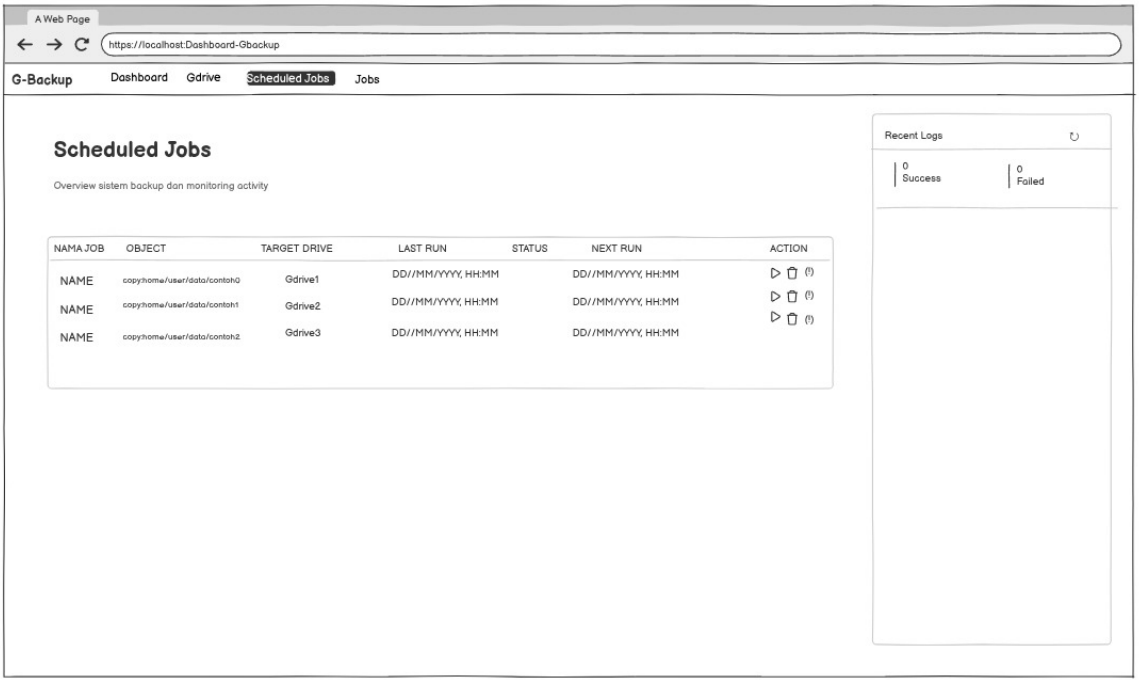
3.4.4 Desain Tampilan Halaman Job



Gambar 3. 9 Tampilan halaman jobs

Halaman Jobs berfungsi sebagai pusat pemantauan dan pengelolaan antrean (Job Monitoring & Queue Management) yang memungkinkan administrator melihat seluruh aktivitas backup dan restore yang terjadwal. Tampilan tabel pada halaman ini menjadi representasi langsung dari logika *Scheduler Service* di backend, karena seluruh informasi seperti waktu eksekusi dan status diperbarui secara dinamis. Setiap baris job menampilkan deskripsi lengkap yang mencakup nama job, serta target GDrive yang digunakan. Informasi waktu seperti *Last Run* dan *Next Run* dihitung secara runtime oleh Scheduler, memberikan gambaran akurat mengenai riwayat dan eksekusi berikutnya. Status job ditampilkan berdasarkan hasil eksekusi terakhir yang diambil dari Log Repository, sehingga dapat digunakan sebagai bahan audit dan evaluasi. Selain itu, pengguna dapat berinteraksi langsung melalui kolom *Action* yang menyediakan tombol untuk menjalankan job secara instan (*Run Now*), menghapus job, atau membuka konfigurasi job untuk melihat detail Pre-Script, command rclone, maupun Post-Script yang digunakan.

3.4.5 Desain Tampilan Halaman scheduled Jobs



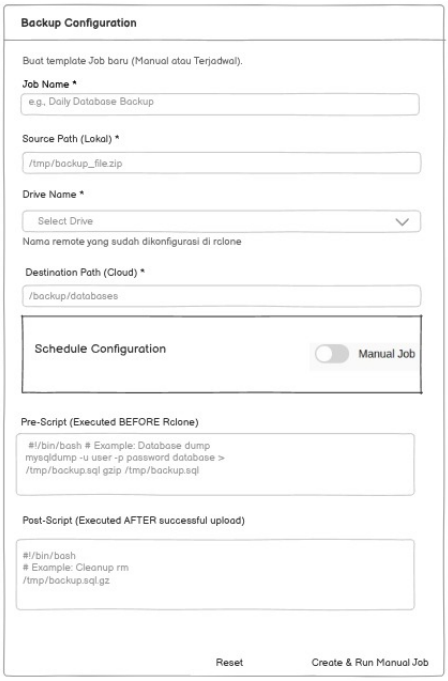
Gambar 3. 10 Halaman Scheduled Jobs

Halaman Scheduled Jobs berfungsi sebagai pusat monitoring utama untuk seluruh job yang diatur berjalan secara otomatis melalui mekanisme CRON, memberikan pengguna gambaran real-time mengenai status eksekusi dan prediksi waktu pemicu berikutnya. Informasi yang ditampilkan divalidasi langsung oleh Service Layer Golang, sehingga setiap perubahan status mencerminkan kondisi sistem yang sebenarnya. Pada tabel utama, setiap job ditampilkan dengan deskripsi lengkap seperti nama job, objek yang ditransfer serta target GDrive yang digunakan, sehingga pengguna dapat memahami sumber dan tujuan proses secara cepat. Bagian penting lainnya adalah informasi waktu, yaitu *Last Run* yang mencatat eksekusi terakhir dan *Next Run* yang dihitung secara runtime oleh SchedulerService berdasarkan string CRON yang terdaftar. Status job ditampilkan untuk menunjukkan apakah proses sebelumnya berhasil atau gagal, data yang diambil dari Log Repository untuk mendukung kebutuhan audit. Selain itu, kolom *Action* menyediakan kontrol interaktif yang memungkinkan pengguna menjalankan job secara manual melalui tombol *Run Now* atau melihat detail konfigurasi seperti Pre-Script, command rclone, dan Post-Script melalui tombol Edit/View Script.

3.4.6 Desain Configuration form

Pada sub bab ini dibahas rancangan form konfigurasi yang digunakan sebagai antarmuka utama bagi pengguna dalam mengatur proses backup dan restore. Form ini berfungsi untuk menyederhanakan interaksi pengguna dengan sistem, di mana seluruh parameter yang diperlukan untuk menjalankan backup maupun restore dapat diatur melalui dashboard web tanpa perlu mengetikkan perintah secara manual. Dengan adanya form konfigurasi, pengguna dapat dengan mudah menentukan target yang akan dibackup atau direstore, tujuan penyimpanan di cloud, jadwal eksekusi. Desain form ini menjadi dasar penting dalam implementasi sistem karena konfigurasi yang dimasukkan akan disimpan di database dan dibaca oleh *executor* saat proses backup maupun restore dijalankan.

1. Backup Configuration form



Gambar 3. 11 Backup Config Form

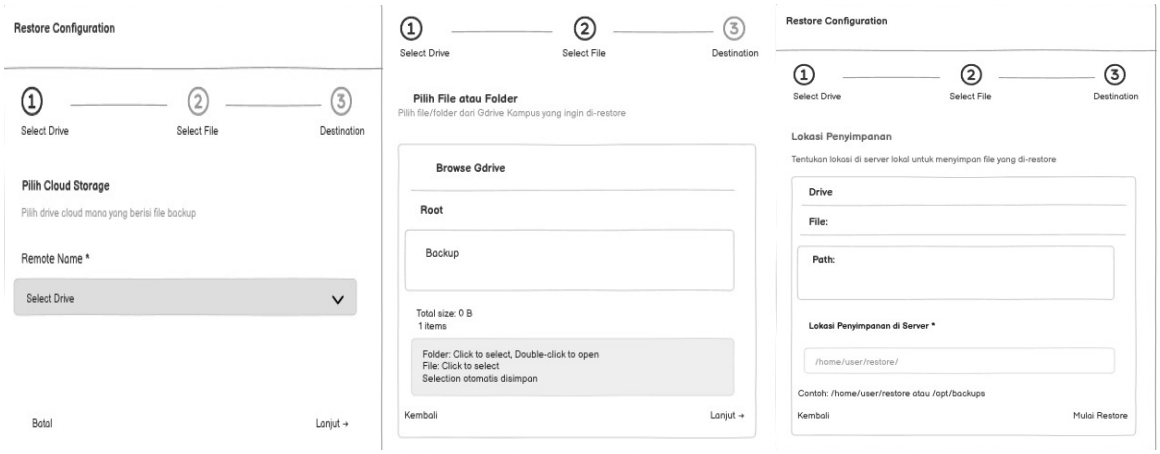
Implementasi antarmuka konfigurasi backup, sebagaimana ditampilkan pada gambar di atas, direalisasikan dalam bentuk formulir modal (*modal form*) yang berfungsi sebagai gerbang utama bagi administrator untuk mendefinisikan parameter tugas pencadangan. Antarmuka ini dirancang untuk memfasilitasi input data yang presisi, dimulai dengan pengisian "Job Name" sebagai identitas unik tugas dan "Source Path" yang mewajibkan input jalur absolut (*absolute path*) ke direktori atau file sumber di server lokal.

Validasi jalur ini sangat krusial untuk memastikan sistem *backend* dapat mengakses target data secara langsung tanpa ambiguitas direktori kerja.

Selanjutnya, untuk menentukan tujuan penyimpanan, sistem mengintegrasikan pemilihan "Drive Name" yang bersifat dinamis. Daftar penyimpanan yang muncul pada menu *dropdown* diambil secara *real-time* dari konfigurasi Rclone di server melalui layanan API, memastikan bahwa pengguna hanya dapat memilih *remote storage* yang telah terautentikasi dengan valid. Pengguna kemudian menentukan "Destination Path" untuk menetapkan struktur direktori penyimpanan di layanan *cloud*. Fleksibilitas operasional diatur melalui komponen *toggle switch* pada bagian "Schedule Configuration". Dalam kondisi *default* atau mode manual, sistem akan memperlakukan konfigurasi ini sebagai tugas insidental yang dieksekusi segera (*immediate execution*), sedangkan jika diaktifkan, antarmuka akan menyediakan opsi penjadwalan berbasis waktu (*Cron Expression*).

Fitur unggulan pada implementasi ini adalah adanya mekanisme *hook* otomatisasi melalui "Pre-Script" dan "Post-Script". Kolom "Pre-Script" memungkinkan administrator menyisipkan perintah *shell* (seperti mysqldump dan kompresi gzip) yang akan dieksekusi sistem sebelum proses transfer data dimulai, memfasilitasi pencadangan *database* yang konsisten. Sebaliknya, "Post-Script" digunakan untuk operasi pembersihan, seperti menghapus file *dump* sementara setelah transfer dinyatakan sukses (*SUCCESS*). Saat tombol eksekusi ditekan, seluruh konfigurasi divalidasi dan dikirim ke *endpoint* API /api/v1/jobs/new. Pada sisi *backend*, data disimpan ke dalam *database* MariaDB, dan khusus untuk tugas manual, layanan *BackupService* akan memicu proses di latar belakang menggunakan *goroutine*, sehingga proses pencadangan berjalan secara asinkron tanpa memblokir antarmuka pengguna.

2. Restore Configuration form



Gambar 3. 12 Restore Config Form

Implementasi antarmuka konfigurasi pemulihan data (*restore*) yang memandu pengguna melalui tiga tahapan (Select Drive, Select File, dan Destination) untuk memastikan akurasi pemulihan data. Proses ini dimulai pada tahap pertama, "Select Drive", di mana sistem menyajikan elemen *dropdown* yang secara dinamis memuat daftar *remote storage* (seperti Google Drive) yang telah terkonfigurasi pada server Rclone. Pengguna diwajibkan memilih salah satu sumber penyimpanan sebelum dapat melanjutkan, memastikan bahwa koneksi ke *cloud* telah valid sebelum eksplorasi data dilakukan.Setelah *drive* dipilih, antarmuka bertransisi ke tahap kedua, "Select File", yang mengintegrasikan komponen GDriveBrowser. Komponen ini berfungsi sebagai penjelajah berkas interaktif yang melakukan komunikasi dengan *backend* untuk menampilkan struktur direktori dan berkas yang ada di dalam *remote storage* yang dipilih. Pengguna dapat menavigasi folder dengan *double-click* dan memilih berkas atau direktori target dengan sekali klik. Mekanisme ini memberikan visualisasi langsung terhadap data yang tersimpan di *cloud*, meminimalisir risiko kesalahan pemilihan berkas yang sering terjadi pada input berbasis teks manual. Status pemilihan (apakah folder atau file) secara otomatis ditangkap oleh sistem untuk diproses pada tahap selanjutnya.

Tahap terakhir adalah "Destination" atau penentuan lokasi penyimpanan, yang menampilkan ringkasan berisi informasi *Drive*, nama *File*, dan *Path* sumber yang telah dipilih sebelumnya sebagai konfirmasi akhir. Pada tahap ini, pengguna diwajibkan memasukkan "Lokasi Penyimpanan di Server" melalui kolom input teks, yang merepresentasikan jalur direktori absolut di server lokal (misalnya /home/user/restore/) tempat data akan dipulihkan.

BAB IV
HASIL DAN PEMABAHASAN

Pada bab ini akan membahas secara rinci mengenai proses implementasi, instalasi, pengujian, serta hasil pengujian dari sistem yang dikembangkan. Pembahasan disusun secara sistematis untuk memberikan gambaran menyeluruh tentang langkah-langkah yang dilakukan dalam membangun dan menguji sistem, termasuk analisis terhadap hasil pengujian yang diperoleh.

4.1 Implementasi

Implementasi sistem Web-Dashboard G-Backup dilakukan dengan mengadopsi arsitektur API-Centric 3-Tier Architecture yang memisahkan tanggung jawab antara Backend (echo) dengan Frontend (Vue), sehingga mulai dari logika sistem, antarmuka pengguna dan interaksi dengan layanan drive terpisah dengan jelas.Pendekatan ini dipilih untuk memastikan skalabilitas dan mempermudah apabila ada rencana pengembangan kedepannya.Secara umum implementasi dibagi jadi 3 lapisan utama yakni, application layer, presentation layer, dan data layer yang memastikan masing masing layer memiliki peran masing masing dan saling berkoorrdinasi dalam mewujudkan proses backup.

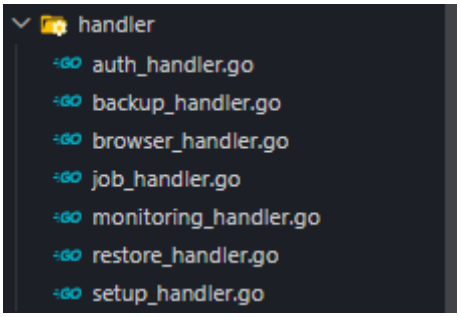
4.1.1 Application Tier

Lapisan ini adalah lapisan yang berisi inti logika dari sistem yang dibangun. Backend diimplementasikan memakai Golang dan Framework Echo.Code diorganisasi dengan pola MSC (Model Service Controller) untuk menjamin pemisahan fungsi antara logika bisnis, operasi database dan pemrosesan request. Secara garis besar, application layer terdiri dari:

a. Struktur code (Model-Service-Controller)

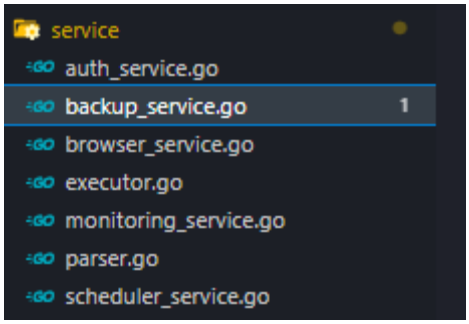
Application Tier memisahkan proses secara tegas melalui tiga komponen utama. Pemisahan ini bertujuan memastikan bahwa setiap perubahan pada satu lapisan tidak berdampak langsung terhadap lapisan lainnya.

- Controller layer: Menerima request HTTP, melakukan validasi dasar, lalu meneruskan eksekusi ke Service Layer.



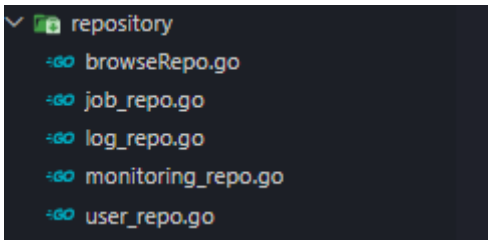
Gambar 4. 1 Controller

- Service layer: Pusat orkestrasi seluruh pekerjaan, mengatur logika eksekusi job, error handling, dan mengatur alur setiap fase backup.



Gambar 4. 2 services

- Repository layer: Menangani query ke database untuk menyimpan metadata job, status, dan konfigurasi pengguna.



Gambar 4. 3 repository

- Dependency injector(Main.go): File main.go menghubungkan semua lapisan sehingga tidak saling bergantung secara langsung.

```
package main

import (
    "fmt"
    "log"
    "net/http"
    "os"
    "path/filepath"
    "time"
```

```
    // Import semua package yang dibutuhkan
    "gbackup-new/backend/internal/handler"
    "gbackup-new/backend/internal/repository"
    "gbackup-new/backend/internal/service"
    "gbackup-new/backend/pkg/database"

    "github.com/joho/godotenv"
    echojwt "github.com/labstack/echo-jwt/v4"
    "github.com/labstack/echo/v4"
    echomid "github.com/labstack/echo/v4/middleware"
)

func loadDotEnv() {
    exePath, err := os.Executable()
    if err != nil {
        fmt.Println("Tidak bisa dapatkan path
executable:", err)
        return
    }
    exeDir := filepath.Dir(exePath)

    // Searching .env di lokal
    candidates := []string{
        filepath.Join(exeDir, ".env"),
        filepath.Join(exeDir, "..", ".env"),
        filepath.Join(exeDir, "..", "..", ".env"),
    }

    for _, p := range candidates {
        if _, err := os.Stat(p); err == nil {
            if err := godotenv.Load(p); err == nil {
                fmt.Println("Loaded .env dari:", p)
                return
            }
        }
    }

    _ = godotenv.Load(".env", "../.env")
}
func main() {
    // --- 1. MEMUAT .ENV ---
    loadDotEnv()

    // Ambil Kunci JWT (Wajib)
    jwtSecretKey := os.Getenv("JWT_SECRET_KEY")
    if jwtSecretKey == "" {
        log.Fatal("FATAL: JWT_SECRET_KEY ada")
    }

    // --- 2. KONEKSI DATABASE ---
    dbInstance := database.Connect()
    if dbInstance == nil {
        log.Fatal("Koneksi DB gagal, instance GORM
nil.")
```

```
    }

    // --- 3. DEPENDENCY INJECTION (DI) ---

    // 3.1. Inisialisasi Repository (Lapisan Data
Access)
    userRepo :=
repository.NewUserRepository(dbInstance)
    jobRepo := repository.NewJobRepository(dbInstance)
    logRepo := repository.NewLogRepository(dbInstance)
    monitorRepo :=
repository.NewMonitoringRepository(dbInstance)
    browserRepo := repository.NewBrowserRepository()

    // 3.2. Inisialisasi Service (Lapisan Logika
Bisnis)
    authSvc := service.NewAuthService(userRepo,
jwtSecretKey)
    monitorSvc :=
service.NewMonitoringService(monitorRepo, logRepo,
jobRepo)
    backupSvc := service.NewBackupService(jobRepo,
logRepo, monitorRepo, monitorSvc) // Orkestrator 3 Fase
    schedulerSvc :=
service.NewSchedulerService(jobRepo, backupSvc)
    browserSvc :=
service.NewBrowserService(browserRepo)

    // 3.3. Inisialisasi Handler (Controller Layer)
    authHandler := handler.NewAuthHandler(authSvc)
    monitorHandler :=
handler.NewMonitoringHandler(monitorSvc, schedulerSvc,
logRepo)
    jobHandler := handler.NewJobHandler(schedulerSvc,
backupSvc, jobRepo) // Membutuhkan BackupSvc untuk
TriggerManual
    backupHandler :=
handler.NewBackupHandler(backupSvc)
    restoreHandler :=
handler.NewRestoreHandler(backupSvc)
    browserHandler :=
handler.NewBrowserHandler(browserSvc)
    setupHandler := handler.NewSetupHandler(authSvc)

    // --- 5. SETUP WEB SERVER ECHO ---
    e := echo.New()
    e.Use(echomid.Logger())
    e.Use(echomid.Recover())
    e.Use(echomid.CORSWithConfig(echomid.CORSConfig{
        AllowOrigins:
[]string{"http://localhost:5173"}, // URL Frontend
        AllowMethods: []string{http.MethodGet,
http.MethodPost, http.MethodDelete},
    }))
```

```
    // --- 6. SETUP ROUTING ---
    e.GET("/", func(c echo.Context) error {
        return c.String(http.StatusOK, "G-Backup New
Architecture is Running!")
    })

    // Rute Publik (Login)
    e.POST("/api/v1/auth/login", authHandler.Login)
    setupGroup := e.Group("/api/v1/setup")
    setupGroup.GET("/status",
setupHandler.GetSetupStatus)          // Cek apakah
admin sudah ada
    setupGroup.POST("/register",
setupHandler.RegisterInitialAdmin) // Registrasi admin
pertama

    // Rute Privat
    r := e.Group("/api/v1")
    r.Use(echojwt.WithConfig(echojwt.Config{
        SigningKey: []byte(jwtSecretKey),
    })) // Menerapkan Middleware JWT

    // Rute Monitoring dan Logs
    r.GET("/monitoring/remotes",
monitorHandler.GetRemoteStatusList)
    r.GET("/monitoring/logs",
monitorHandler.GetJobLogs)
    r.GET("/monitoring/jobs",
monitorHandler.GetScheduledJobs)

    // Rute Job Management (Create, List, Trigger)
    r.GET("/jobs/scheduled",
jobHandler.GetScheduledJobs)    // List Job Monitoring
    r.GET("/jobs/script/:id", jobHandler.GetJobScript)
// Pratinjau Script
    r.POST("/jobs/trigger/:id",
jobHandler.TriggerManualJob) // Tombol "Run Now"
    r.GET("/jobs/manual", jobHandler.GetManualJob)
// List Job Manual
    r.DELETE("/jobs/delete/:id", jobHandler.DeleteJob)
// Hapus Job
    r.PUT("/jobs/update/:id", jobHandler.UpdateJob)
// Update Job
    r.GET("/jobs/:id", jobHandler.GetJobByID)
// Get Job by ID
    r.GET("/jobs/alljobs", monitorHandler.GetAllJobs)
// Get All Jobs (scheduled + Manual)

    // Rute Aksi
    r.POST("/jobs/new", backupHandler.CreateNewJob)
// Create Backup (Manual/Auto)
    r.POST("/jobs/restore",
restoreHandler.TriggerRestore)        // Create Restore
    r.GET("/browser/files", browserHandler.ListFiles)
//browse files
```

```
    r.GET("/browser/remotes",
browserHandler.GetAvailableRemotes) //list remotes
    r.GET("/browser/info", browserHandler.GetFileInfo)
//file info

    // --- 7. START DAEMONS (Goroutines) ---
    schedulerSvc.StartDaemon() // Memulai CRON Daemon
    monitorSvc.StartMonitoringDaemon()

    go func() {
        time.Sleep(2 * time.Second)
        fmt.Println("[Startup] Inisialisasi status
remote monitoring...")

        if err := monitorSvc.SyncRemotesWithRclone();
err != nil {
            fmt.Printf("❌ ERROR: Gagal sync remote:
%v\n", err)
            return
        }

        remotes, err :=
monitorSvc.GetRemoteStatusList()
        if err != nil {
            fmt.Printf("❌ ERROR: Gagal mengambil
remote: %v\n", err)
            return
        }

        if len(remotes) > 0 {
            fmt.Printf("✅ Menemukan %d remote, memulai
update status...\n", len(remotes))
            for _, remote := range remotes {
                go
monitorSvc.UpdateRemoteStatus(remote.RemoteName)
            }
        } else {
            fmt.Println("⚠️ Tidak ada remote yang
dikonfigurasi di rclone.conf")
        }

        fmt.Println("✅ Inisialisasi remote monitoring
selesai.")
    }()

    fmt.Println("\nBackend diinisialisasi. Menjalankan
Echo server di port 8080")
    e.Logger.Fatal(e.Start(":8080"))
}
```

b. Implementasi Script Runner

Proses backup dieksekusi oleh Script Runner berbasis Goroutine, yang mengatur seluruh langkah backup mulai dari pre-script, eksekusi Rclone, hingga post-script. Mekanisme ini memastikan bahwa setiap job berjalan secara terisolasi dan tidak mengganggu proses lain. Alur proses script runner mencakup:

1. **Fase Pre-script:** Menjalankan perintah awal seperti mysqldump atau zip. Sistem otomatis menyuntikkan *Bash Strict Mode* (set -eo pipefail) agar kesalahan terdeteksi sejak awal.

```
    if job.PreScript != "" {
        fmt.Printf("[WORKER %d] Menjalankan Pre-
Script...\n", job.ID)
        hardenedPreScript := fmt.Sprintf("set -eo pipefail;
\n%s", job.PreScript)
        preScriptArgs := []string{"bash", "-c",
hardenedPreScript}

        result := ExecuteCliJob(preScriptArgs)
        if !result.Success {
            fmt.Printf("❌ [WORKER %d] Pre-Script
GAGAL.\n", job.ID)
            finalResult = result
            finalStatus = "FAIL_PRE_SCRIPT"
            s.handleJobCompletion(job, finalResult,
finalStatus)
            return
        }
    }
```

2. **Fase Rclone Execution:** Backend menyusun command Rclone berdasarkan konfigurasi user. Eksekusi dilakukan melalui os/exec, dan output JSON-nya diparsing untuk mendapatkan informasi checksum, transfer rate, status sukses, dan error.

```
func (s *backupServiceImpl) buildRcloneArgs(job
models.ScheduledJob, runtimeDestPath string) []string {
    isRestore := job.OperationMode == "RESTORE"
    command := strings.ToLower(job.RcloneMode)
    // Tentukan apakah sumber adalah file atau folder
    sourceInfo, err := os.Stat(job.SourcePath)
    isSourceDir := false
    if err == nil {
        isSourceDir = sourceInfo.IsDir()

    var SourcePath, Destination string
    if isRestore {
```

```
        SourcePath = fmt.Sprintf("%s:%s", job.RemoteName,
job.SourcePath)
        Destination = job.DestinationPath
        command = "copy"
    } else {
        SourcePath = job.SourcePath
        Destination = fmt.Sprintf("%s:%s", job.RemoteName,
runtimeDestPath)
        if !isSourceDir {
            command = "copyto"
        } else {
            command = "copy"
        }
    }
    args := []string{
        "rclone",
        command,
        SourcePath,
        Destination,
        "--checksum",
        "--no-traverse",
        "--progress",    //Tampilkan progress dengan stats
        "--stats", "5s", //Print stats setiap 5 detik
        "--stats-log-level", "INFO", //Change to INFO level
        "--human-readable",
    }
    return args
}
```

Eksekusi Rclone

```
  fmt.Printf("[WORKER %d] Menjalankan Rclone...\n", job.ID)
    rcloneArgs := s.buildRcloneArgs(job, runtimeDestPath)
    resultRclone := ExecuteCliJob(rcloneArgs)

    if !resultRclone.Success {
        fmt.Printf("❌ [WORKER %d] Rclone GAGAL.\n", job.ID)
        finalResult = resultRclone
        finalStatus = "FAIL_RCLONE"
        s.handleJobCompletion(job, finalResult, finalStatus)
        return
    }

    lines := strings.Split(resultRclone.Output, "\n")
    var transferStatus string
    for _, line := range lines {
        if strings.Contains(line, "Transferred:") {
            transferStatus = strings.TrimSpace(line)
        }
    }
```

```
    if transferStatus != "" {
        fmt.Printf(" [WORKER %d] Stats: %s\n", job.ID,
transferStatus)
        // menyimpan  Log
        resultRclone.Output = fmt.Sprintf("%s\n\n%s",
transferStatus, resultRclone.Output)
    }
```

3. **Fase Post-script:** Menjalankan perintah user sesudah data berhasil di backup, misal untuk clean up di server local.

```
if job.PostScript != "" {
        fmt.Printf("[WORKER %d] Menjalankan Post-
Script...\n", job.ID)
        hardenedPostScript := fmt.Sprintf("set -eo pipefail;
\n%s", job.PostScript)
        postScriptArgs := []string{"bash", "-c",
hardenedPostScript}
        resultPost := ExecuteCliJob(postScriptArgs)
        if !resultPost.Success {
            fmt.Printf("❌ [WORKER %d] Post-Script
GAGAL.\n", job.ID)
            finalResult = resultPost
            finalStatus = "FAIL_POST_SCRIPT"
            s.handleJobCompletion(job, finalResult,
finalStatus)
            return // Hentikan eksekusi
        }
```

4. **Status tracking:** Setiap fase memiliki kegagalan yang berbeda beda, sistem menggunakan error ini untuk memperbaiki status job di database sehigga user bisa tahu secara detail jika terjadi kegagalan pada job.

```
func (s *backupServiceImpl) handleJobCompletion(job
models.ScheduledJob, result RcloneResult, status string) {
    LogMutex.Lock()
    defer LogMutex.Unlock()
    stats := parseRcloneStats(result.Output)
    logMessage := result.ErrorMsg

    // 1. Catat ke tabel Logs (dengan TransferredBytes yang
sudah di-parse)
    newLog := &models.Log{
        JobID:            nil,
        JobName:          job.JobName,
        SourcePath:       job.SourcePath,
        Status:           status,
```

```
        Message:          logMessage,
        DurationSec:      int(result.Duration.Seconds()),
        TransferredBytes: result.TransferredBytes, // ✅
UPDATED: dari parsed bytes
        Timestamp:        time.Now(),
    }

    if job.ID != 0 {
        newLog.JobID = &job.ID
    }

    fmt.Printf("[LOG DEBUG] Saving ID: %d | Status: %s |
Bytes: %d\n", job.ID, status, result.TransferredBytes)
    s.LogRepo.CreateLog(newLog)

    // 2. Update status akhir di tabel scheduled_jobs
    if job.ID != 0 {
        // 2. Update status akhir di tabel scheduled_jobs
        dbStatus := status
        if status == "SUCCESS" {
            dbStatus = "COMPLETED"
        }

        if job.ScheduleCron == "" {
            dbStatus = "PENDING"
        }

        s.JobRepo.UpdateLastRunStatus(job.ID, time.Now(),
dbStatus)
    }
    // ✅ TAMBAHAN: Log ringkasan
    if status == "SUCCESS" {
        fmt.Printf("✅ [COMPLETE] Job %d: Transferred %.2f
GB in %d seconds (Speed: %s)\n",
            job.ID,
            float64(result.Duration)/1e9/1e9,
            stats.Speed,
        )
    } else {
        fmt.Printf("❌ [FAILED] Job %d: %s\n", job.ID,
status)
    }
}
```

4.1.2 Presentation Layer

Lapisan yang berperan sebagai jembatan supaya user bisa berinteraksi dengan logika sistem, lapisan ini dibangun dengan vue.js dengan pola Component-Based Architecture (CBA) untuk memungkinkan UI yang modular dan mudah dikembangkan.

Beberapa tanggung jawab utama Presentation Layer meliputi:

1. Parsing data mentah dari endpoint API.

    - Parsing Log

```
function formatFullTimestamp(timestamp) {
  if (!timestamp) return '-';
  try {
    return new Date(timestamp).toLocaleString('id-ID', {
      dateStyle: 'medium',
      timeStyle: 'medium'
    });
  } catch (e) {
    return timestamp;
  }
}
function formatFileSize(bytes) {
  // Konversi ke number untuk memastikan keamanan
  const size = Number(bytes);
  if (!size || size === 0 || isNaN(size)) return '0 B';
  const units = ['B', 'KB', 'MB', 'GB', 'TB'];
  const k = 1024;
  const i = Math.floor(Math.log(size) / Math.log(k));

  return parseFloat((size / Math.pow(k, i)).toFixed(2)) + '
' + units[i];
}
function formatJSON(jsonInput) {
  if (!jsonInput) return '';
  try {
    // Jika input sudah object, stringify langsung. Jika
string, parse dulu.
    const obj = typeof jsonInput === 'string' ?
JSON.parse(jsonInput) : jsonInput;
    return JSON.stringify(obj, null, 2);
  } catch (e) {
    return jsonInput;
  }
}
```

- Parsing data untuk tampilan storage

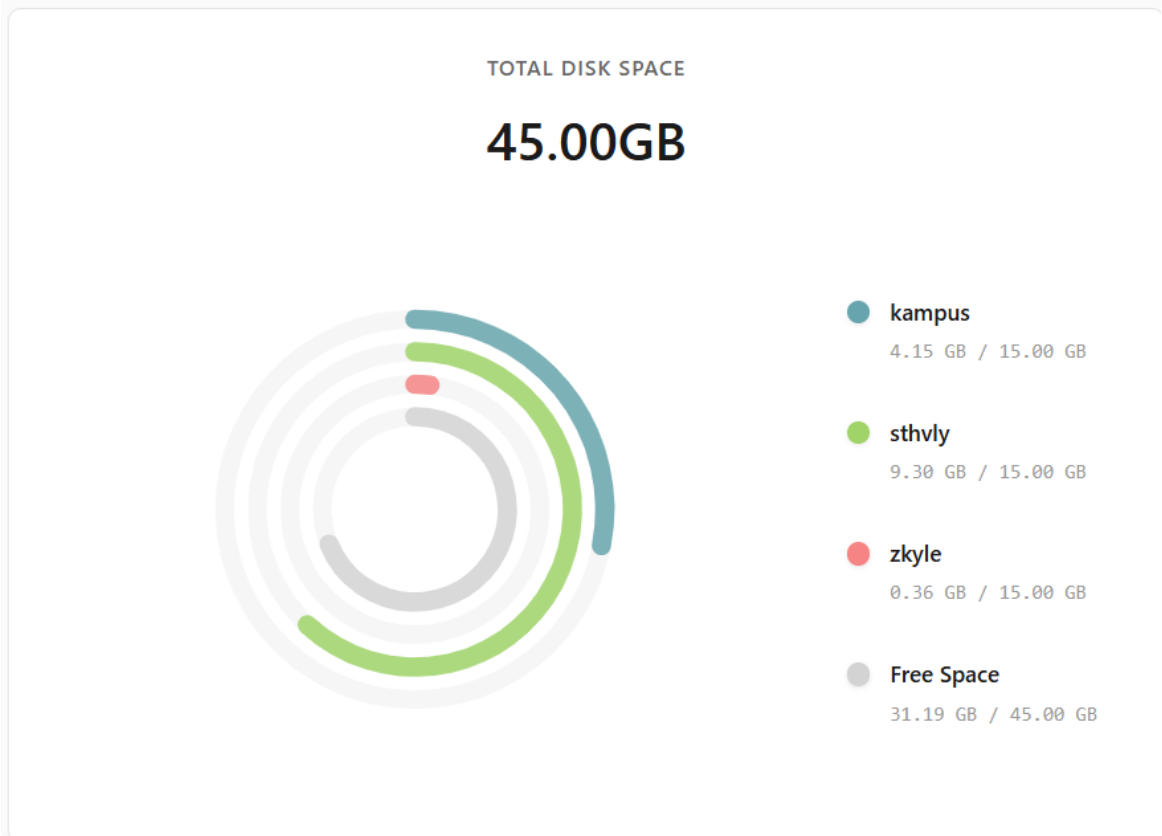
```
const totalStorageGB = computed(() => {
  const total = remotes.value.reduce((sum, remote) => {
    if (remote.status_connect === 'CONNECTED') {
      return sum + (remote.total_storage_gb || 0);
    }
    return sum;
  }, 0);
  return total.toFixed(2);
});
```

- Parsing untuk data next job

```
const nextJob = computed(() => {
  if (!Array.isArray(scheduledJobs.value) ||
scheduledJobs.value.length === 0) return null;
  const futureJobs = scheduledJobs.value
    .map(j => ({ ...j, nextRunDate: new Date(j.next_run) }))
    .filter(j => j.nextRunDate && j.nextRunDate > new
Date())
    .sort((a, b) => a.nextRunDate - b.nextRunDate);
  return futureJobs.length > 0 ? {
    jobType: futureJobs[0].type,
    nextRun: futureJobs[0].nextRunDate,
    name: futureJobs[0].job_name,
    remoteName: futureJobs[0].gdrive_target,
    scheduleCron: futureJobs[0].next_run,
    sourcePath: futureJobs[0].source_path,
  } : null;
});
```

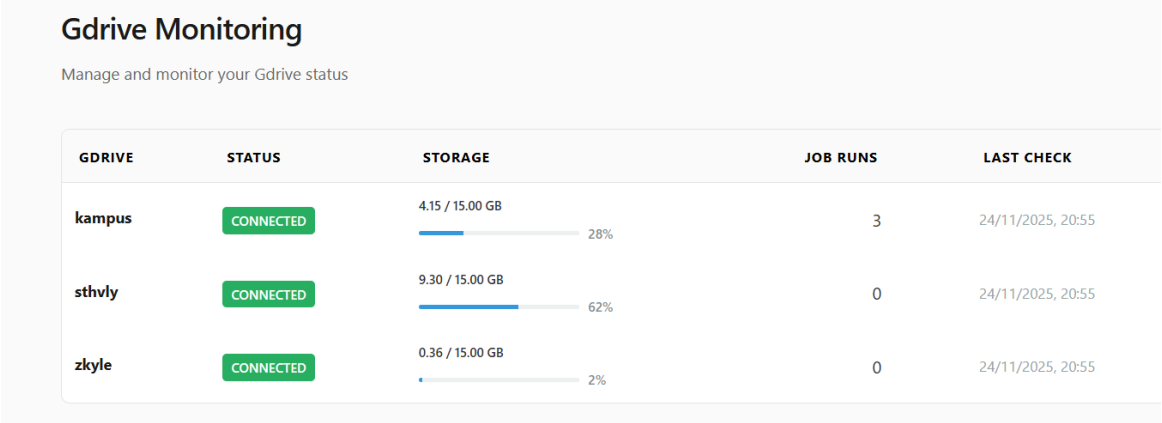
2. Menyajikan informasi dalam bentuk visual seperti chart storage, status storage dan job, serta audit log.

   - **Visualisasi Storage**



Gambar 4. 4 Storage Chart

Gambar diatas merupakan visualisasi storage dari tiap drive yang terhubung, menyajikan total storage dari semua Gdrive yang dikoneksikan melalui rclone, untuk tiap line berwarna mewakili masing masing drive untuk mewujudkan visualisasi yang dapat lebih mudah dicerna baik iitu dari berapa banyak storage yang digunakan oleh tiap drive sampai total storage kosong yang tersedia.Selanjutnya.

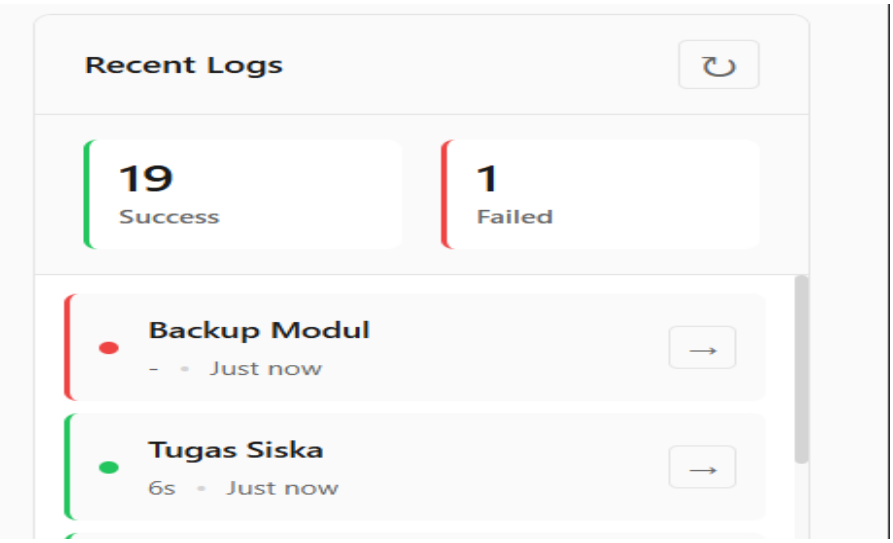


Gambar 4. 5 Storage Detail

Gambar diatas merupakan gambaran lebih detail untuk tiap drive yang terkoneksi, mulai dari status koneksi drive dengan rclone sampai ke detail storage masing masing drive, serta ada berapa banyak Job yang terhubung ke masing masing drive.Halaman *Gdrive Monitoring* berfungsi sebagai pusat kontrol untuk memantau kondisi seluruh akun Google Drive yang terhubung dengan sistem. Pada tampilan ini, setiap drive ditampilkan secara terpisah beserta status koneksinya, sehingga pengguna dapat dengan mudah memastikan apakah sebuah akun sedang dalam keadaan *connected* atau mengalami gangguan. Selain itu, halaman ini menyajikan informasi penggunaan storage secara visual melalui indikator bar, yang menunjukkan kapasitas terpakai dibanding total kuota yang tersedia.

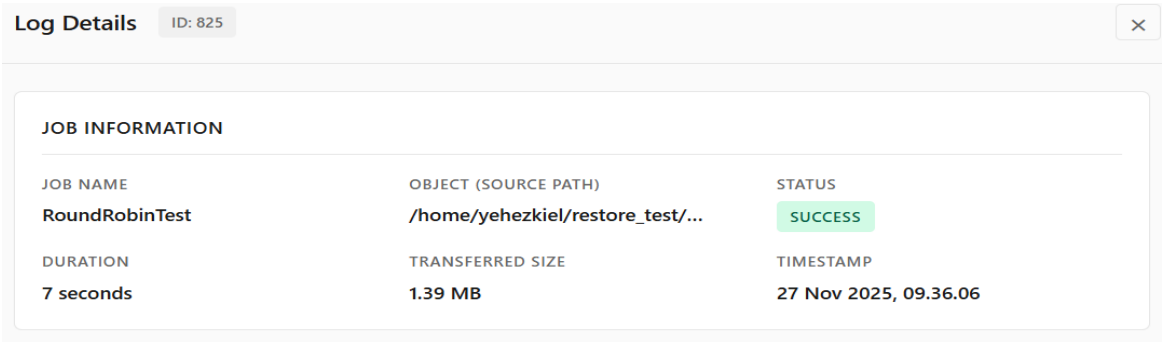
Dengan demikian, pengguna dapat mengetahui persentase pemakaian dan mengantisipasi risiko kehabisan ruang penyimpanan. Kolom *Job Runs* memperlihatkan jumlah operasi backup atau proses lain yang pernah dijalankan pada masing-masing drive, memberikan gambaran aktivitas dan beban penggunaan sebuah akun. Sementara itu, kolom *Last Check* menampilkan waktu terakhir sistem melakukan pemeriksaan kondisi drive tersebut, memastikan bahwa monitoring selalu up-to-date. Secara keseluruhan, halaman ini memberikan transparansi penuh terhadap status Google Drive dan membantu pengguna menjaga kesehatan serta kesiapan setiap akun yang digunakan dalam proses backup maupun restore.

- **Visualisasi Log**



Gambar 4. 6 Live Log

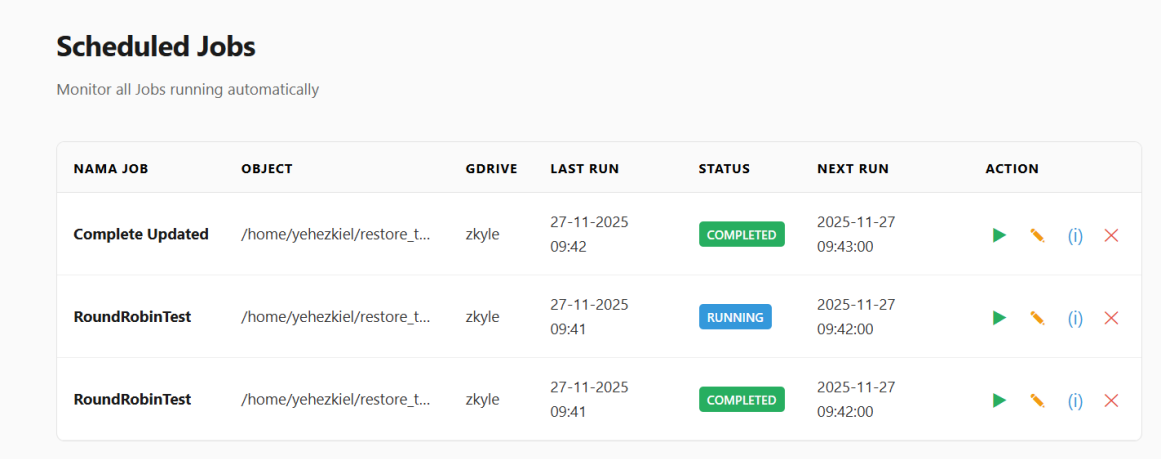
Visualisasi log diatas dibuat supaya user bisa tahu job mana saja yang sudah selesai dan status penyelesaian dari job, visualisasi ini juga dibatasi hingga 20 log saja, untuk membatasi data yang tidak perlu, kemudian pada tanda panah ketika di klik maka akan manampilkan informasi detail mengenai job yang baru selesai.



Gambar 4. 7 Live Log

Tampilan *Log Details* menampilkan ringkasan lengkap dari satu proses job yang telah dijalankan. Informasi penting seperti nama job, lokasi objek sumber, dan status eksekusi ditampilkan secara jelas sehingga pengguna dapat langsung mengetahui apakah proses berjalan sukses atau mengalami kegagalan. Selain itu, detail durasi eksekusi, ukuran data yang berhasil ditransfer, serta timestamp pencatatan log membantu pengguna melakukan pengecekan dan analisis lebih akurat terhadap hasil setiap operasi. Dengan penyajian yang ringkas, halaman ini memudahkan pemantauan setiap aktivitas backup atau restore dalam sistem.

- **Monitoring Job**

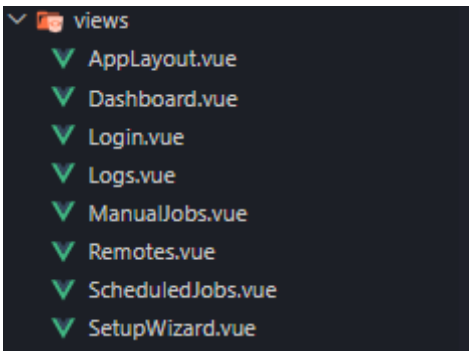


Gambar 4. 8 Monitoring Job

Implementasi halaman pemantauan pekerjaan terjadwal (*Scheduled Jobs*), sebagaimana ditampilkan pada gambar di atas, dirancang sebagai pusat kendali utama bagi administrator untuk mengawasi seluruh siklus hidup tugas pencadangan yang berjalan secara otomatis. Antarmuka ini menyajikan data dalam format tabel. visibilitas *real-time* terhadap status operasional sistem. Setiap baris pada tabel merepresentasikan satu konfigurasi pekerjaan unik dengan atribut-atribut kunci yang ditampilkan secara ringkas. Kolom "Nama Job" dan "Object" memberikan identifikasi cepat mengenai apa yang sedang dicadangkan dan lokasinya di server lokal, sementara kolom "GDrive" menunjukkan tujuan penyimpanan awan yang terkait.

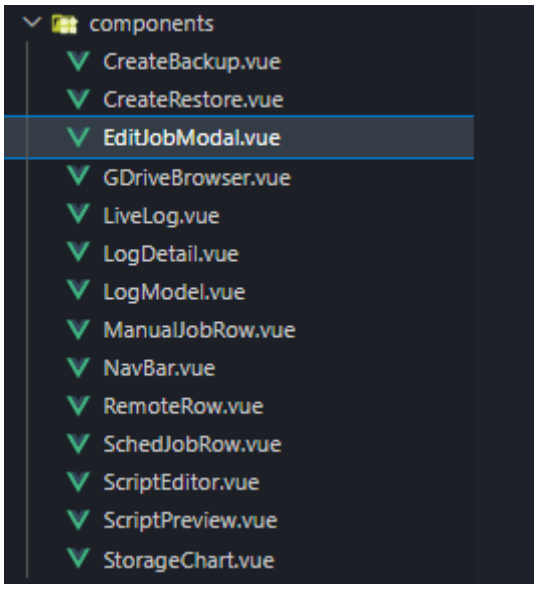
Fitur krusial dari antarmuka ini adalah mekanisme pelacakan waktu dan status eksekusi. Kolom "Last Run" dan "Next Run" memberikan informasi kronologis yang presisi, memungkinkan administrator untuk memverifikasi kapan terakhir kali sistem bekerja dan memprediksi eksekusi selanjutnya berdasarkan jadwal *Cron* yang telah dikonfigurasi. Status kesehatan pekerjaan divisualisasikan melalui indikator "Status" yang menggunakan kode warna intuitif; hijau dengan label "COMPLETED" menandakan keberhasilan operasi, sementara biru dengan label "RUNNING" menunjukkan bahwa proses sedang berlangsung secara aktif di latar belakang.

3. Mengatur views seperti *RemoteConfig*, *JobMonitoring*, dan *Dashboard* dengan komponen-komponen reusable. Berikut adalah bagian yang mengatur tampilan dasar untuk Dashboard, remote dan job monitoring.



Gambar 4. 7 Views

folder views merupakan folder tempat menyimpan tampilan dasar untuk UI dan tiap views punya component masing masing yang dapat reuseable.



Gambar 4. 8 Reuseable Component

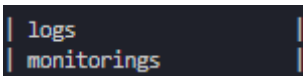
Dengan pendekatan ini, setiap update pada backend tidak mengharuskan perubahan besar pada UI, sehingga sistem tetap fleksibel dan mudah dipelihara.

4.1.3 Data Layer

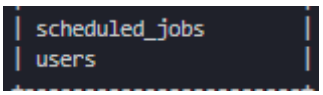
Data Layer bertanggung jawab terhadap persistensi metadata dan interaksi dengan sistem eksternal. Basis data menggunakan MariaDB untuk menyimpan konfigurasi pengguna, status job, dan log eksekusi. Untuk operasi terhadap penyimpanan cloud, sistem menggunakan Rclone sebagai antarmuka eksternal.

Fungsi utama Data Layer meliputi:

- **MariaDB** → menyimpan konfigurasi remote, audit trail, jadwal job, dan status proses secara real time.

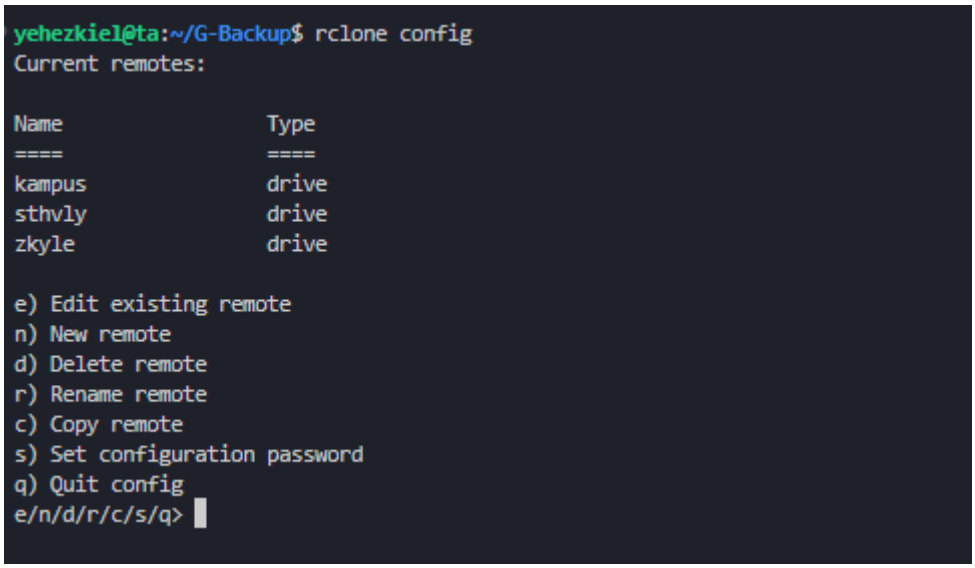


Gambar 4. 9 Database



Gambar 4. 10 Databaase

- **Rclone CLI** → menjalankan operasi file (copy, sync, move, check) dan monitoring storage melalui output JSON yang diproses oleh backend.



Gambar 4. 11 Rclone CLI

4.2 Instalasi system

4.2.1 Instalasi dan Konfigurasi Sistem

Pada sub-bab ini akan dipaparkan prosedur instalasi dan konfigurasi sistem G-Backup secara menyeluruh, dimulai dari identifikasi persyaratan sistem hingga tahapan instalasi menggunakan script otomatis berbasis Bash Shell. Script instalasi yang dikembangkan dirancang dengan mekanisme interaktif dan fault-tolerant yang mampu mendeteksi dependencies, melakukan instalasi perangkat lunak pendukung, mengkonfigurasi database, serta melakukan build aplikasi secara sequential tanpa memerlukan intervensi manual yang kompleks.

4.2.1.1 Persyaratan Perangkat Lunak

Tabel berikut menjelaskan perangkat lunak beserta versi minimum yang dibutuhkan untuk menjalankan sistem G-Backup. Setiap komponen memiliki peran spesifik dalam arsitektur sistem, mulai dari sistem operasi sebagai platform dasar, bahasa pemrograman untuk runtime aplikasi, database untuk penyimpanan metadata, hingga utilitas Rclone sebagai mesin transfer data ke cloud storage. Instalasi seluruh dependencies ini akan dilakukan secara otomatis oleh script instalasi yang telah disediakan.

Tabel 4. 1 Software Requirements

PerangkatLunak	Versi Minimal	Fungsi
Sistem Operasi	Ubuntu 20.04 / Debian 10	Host OS (Linux direkomendasikan)
Go (Golang)	v1.20.x	Kompilasi Backend & API
Node.js	v18.x (LTS)	Runtime untuk Vue.js & Vite
NPM	v9.x	Package manager untuk Interface
MariaDB / MySQL	v10.4 / v8.0	Penyimpanan Metadata & Log
Rclone CLI	v1.60 atau terbaru	Mesin transfer data ke Cloud
Browser (Client)	Chrome/Firefox terbaru	Antarmuka Dashboard

4.2.1.2 Kebutuhan Sistem (System Requirements)

Sebelum melakukan instalasi, perlu dipastikan bahwa sistem memenuhi persyaratan minimum yang telah ditentukan. Persyaratan ini dibagi menjadi dua kategori utama: persyaratan perangkat keras yang mencakup spesifikasi komponen fisik server, dan persyaratan perangkat lunak yang mencakup sistem operasi serta dependencies yang dibutuhkan oleh aplikasi. Pemenuhan persyaratan ini menjamin sistem dapat beroperasi

dengan stabil dan optimal, terutama saat menangani proses backup berukuran besar yang memerlukan resources komputasi dan bandwidth jaringan yang memadai.

Tabel 4. 2 System Requirements

Komponen	Spesifikasi Minimum	Rekomendasi
Processor (CPU)	1 Core (1.0 GHz)	2 Cores atau lebih
Memory (RAM)	1 GB	2 GB
Storage (Disk)	10 GB Available space*	40 GB+ (SSD lebih baik)
Network	Koneksi Internet (Upload)	Koneksi Stabil (Bandwidth tinggi)

4.2.2 Langkah-Langkah Instalasi

Sistem G-Backup menyediakan script instalasi otomatis yang menangani seluruh proses instalasi. Berikut adalah langkah-langkah yang perlu dilakukan:

1. **Persiapan**

   Pastikan sistem terhubung dengan internet dan memiliki akses sudo. Download atau clone repository sistem G-Backup ke dalam direktori lokal.

2. **Memberikan Permission Execute**

   Berikan izin eksekusi pada script instalasi

```
chmod +x install.sh
```

3. **Menjalankan Script Instalasi**

   Jalankan script instalasi dengan perintah:

```
./install.sh
```

4. **Proses Instalasi Otomatis**

   Script akan menjalankan tahapan berikut secara otomatis:

   A. Pengecekan Dependencies

   - Script memeriksa ketersediaan Go, Node.js, NPM, MySQL, dan Rclone
   - Jika belum tersedia, sistem akan menginstall secara otomatis

   B. Instalasi Go

   - Jika Go belum terinstall atau versi < 1.20, sistem akan menginstall Go versi 1.23.5
   - Proses download dan instalasi ditampilkan dengan progress bar

C. Instalasi Node.js

- Jika Node.js belum terinstall atau versi < 18, sistem akan menampilkan prompt
- Pilih y untuk menginstall Node.js LTS versi terbaru
- Pilih n untuk melanjutkan dengan versi yang ada (tidak direkomendasikan)

D. Instalasi Dependencies Lain

- MySQL Client untuk koneksi database
- Rclone untuk transfer data ke cloud storage

5. Konfigurasi Database

Setelah instalasi dependencies selesai, sistem akan menampilkan wizard konfigurasi database:

Jika MySQL belum terinstall:

1. Masukkan username administrator MySQL (tekan Enter untuk default: root)
2. Masukkan password (minimal 6 karakter)
3. Konfirmasi password
4. Tunggu proses instalasi MySQL Server selesai

Jika MySQL sudah terinstall:

1. Masukkan username MySQL yang sudah ada (tekan Enter untuk default: root)
2. Masukkan password
3. Sistem akan memverifikasi kredensial (maksimal 3 percobaan)

6. Konfigurasi Parameter Database

Masukkan informasi berikut (tekan Enter untuk menggunakan nilai default):

- **Database name**: Nama database yang akan digunakan (default: gbackup_db)
- **Database host**: Alamat server database (default: 127.0.0.1)
- **Database port**: Port MySQL (default: 3306)

7. Konfigurasi Service Port

Sistem akan otomatis mencari port yang tersedia untuk aplikasi.

8. **Build Aplikasi**

   Script akan otomatis:

   - Membuat file konfigurasi .env di direktori Backend
   - Mendownload dependencies Go
   - Melakukan kompilasi aplikasi Backend
   - Menginstall dependencies Frontend (jika ada)

9. **Instalasi Selesai**

   Setelah proses instalasi selesai, sistem akan menampilkan:

   - URL akses aplikasi: http://[IP_ADDRESS]:[PORT]
   - Instruksi tambahan jika diperlukan

   Contoh output:

```
✓ System successfully installed
Access URL: http://192.168.1.100:8080
```

4.2.3 Konfigurasi Sistem

Setelah instalasi selesai, lakukan konfigurasi tambahan berikut untuk mengoperasikan sistem

1. **Konfigurasi Rclone (Cloud Storage)**

   Sebelum dapat menggunakan sistem backup, perlu mengkonfigurasi koneksi ke cloud storage. Jalankan wizard konfigurasi Rclone

```
rclone config
```

   Langkah konfigurasi untuk Google Drive:

   1. Ketik n untuk membuat remote baru, tekan Enter
   2. Masukkan nama remote (contoh: gdrive-backup), tekan Enter
   3. Ketik drive untuk Google Drive atau ketik angka 22, tekan Enter
   4. Kosongkan client_id (tekan Enter)
   5. Kosongkan client_secret (tekan Enter)
   6. Pilih 1 untuk full access, tekan Enter
   7. Kosongkan root_folder_id (tekan Enter)
   8. Kosongkan service_account_file (tekan Enter)
   9. Ketik n untuk skip advanced config, tekan Enter
   10. Ketik y untuk auto config, tekan Enter
   11. Browser akan terbuka otomatis, login dengan akun Google

12. Klik "Allow" untuk memberikan akses
13. Kembali ke terminal, ketik n untuk team drive, tekan Enter
14. Ketik y untuk konfirmasi, tekan Enter
15. Ketik q untuk quit, tekan Enter

Remote Google Drive berhasil dikonfigurasi dan siap digunakan.

2. Menjalankan Sistem

Setelah instalasi selesai, sistem siap dijalankan menggunakan script start.sh.

A. Berikan Permission Execute

```
chmod +x start.sh
```

B. Jalankan Script

```
./start.sh
```

C. Proses Startup

Script start.sh akan melakukan tahapan berikut secara otomatis:

1. **Verifikasi Konfigurasi**
   - Memeriksa file konfigurasi Backend/.env
   - Jika tidak ditemukan, sistem akan meminta menjalankan install.sh
2. **Auto-Rebuild Detection**
   - Sistem memeriksa apakah ada perubahan pada kode Backend
   - Jika terdeteksi perubahan, akan otomatis melakukan rebuild
   - Progress ditampilkan dengan progress bar
3. **Frontend Dependencies Check**
   - Memeriksa apakah node_modules sudah terinstall
   - Jika belum atau ada update package.json, akan otomatis install dependencies
4. **Database Connection Test**
   - Memverifikasi koneksi ke database MySQL
5. **Service Startup**
   - Menjalankan Backend service
   - Menampilkan URL akses

D. Output Sukses

Jika semua tahap berhasil, akan muncul output seperti berikut:

```
✓ Application is up to date
✓ System started successfully
Access URL:
  ➜ API Service: http://192.168.1.100:8080 (primary)
  ➜ Web Interface: http://192.168.1.100:5173 (primary)
Press Ctrl+C to stop
```

E. Multiple Network Interfaces

Jika sistem memiliki lebih dari satu network interface, akan ditampilkan semua URL yang tersedia:

```
Access URL (multiple networks):
  ➜ API Service: http://192.168.1.100:8080 (primary)
  ➜ Web Interface: http://192.168.1.100:5173 (primary)
    http://10.0.0.50:8080
    http://10.0.0.50:5173
Gunakan URL yang ditandai (primary) untuk akses optimal.
```

4.3 Pengujian

Metode pengujian dilakukan untuk memastikan bahwa sistem G-Backup berfungsi sesuai dengan kebutuhan, mampu menangani skenario kegagalan, serta menghasilkan hasil yang konsisten.Pengujian dilakukan untuk memvalidasi fungsionalitas sistem. Alat uji yang dipakai pada pengujian sistem ini adalah Web Browser untuk memverifikasi tampilan dan dan Postman kinjerja sistem.Pengujian dibagi berdasarkan skenario fungsionalitas yang telah didefiniskan pada BAB III(keamanan, Monitoring, eksekusi job dan lain lain).

4.3.1 Pengujian Autentikasi

Pengujian pada modul autentikasi merupakan tahap awal yang sangat krusial untuk memastikan bahwa hak akses ke dashboard manajemen backup hanya dimiliki oleh personil yang berwenang. Fokus utama dari pengujian ini adalah memverifikasi keandalan sistem dalam mengenali kredensial pengguna, menangani kesalahan login, serta mengamankan rute terproteksi melalui mekanisme *JSON Web Token* (JWT). Dengan pengujian ini, diharapkan integritas sistem tetap terjaga dari upaya akses tidak sah.

Tabel 4. 3 Pengujian Autentikasi

No	Fungsi yang Diuji	Skenario Pengujian	Data Input	Hasil yang Diharapkan	Hasil Pengujian	Status
1	Login Validation	Percobaan login dengan kolom username dan password kosong	`username: ""password: ""`	Sistem menolak login dan menampilkan pesan input tidak valid	`message: "input tidak valid"`	✓ Pass
2	Brute Force Guard	Percobaan login dengan kredensial tidak valid	`Username: "yehezkiel"Password: "admin"`	Sistem menolak login dan mengembalikan HTTP 401 Unauthorized	`method: POSTuri: /api/v1/auth/loginstatus: 401`	✓ Pass
3	Direct Login	Login menggunakan kredensial yang valid	`Username: "yehezkiel"Password: "kiel"`	Sistem menerima login dan mengembalikan HTTP 200	`method: POSTuri: /api/v1/auth/loginstatus: 200`	✓ Pass
4	Route Guard	Mengakses halaman terproteksi tanpa login	`URL: http://10.31.94.3:5173/scheduled`	Sistem mengarahkan pengguna ke halaman login	`Not authenticated, redirect to login`	✓ Pass
5	Setup Wizard Access	Akses aplikasi pada penggunaan pertama kali	`URL: http://10.31.94.3:5173/login`	Sistem mengarahkan ke halaman setup wizard	`method: GETuri: /api/v1/setup/status`	✓ Pass

4.3.2 Pengujian Job Management

Pengujian pada modul manajemen job bertujuan untuk memverifikasi kemampuan sistem dalam mengelola siklus hidup template backup dan restore. Proses ini mencakup pengujian fungsionalitas CRUD (*Create, Read, Update, Delete*) terhadap data job yang tersimpan di database MariaDB. Melalui pengujian ini, dipastikan bahwa pengguna dapat mengonfigurasi parameter teknis seperti jalur sumber, remote tujuan, dan mode operasional

secara akurat, serta memastikan sinkronisasi data antara antarmuka web dengan lapisan basis data tetap konsisten.

Tabel 4. 4 Pengujian Job Management

Fungsi yang diuji	Skenario Pengujian	Data Input	Hasil yang diharapkan	Hasil Pengujian	Status
Create Manual Job	Membuat job tanpa cron	jobName:Backup SourcePath:var/Test.txt backupMode: Copy Drive: Kampus Destination path: Test max retention: 3 cron: - Pre-script: - Post-script: -	Backup berjalan, files terupload ke Google Drive	"Job 84 (Backup) disimpan dan dipicu langsung."	✓ Pass
Create Scheduled Job	Membuat job dengan cron	jobName:Backup SourcePath:var/Test.txt backupMode: Copy Drive: Kampus Destination path: Test max retention: 3 cron: * 8 * * * Pre-script: - Post-script: -	Backup berjalan sesuai jadwal, files terupload ke Google Drive	"Job 84(Backup) disimpan untuk Scheduler." z	✓ Pass
Create Job dengan Pre-Script	membuat job dengan pre-script	date > /home/yehezkiel/FOLDER_DEMO/tes.txt	Job dengan pre-script tersimpan	"Job 85 (Backup) disimpan"	✓ Pass
Create Job dengan Post-Script	membuat job dengan post script	rm /home/yehezkiel/FOLDER_DEMO/tes.txt	Job dengan post-script tersimpan	"Job 86 (Backup) disimpan"	✓ Pass
Read Single Job	membuka detail job	Job ID: 77	Detail lengkap job ditampilkan	"method":"GET","uri":"/api/v1/jobs/77"	✓ Pass
Read All Jobs	Membuka daftar job	-	Semua job ditampilkan	"method":"GET","uri":"/api/v1/jobs/alljobs"	✓ Pass
Update Job Config	Mengubah konfigurasi job yg ada	jobName:Unit-Test SourcePath:var/DOC.txt backupMode: SYNC Drive: sthvly Destination path: Unit-Test max retention: 5 cron: * 9 * * * Pre-script: - Post-script: -	Konfigurasi job terbarukan	"method":"PUT","uri":"/api/v1/jobs/update/77"	✓ Pass
Delete Job with Confirmation	User delete dengan konfirmasi	Job ID: 3 User confirms	Job terhapus setelah konfirmasi	"method":"DELETE","uri":"/api/v1/jobs/delete/85"	✓ Pass

4.3.3 Pengujian Operasi Backup

Skenario pengujian operasi backup dilakukan untuk memvalidasi alur kerja utama (core workflow) sistem dalam memindahkan data dari server lokal ke cloud storage Google Drive. Pengujian ini mencakup verifikasi parameter seperti source path, pemilihan remote, hingga pembentukan folder tujuan di sisi cloud. Keberhasilan pengujian ini diukur dari kesesuaian data yang diunggah dengan data asli serta perubahan status job yang akurat pada antarmuka pengguna.

Tabel 4. 5 Pengujian Operasi Backup

Fungsi yang diuji	Skenario Pengujian	Data Input	Hasil yang diharapkan	Hasil Pengujian	Status
Manual Trigger Backup	Trigger tombol run	Job ID: 78	Job berhasil berjalan	"[WORKER 78] Job Post-Test: Memulai Eksekusi 3 Fase..."	✓ Pass
Auto Trigger Backup	Scheduler Trigger tombol run	Job ID: 6	Job berjalan Tepat waktu	"[SCHEDULER] Dispatching Job 77 (Pre-Test)"	✓ Pass
Backup Dengan Pre-script	Membuat prescript pada job	date /home/yehezkiel/FOLDER_DEMO/tes.t	Pre-script dieksekusi sebelum backup	"[WORKER 77] Menjalankan Pre-Script..."	✓ Pass
Backup dengan Post-script	Membuat Postscript pada job	rm /home/yehezkiel/FOLDER_DEMO/tes.txt	Post-script dieksekusi setelah backup	"[WORKER 77] Menjalankan Post-Script..."	✓ Pass
Parallel job run	Schedule 3 job pada waktu yang sama	Job ID 6, 9 dan 10	Tidak ada gangguan pada ketiga job saat di eksekusi	"[WORKER 6] Job Post-Test: Memulai Eksekusi 3 Fase..." "[WORKER 9] Job Post-Test: Memulai Eksekusi 3 Fase..." "[WORKER 10] Job Post-Test: Memulai Eksekusi 3 Fase..."	✓ Pass

4.3.4 Pengujian Operasi restore

Pengujian operasi restore bertujuan untuk menjamin fungsionalitas sistem dalam melakukan pemulihan data jika terjadi insiden pada server lokal. Tahapan ini menguji kemampuan orkestrator rclone dalam menarik file atau folder tertentu dari cloud storage dan menempatkannya kembali ke direktori lokal yang ditentukan. Aspek yang ditekankan

dalam pengujian ini adalah ketepatan jalur penyimpanan, penanganan konflik file (*overwrite*), dan integritas file hasil pemulihan.

Tabel 4. 6 Pengujian Operasi restore

Fungsi yang diuji	Skenario Pengujian	Data Input	Hasil Yang diharapkan	Hasil Pengujian (Observasi)	Status
Restore File	Melakukan restore dengan objek file	Source Path:Restore_Tes/kiel.py Drive: Kampus Destination Path: restore_test	Kiel.py dipindahkan ke folder restore.py	"[WORKER 0] Job Restore-kampus: Memulai Eksekusi 3 Fase..."	✓ Pass
Restore Folder	Melakukan restore dengan objek folder	SourcePath : Restore_Tes Drive: Kampus DestinationPath: restore_test	Folder Restore_Tes dicopy ke direktori restore_test	"[WORKER 0] Job Restore-kampus: Memulai Eksekusi 3 Fase..."	✓ Pass
Overwrite function	Melakukan restore pada file/folder yg sudah ada	Source Path:kiel.py Drive: Kampus Destination Path: restore_test	File lama di overwrite	[Observasi] "Nama file tetap tapi isinya berubah"	✓ Pass
Auto create directory	Melakukan restore pada direktoru	Source Path:A.txt Drive: Kampus Destination Path: restore_test/FolderTes	Direktori FolderTes dibuat otomatis	[Observasi] " Direktori FolderTes dibuat dan target di copy"	✓ Pass

4.3.5 Pengujian Eksekusi Script

Fitur *Script Runner* yang melibatkan Fase *Pre-Script* dan *Post-Script* merupakan komponen unik yang memberikan fleksibilitas pada sistem. Pengujian ini dilakukan untuk melihat sejauh mana backend Golang mampu mengeksekusi perintah *shell* kustom secara berurutan sebelum dan sesudah proses transfer data. Narasi pengujian ini akan menyoroti sinkronisasi antara proses kompresi data, ekspor database, dan pembersihan file temporer dengan mesin utama rclone.

Tabel 4. 7 Pengujian Eksekusi Script

Fungsi yang diuji	Skenario Pengujian	Data Input	Hasil yang diharapkan	Hasil Pengujian	Status
Pre-Script Database Export	Menginput script untuk dump database	mysqldump -u yehezkiel -p kiel tugas_akhir > /tmp/backup.sql	Database di dump dengan format .sql	[Observasi] File .sql tercipta di folder /tmp	✓ Pass
Pre-Script Folder Compression	Menginput script untuk men-ZIP folder	zip -r /home/yehezkiel/ FOLDER_DEMO/TES.zip /var/www/html	Seluruh folder /html dikompresi	[Observasi]File html.zip tersedia	✓ Pass

Fungsi yang diuji	Skenario Pengujian	Data Input	Hasil yang diharapkan	Hasil Pengujian	Status
			menjadi satu file .zip	di direktori /tmp	
Pre-script Split Backup	Menginput script untuk memecah file jadi 3 bagian	`TOTAL_SIZE=$(stat -c%s "/tmp/data.zip"); PART_SIZE=$((TOTAL_SIZE / 3 + 1)); split -b $PART_SIZE -d --additional-suffix=.part /tmp/data.zip /tmp/backup_`	File dipecah jadi 3 bagian dengan nama backup_00.part, backup_01.part, dan backup_02.part	[Observasi] File terpecah menjadi 3 bagian dengan suffix .part	✓ Pass
Post-Script Cleanup Temporary File	Menginput script untuk menghapus file hasil dump	`rm /tmp/backup.sql`	File sampah hasil pre-script terhapus	[Observasi] File backup.sql terhapus dari folder tmp/	✓ Pass
Post-Script Local Directory Reset	Menginput script untuk membersihkan folder staging	`rm -rf /tmp/staging/*`	Membersihkan folder(*staging*)	[Observasi] Folder /tmp/staging kosong	✓ Pass

4.3.6 Pengujian Operation Mode

Pengujian pada modul mode operasional bertujuan untuk memverifikasi ketepatan logika sistem dalam menangani dua metode sinkronisasi data yang berbeda, yaitu mode COPY dan mode SYNC. Pengujian ini sangat penting karena kedua mode memiliki perilaku yang kontras terhadap integritas data di sisi tujuan (*destination*). Mode COPY diuji untuk memastikan sistem hanya menambah file baru tanpa memodifikasi file asing di cloud, sementara mode SYNC diuji untuk memastikan konsistensi penuh di mana folder tujuan dipaksa identik dengan sumber. Validasi dilakukan dengan mengamati perubahan struktur file pada dashboard Google Drive setelah setiap proses eksekusi selesai.

Tabel 4. 8 Pengujian Operation Mode

Fungsi yang diuji	Skenario Pengujian	Data Input	Hasil yang diharapkan	Hasil Pengujian	Status
Backup Mode: COPY	Melakukan Backup dengan aditif	`Source: kiel.py` `Mode: COPY`	File baru terupload, file lama di remote tidak dihapus	`Job COPY: Memulai Eksekusi 3 Fase`	✓ Pass
Backup Mode: SYNC	Melakukan backup dengan destruktif	`Source: Kiel.py` `Mode: SYNC`	Folder remote identik dengan lokal.	`Job SYNC: Memulai Eksekusi 3 Fase...`	✓ Pass
Integrity Check [COPY]	Verifikasi penambahan file	`Source: Kiel.py` `Mode: COPY`	File di remote bertambah	[Observasi] File	✓ Pass

			dengan versi terbaru	bertambah dengan timestamp sebagai pembeda	
Integrity Check [SYNC]	Verifikasi perubahan konten file	Source: Kiel.py Mode: SYNC	Isi File di remote berubah sesuai versi terbaru	[Observasi] File di overwrite ke versi terbaru	✓ Pass

4.3.7 Pengujian Retensi Data

Pengujian kebijakan retensi data difokuskan pada fungsionalitas optimasi penyimpanan cloud melalui mekanisme penghapusan otomatis. Sesuai dengan konfigurasi *Max Retention*, pengujian ini memverifikasi kemampuan sistem dalam memantau jumlah salinan backup yang tersedia di remote dan melakukan tindakan pembersihan (*purge*) terhadap versi tertua jika batas maksimal telah terlampaui. Hal ini penting untuk memastikan penggunaan kuota Google Drive tetap efisien dan teratur tanpa intervensi manual dari pengguna.

Tabel 4. 9 Pengujian Retensi Data

Fungsi yang diuji	Skenario Pengujian	Data Input	Hasil yang diharapkan	Hasil Pengujian	Status
Retention Limit	Membuat job dengan retention 7	Source: tes.txt Destination: RETENSI/Limit7 Max_retention: 7 Backup File = 0	Source: tes.txt Destination: RETENSI/Limit1 Max_retention: 1 Backup File = 0	"Job created - ID: 89, Mode: copy, OperationMode: BACKUP, MaxRetention: 7"	✓ Pass
Retention Trigger	Jalankan backup dengan retention 7	Source: tes.txt Destination: RETENSI/ Limit7 Max_retention: 7 Backup File = 0	7 backup tersimpan di GDrive	"[Round Robin] Found 7 backup items (limit: 7)"	✓ Pass
Continuous Retention	Jalankan sampai 10 kali	Source: tes.txt Destination: RETENSI/ Limit7 Max_retention: 7 Backup File = 7	Hanya 7 backup terbaru tersimpan (backup tertua terhapus)	"[Round Robin] Deleting 1 oldest backup(s)"	✓ Pass
Min Retention Limit	Retentioin = 1	Source: tes.txt Destination: RETENSI/Limit1 Max_retention: 1 Backup File = 0	Hanya 1 backup terbaru tersimpan	"[Round Robin] Found 1 backup items (limit: 1) [Round Robin] Deleting 1 oldest backup"	✓ Pass
File Checking	Retention > Existing Backups	Source: tes.txt Destination: RETENSI/Limit5 Max_retention: 5 Backup File = 2	Tidak ada penghapusan (2 < 5, masih dalam limit)	"[Round Robin] No cleanup needed (2/5)"	✓ Pass
Retention Update	Update Retention (8 → 3)	Source: tes.txt Destination: RETENSI/Limit8 Max_retention:3 Backup File = 8	Simpan 3 terbaru hapus 5 terlama	"[Round Robin] Found 8 backup items (limit: 3)	✓ Pass

				[Round Robin] Deleting 6 oldest backup(s)"	
Retention Update	Update Retention (3 → 10)	Source: tes.txt Destination: RETENSI/Limit3 Max_retention: 10 Backup File = 3	Tidak ada penghapusan (3 < 10, masih dalam limit)	"[Round Robin] Found 3 backup items (limit: 10) [Round Robin] No cleanup needed (3/10)"	✓ Pass
Management File	Manual Deletion by User (User manually deletes 3)	Source: tes.txt Destination: RETENSI/Limit3 Max_retention: 10 Backup File = 4 Manual Deleted file = 3	Sistem hanya mendeteksi 5 file, simpan semua (5 < 8)	Round Robin] Found 1 backup items (limit: 10) [Round Robin] No cleanup needed (1/10)	✓ Pass

4.3.8 Pengujian Penanganan Error

Sistem yang tangguh tidak hanya dinilai dari fungsinya saat berjalan normal, tetapi juga bagaimana ia bereaksi terhadap kondisi tidak ideal. Pengujian *error handling* dilakukan dengan mensimulasikan berbagai kegagalan, seperti jalur file yang salah, kegagalan perintah skrip, hingga terputusnya koneksi internet. Tujuan utamanya adalah memastikan sistem menerapkan prinsip *fail-fast*, di mana proses segera dihentikan saat terjadi error untuk mencegah korupsi data, serta memberikan pesan kesalahan yang informatif kepada pengguna.

Tabel 4. 10 Pengujian Penanganan Error

Fungsi yang diuji	Skenario Pengujian	Data Input	Hasil yang diharapkan	Hasil Pengujian	Status
Path Validation	Menjalankan backup dengan path lokal yang tidak ditemukan	source_path: "/root/data_palsu"	Rclone mengembalikan error dan sistem mencatat kegagalan autentikasi	"[FAILED] Job 88: FAIL_SOURCE_CHECK"	✓ Pass
Pre-Script execution	Simulasi kesalahan perintah pada fase Pre-Script	pre_script: "rm /file/gaib"	Eksekusi segera dihentikan (*Fail-Fast*), status menjadi FAIL_PRE_SCRIPT	"[FAILED] Job 88: FAIL_PRE_SCRIPT"	✓ Pass
Post-Script execution	Simulasi kesalahan perintah pada fase Pre-Script	remote_name: "Gdrive_Expired"	Eksekusi segera dihentikan (*Fail-Fast*), status menjadi FAIL_PRE_SCRIPT	[FAILED] Job 88: FAIL_POST_SCRIPT	✓ Pass
Validasi Jadwal	Menginput ekspresi cron dengan format yang tidak standar	cron: "jam 2 siang"	Backend menolak penyimpanan karena format penjadwalan tidak valid	"ERROR: Gagal parsing cron expression"	✓ Pass

Konflik Retensi	Dua job dengan folder tujuan sama namun batas retensi berbeda	`Job A (MR:3), Job B (MR:10), Target: /Backup`	Kebijakan retensi terkecil mendominasi (menghapus folder hingga sisa 3)	`[Observasi]: Setelah Job A jalan, folder di GDrive berkurang menjadi 3 meskipun Job B baru saja dijalankan.`	✓ Pass

4.3.9 Pengujian Monitoring

Pengujian pada modul monitoring bertujuan untuk memverifikasi kemampuan sistem dalam menyajikan visualisasi data *real-time* mengenai kondisi *cloud storage* yang terhubung. Pengujian ini sangat penting untuk memastikan integrasi antara *Service Layer* Golang dengan perintah rclone about berjalan secara akurat. Parameter yang diuji meliputi ketepatan penghitungan kapasitas (Total, Used, dan Free GB), status konektivitas remote, serta kemampuan sistem dalam memberikan pesan peringatan (*warning*) jika penggunaan penyimpanan telah mendekati ambang batas kritis. Melalui pengujian ini, diharapkan administrator dapat mengambil keputusan pemeliharaan data secara proaktif berdasarkan informasi yang ditampilkan pada dashboard.

Tabel 4. 11 Pengujian Monitoring

Fungsi yang diuji	Skenario Pengujian	Data Input	Hasil yang diharapkan	Hasil Pengujian	Status
Remote Checking	Melakukan pengecekan status remote	`remoteName: "Kampus"`	Metrik GB terhitung, dan jumlah job aktif terdata	`[SYNC] Found 3 remote(s) in rclone.conf: [kampus sthvly zkyle]`	✓ Pass
Sync Rclone.conf	Menambahkan remote baru pada file rclone.conf	`Nama remote baru: Work-Drive`	Sistem mendeteksi perbedaan dan menambahkan remote baru ke database secara otomatis	`[SYNC] Found 1 new remote(s): [Work-Drive], Adding 'Work-Drive' to database, Successfully added 'Work-Drive'`	✓ Pass
Email Extraction	Mengambil email pemilik dari file rclone.conf	`remoteName: "Kampus"`	Sistem berhasil mencari section remote, mem-parsing token JSON, dan melakukan call API.	`mail ditemukan untuk kampus: kilerumapea @gmail.com`	✓ Pass
Storage Check	Mengambil data storage remote	`Used: 13GB / 15GB`	Metrik GB terhitung dan ditampilkan	`”kampus: 3.84 GB / 15.00 GB (25.6%)”`	✓ Pass
Storage Warning	Peringatan kapasitas penyimpanan kritis	`Used: 13GB / 15GB (86.6%)`	Sistem memberikan pesan peringatan (*Warning Threshold* 85%)	`”Storage terisi 98.7%. PERINGATAN!”`	✓ Pass

4.3.10 Pengujian Instalasi

Pengujian instalasi dilakukan untuk memverifikasi keandalan skrip otomatis install.sh dalam menyiapkan lingkungan operasional server secara mandiri (*self-contained*). Fokus utama pengujian ini mencakup deteksi otomatis terhadap dependensi perangkat lunak esensial (seperti Go, Node.js, MariaDB, dan Rclone), validasi pembuatan file konfigurasi .env berbasis input dinamis pengguna, serta proses kompilasi kode sumber backend menjadi biner eksekusi. Tahapan ini bertujuan untuk menjamin bahwa sistem G-Backup dapat diimplementasikan pada berbagai infrastruktur server berbasis Linux dengan intervensi manual yang minimal namun tetap menghasilkan konfigurasi

Tabel 4. 12 Pengujian Instalasi

Fungsi yang Diuji	Skenario Pengujian	Data Input	Hasil yang Diharapkan	Hasil Pengujian	Status
check_go_version()	Pengecekan versi Go yang sudah terinstall	`Go version 1.23.5`	Menampilkan "✓ Go 1.23 (compatible)" dan return 0	`✓ Go 1.23 (compatible)`	✓ Pass
install_go_official()	Instalasi Go pada arsitektur x86_64	`uname -m = x86_64`	Download go1.23.5.linux-amd64.tar.gz, ekstrak ke /usr/local/go, update PATH, verifikasi instalasi berhasil	`Downloading Go 1.23.5... Installing... ✓ Go 1.23.5 installed`	✓ Pass
check_node_version()	Pengecekan Node.js >= 18	`node -v = v20.10.0`	Menampilkan "✓ Node.js v20 (compatible with Vite 5)" dan return 0	`✓ Node.js v20 (compatible with Vite 5)`	✓ Pass
install_node_official()	Instalasi Node.js LTS via nvm	`nvm install --lts`	nvm terinstall, Node.js LTS terinstall, alias default dibuat, menampilkan versi yang terinstall	`Installing nvm... Installing Node.js LTS... ✓ Node.js v20.11.0 installed`	✓ Pass
update_node_version()	User memilih update Node.js	`Input: y`	Memanggil install_node_official(), Node.js ter-update ke versi LTS, return 0	`Vite 5 requires Node.js 18 or higher Update Node.js to LTS version? (y/n): y ✓ Node.js v20.11.0 installed`	✓ Pass
NPM Check	Pengecekan npm yang sudah terinstall	`npm --version`	Menampilkan "✓ npm vX.X.X"	`Checking npm... ✓ npm v10.2.4`	✓ Pass

Fungsi yang Diuji	Skenario Pengujian	Data Input	Hasil yang Diharapkan	Hasil Pengujian	Status
install_dep()	Instalasi mysql client	`mysql belum terinstall`	Instalasi default-mysql-client via apt, menampilkan "✓ mysql installed"	`Installing mysql... Downloading Done ✓ mysql installed`	✓ Pass
Rclone Installation	Instalasi rclone	`rclone belum terinstall`	Download dan instalasi via curl script, menampilkan "✓ rclone installed"	`Installing rclone... Downloading Done ✓ rclone installed`	✓ Pass
check_mysql()	MySQL server sudah running	`systemctl is-active mysql`	Menampilkan "✓ MySQL server running" dan return 0	`✓ MySQL server running ✓ MySQL ready`	✓ Pass
MySQL Installation	Instalasi MySQL dengan kredensial baru	`Username: admin Password: admin123`	MySQL server terinstall, service aktif dan enable, user dibuat dengan privileges lengkap	`MySQL not found. Installing MySQL server... Configure MySQL Administrator Account MySQL username [root]: admin MySQL password for 'admin': ****** Confirm password: ****** Installing MySQL server... ✓ MySQL server installed Creating user 'admin'... ✓ User 'admin' created`	✓ Pass
MySQL Authentication	Login dengan kredensial existing	`Username: root Password: rootpass`	Autentikasi berhasil, menampilkan "✓ Authentication successful"	`Enter your existing MySQL credentials MySQL username [root]: root MySQL password: ******** ✓ Authentication successful ✓ Will use MySQL user: root`	✓ Pass
Database Creation	Membuat database baru	`DB_NAME: backup_system DB_USER: root DB_PASS: rootpass`	Database backup_system dibuat, koneksi test berhasil	`Setting up database... ✓ Database 'backup_system' created ✓ Database`	✓ Pass

Fungsi yang Diuji	Skenario Pengujian	Data Input	Hasil yang Diharapkan	Hasil Pengujian	Status
				`connection test passed`	
Rclone Config Path	Deteksi dan pembuatan direktori rclone	`Current user: testuser`	Path: /home/testuser/.config/rclone/rclone.conf, direktori dibuat jika belum ada	`Storage Configuration Created rclone config directory` ✓ `Rclone path: /home/testuser/.config/rclone/rclone.conf`	✓ Pass
is_port_available()	Mencari port yang tersedia	`Port 8080 available`	Return true, APP_PORT = 8080	`Finding available port... Done` ✓ `Service port: 8080`	✓ Pass
JWT Secret Generation	Generate JWT secret key	`openssl rand -base64 32`	JWT secret 32 karakter base64 ter-generate, disimpan ke .env	✓ `Security token generated`	✓ Pass
Environment File Creation	Membuat file Backend/.env	`DB_USER, DB_PASS, DB_HOST, DB_PORT, DB_NAME,`	File .env dibuat dengan semua konfigurasi yang diperlukan	`Creating configuration file...` ✓ `Configuration created`	✓ Pass
Go Build	Build aplikasi Go Backend	`go build -o app ./cmd/main.go`	Dependencies di-download, go mod tidy dijalankan, binary "app" ter-compile, menampilkan "✓ Application built"	`Building application... Downloading dependencies... Done Compiling... Done` ✓ `Application built`	✓ Pass
Frontend Installation	Install dependencies Frontend	`npm install`	package.json terbaca, dependencies terinstall, menampilkan "✓ Frontend ready"	`Installing frontend... Installing packages... Done` ✓ `Frontend ready`	✓ Pass
get_network_ip()	Deteksi IP jaringan (single IP)	`Single network IP: 192.168.1.100`	Return IP 192.168.1.100, status 0	`IP: 192.168.1.100 terdeteksi`	✓ Pass
get_network_ip()	Deteksi IP jaringan (multiple IPs)	`IPs: 192.168.1.100, 10.0.0.50`	Prioritas IP 192.168.x.x, return 192.168.1.100	`IP: 192.168.1.100 dipilih sebagai primary`	✓ Pass
Installation Complete	Menampilkan URL akses sistem	`SERVER_IP: 192.168.1.100 APP_PORT: 8080`	Menampilkan "Access URL: http://192.168.1.100:8080" dan note reload shell	✓ `System successfully installed` `Access URL: http://192.168.1.100:8080`	✓ Pass

4.4 Pembahasan

Berdasarkan hasil implementasi dan pengujian yang telah dipaparkan pada sub-bab sebelumnya, sistem G-Backup telah berhasil dikembangkan sebagai solusi manajemen pencadangan data berbasis web. Pada bagian ini, akan dibahas secara mendalam mengenai aspek teknis, keputusan arsitektur, dan mekanisme logika yang diterapkan untuk menyelesaikan permasalahan manajemen backup manual.

4.4.1 Implementasi Rclone untuk Otomasi Backup

Tantangan utama dalam pengembangan sistem ini adalah menjembatani komunikasi antara antarmuka web (*stateless*) dengan utilitas Rclone yang berbasis baris perintah (*Command Line Interface* / CLI). Sistem ini mengatasi hal tersebut dengan menerapkan mekanisme *wrapper* menggunakan pustaka bawaan Go, yaitu os/exec.
Alih-alih menjalankan perintah Rclone secara langsung di terminal, *Service Layer* pada *backend* bertindak sebagai orkestrator yang menyusun argumen perintah secara dinamis berdasarkan input pengguna dari *frontend* (seperti pemilihan *remote* tujuan dan jalur sumber).

Keunggulan utama dari implementasi ini adalah penggunaan Goroutines (konkurensi ringan pada Go). Saat pengguna memicu *job* backup, sistem menjalankan proses Rclone di dalam *goroutine* terpisah. Hal ini memungkinkan server untuk segera mengembalikan respons HTTP ke antarmuka pengguna (*non-blocking*), sementara proses backup yang memakan waktu lama tetap berjalan di latar belakang. Mekanisme ini memastikan antarmuka web tetap responsif dan tidak mengalami *timeout* meskipun sedang menangani transfer data berukuran besar.

4.4.2 Penerapan Arsitektur Layered

Untuk menjamin skalabilitas dan kemudahan pemeliharaan kode (*maintainability*), pengembangan *backend* tidak dilakukan secara monolitik dalam satu file, melainkan menerapkan pola Layered Architecture yang memisahkan tanggung jawab ke dalam tiga lapisan utama:

- **Handler Layer (Controller):** Bertanggung jawab hanya untuk menerima permintaan HTTP, memvalidasi input JSON, dan mengembalikan respons standar. Lapisan ini tidak mengandung logika bisnis.

- **Service Layer:** Merupakan inti dari logika bisnis sistem. Lapisan ini menangani orkestrasi tugas, seperti penentuan apakah sebuah *job* harus dijalankan secara manual atau terjadwal, serta pemanggilan eksekutor sistem.
- **Repository Layer:** Bertanggung jawab khusus untuk interaksi dengan basis data menggunakan ORM (GORM). Lapisan ini mengisolasi *query* SQL dari logika bisnis.

Penerapan pola ini terbukti efektif dalam pengujian, di mana perubahan pada logika validasi data di *Service Layer* tidak mempengaruhi struktur kode pada *Repository* maupun *Handler*, sehingga meminimalisir risiko *bug* regresi saat pengembangan fitur baru.

4.4.3 Manajemen efisiensi penyimpanan dengan algoritma rotasi

Salah satu masalah utama dalam sistem backup konvensional adalah penumpukan data yang menyebabkan penyimpanan *cloud* cepat penuh dan harus dilakukan pembersihan secara manual. Sistem ini mengatasi masalah tersebut melalui fitur Retensi data yang menerapkan algoritma rotasi RR (Round Robin).

Mekanisme ini bekerja pada fase *Pre-Execution* untuk Rclone di *backend*. Sebelum melakukan *backup* baru, sistem melakukan inspeksi terhadap direktori tujuan di Google Drive menggunakan perintah rclone lsjson. Data JSON yang diterima kemudian diurutkan berdasarkan waktu modifikasi (*timestamp*). Jika jumlah file melebihi batas MaxRetention yang ditentukan pengguna, sistem secara otomatis menghapus file dengan *timestamp* tertua. Pembahasan hasil pengujian menunjukkan bahwa fitur ini berhasil menjaga kuota penyimpanan tetap stabil tanpa memerlukan intervensi penghapusan manual oleh administrator.

4.4.4 Parsing Data Sistem

Sistem G-Backup memiliki kemampuan interoperabilitas untuk membaca dan mengolah data yang dihasilkan oleh proses eksternal (*subprocess*). Hal ini terlihat pada mekanisme pencatatan *log*. Rclone secara bawaan mengeluarkan *output* teks yang tidak terstruktur di terminal .Untuk menyajikan data yang informatif kepada pengguna, *backend* mengimplementasikan fungsi *parser* menggunakan Regular Expressions(Regex). Sistem menangkap aliran teks dari Rclone, mendeteksi pola string tertentu (seperti "Transferred: X MB" dan durasi waktu), lalu mengonversinya menjadi tipe data numerik (int64 untuk bytes dan int untuk detik).Dengan mekanisme ini, data yang tersimpan di tabel logs bukan sekadar teks mentah, melainkan data yang dapat diolah kembali. Hal ini memungkinkan *frontend*

untuk memvisualisasikan data tersebut dalam format yang lebih mudah dibaca manusia, seperti konversi otomatis satuan Byte ke GB atau TB pada antarmuka tabel riwayat.

4.4.5 Sistem Monitoring

Aspek yang dibahas selanjutny adalah mekanisme monitoring. Sistem tidak melakukan pengecekan ke API Google Drive setiap kali pengguna me-refresh halaman dashboard, karena hal tersebut dapat menyebabkan latensi tinggi dan risiko terkena pembatasan kuota API (Rate Limiting).

Sebagai gantinya, sistem menerapkan pola Daemon Worker menggunakan time.Ticker pada Go. Proses ini berjalan secara mandiri di latar belakang setiap interval waktu tertentu (1 menit) untuk memperbarui status koneksi dan sisa penyimpanan semua remote drive. Hasil pengecekan disimpan ke dalam database lokal. Dengan demikian, saat pengguna mengakses dashboard, sistem hanya perlu mengambil data terakhir dari database lokal yang menghasilkan waktu muat halaman yang cepat.

4.4.6 Otomasi Instalasi dengan Shell Script

Untuk mempermudah proses deployment sistem G-Backup, dikembangkan script instalasi otomatis berbasis Bash yang menangani seluruh tahap instalasi dependencies, konfigurasi database, dan build aplikasi. Script ini menerapkan beberapa prinsip engineering yang penting:

- **Deteksi Versi Dependencies Otomatis:** Script mengimplementasikan fungsi check_go_version() dan check_node_version() yang mampu mendeteksi versi software yang sudah terinstall dan membandingkannya dengan requirement minimum sistem. Untuk Go, sistem memeriksa apakah versi >= 1.20, sedangkan untuk Node.js memeriksa kompatibilitas dengan Vite 5 (>= v18). Mekanisme ini mencegah instalasi redundan dan menjamin kompatibilitas teknologi yang digunakan

- **Instalasi Multi-Source dengan Fallback Mechanism:** Sistem menerapkan strategi instalasi berlapis dengan prioritas sumber official. Untuk Go, script akan mencoba mengunduh binary official dari golang.org terlebih dahulu. Jika gagal (misalnya karena network issue atau arsitektur tidak didukung), sistem secara otomatis melakukan fallback ke instalasi melalui package manager sistem (apt).

Strategi yang sama diterapkan pada Node.js dengan prioritas nvm (Node Version Manager) kemudian fallback ke NodeSource repository.

- **Konfigurasi Database Interaktif dengan Validasi:** Script mengimplementasikan wizard interaktif untuk setup MySQL yang mencakup validasi input multi-level. Password MySQL divalidasi dengan dua kriteria: tidak boleh kosong. Sistem juga menerapkan mekanisme konfirmasi password untuk mencegah kesalahan input. Untuk autentikasi MySQL existing, disediakan retry mechanism dengan maksimal 3 percobaan untuk menghindari account lockout akibat kesalahan input berulang.

- **Port Allocation dengan Conflict Detection:** Fungsi is_port_available() menggunakan utilitas lsof untuk mendeteksi port yang sedang digunakan oleh proses lain. Script akan mencoba port-port default (8080, 3000, 3001, dll) secara berurutan.

4.4.7 Implementasi Konkurensi Berbasis Goroutines

Implementasi Goroutines pada proyek G-Backup difokuskan sebagai solusi untuk menangani proses transfer data yang memakan waktu lama agar tidak menghambat kinerja aplikasi secara keseluruhan. Secara teknis, sistem menerapkan Goroutines pada saat pemanggilan Rclone di *Service Layer*; ketika sebuah perintah backup dieksekusi, sistem menjalankan fungsi tersebut di jalur terpisah menggunakan instruksi konkurensi. Hal ini menciptakan mekanisme non-blocking yang memungkinkan *Backend* tetap aktif melayani permintaan lainnya dari pengguna, seperti memantau status atau melihat log, sementara proses pengunggahan file ke Google Drive tetap berlangsung secara stabil di latar belakang tanpa menyebabkan antarmuka web mengalami pembekuan (*freezing*).

Selain untuk proses backup, Goroutines diimplementasikan sebagai Daemon Worker yang bertugas melakukan pemantauan status penyimpanan secara berkala. Proses ini berjalan secara mandiri sejak aplikasi dimulai untuk melakukan pemeriksaan koneksi dan sisa kuota pada Google Drive tanpa mengganggu jalur komunikasi utama antara *Frontend* dan *Backend*. Dengan pola ini, sistem G-Backup dapat menangani banyak tugas sekaligus secara paralel, di mana setiap *job* backup yang aktif dijalankan dalam jalurnya masing-masing. Implementasi ini menjamin bahwa sistem tetap responsif, terutama saat menangani beban kerja tinggi atau banyak jadwal backup yang berjalan secara bersamaan pada server.

BAB V
KESIMPULAN DAN SARAN

Bab ini menyajikan kesimpulan dari seluruh rangkaian proses penelitian, perancangan, implementasi, dan pengujian sistem G-Backup, serta memberikan saran pengembangan yang dapat dijadikan acuan untuk peningkatan kualitas dan keberlanjutan sistem di masa mendatang.

5.1 Kesimpulan

Pengembangan sistem G-Backup telah berhasil menghasilkan sebuah aplikasi manajemen pencadangan data yang fungsional dan responsif. Implementasi *backend* menggunakan bahasa pemrograman Go dengan pola *Layered Architecture* terbukti mampu menyediakan fondasi sistem yang mudah dikelola. Pemanfaatan fitur Goroutines memungkinkan sistem menjalankan proses *backup* dan *monitoring* secara asinkron (*non-blocking*), sehingga antarmuka pengguna tetap responsif meskipun sistem sedang melakukan transfer data berukuran besar ke layanan penyimpanan awan.Integrasi sistem dengan Google Drive melalui utilitas Rclone telah terimplementasi menggunakan teknik *command wrapper*. Mekanisme ini memungkinkan aplikasi untuk menyusun instruksi CLI secara dinamis dan melakukan *parsing* data melalui *Regular Expressions* (Regex) untuk menghasilkan log aktivitas data.

Dari sisi operasional, sistem mampu mengelola kapasitas penyimpanan secara mandiri melalui fitur *Retention Policy* dengan menerapkan algoritma rotasi RR (Round Robin). Mekanisme ini secara efektif menjaga jumlah arsip *backup* tetap dalam batas yang ditentukan, sehingga penggunaan ruang penyimpanan tetap efisien tanpa memerlukan intervensi manual dari administrator.Seluruh aktivitas sistem, mulai dari status koneksi *remote*, sisa kapasitas penyimpanan, hingga riwayat eksekusi tugas, disajikan secara informatif melalui *dashboard* berbasis Vue.js. Secara keseluruhan, sistem G-Backup telah memenuhi tujuan penelitian sebagai solusi otomatisasi *backup*, efisien dan mudah dioperasikan melalui antarmuka berbasis web.

5.2 Saran

Berdasarkan evaluasi terhadap batasan dan potensi sistem G-Backup saat ini, terdapat beberapa rekomendasi pengembangan yang dibagi menjadi aspek fungsionalitas sistem dan aspek operasional infrastruktur:

1. **Integrasi Otentikasi OAuth2 via Antarmuka Web:** Keterbatasan utama sistem saat ini adalah proses penambahan *remote storage* (Google Drive) yang masih memerlukan interaksi CLI di server. Pengembangan selanjutnya diharapkan dapat mengintegrasikan alur OAuth2 langsung di aplikasi web, sehingga pengguna dapat menghubungkan akun *cloud* mereka semudah melakukan "Login with Google".
2. **Sistem Notifikasi Proaktif (*Alerting*):** Saat ini, pemantauan status dilakukan secara pasif melalui *dashboard*. Disarankan untuk membangun layanan notifikasi (*Push Notification*) yang terintegrasi dengan Telegram Bot, Slack, atau Email SMTP. Fitur ini akan mengirimkan peringatan *real-time* kepada administrator jika terjadi kegagalan *backup* kritis atau jika kapasitas penyimpanan *cloud* mendekati ambang batas.
3. **Fitur Export Laporan (Audit Trail):** Untuk kebutuhan audit IT, disarankan menambahkan fitur untuk mengekspor riwayat *log* backup ke dalam format dokumen standar seperti PDF atau Excel (CSV). Hal ini akan memudahkan administrator dalam membuat laporan pertanggungjawaban keamanan data bulanan/tahunan.
4. **Fitur Pelaporan dan Audit Trail:** Untuk kebutuhan dokumentasi dan audit IT, sistem dapat dikembangkan agar mampu mengekspor riwayat log *backup* ke dalam format dokumen standar seperti PDF atau Excel (CSV). Hal ini akan memudahkan administrator dalam menyusun laporan data secara periodik.
5. **Penerapan Enkripsi Data Sisi Klien:** Sebagai peningkatan aspek keamanan, pengembangan di masa depan dapat menambahkan fitur enkripsi otomatis menggunakan algoritma AES-256 sebelum data dikirim ke *cloud storage*. Hal ini menjamin bahwa data yang tersimpan di Google Drive tidak dapat dibaca oleh pihak manapun tanpa kunci dekripsi yang tepat, bahkan jika akun *cloud* tersebut berhasil disusupi.

Daftar Pustaka

[21][1] J. F. Andry, "PENGEMBANGAN APLIKASI BACKUP DAN RESTORE SECARA AUTOMATISASI MENGGUNAKAN SDLC UNTUK MENCEGAH BENCANA," *J. Muara Sains Teknol. Kedokt. Dan Ilmu Kesehat.*, vol. 1, no. 1, May 2017, doi: 10.24912/jmstkik.v1i1.389.

[2] A. Nandugudi, C. Nuessle, G. Challen, E. Miluzzo, and Y.-F. Chen, "The PocketLocker Personal Cloud Storage System," in *Proceedings of the 6th International Conference on Mobile Computing, Applications and Services*, Austin, United States: ICST, 2014. doi: 10.4108/icst.mobicase.2014.257782.

[3] M. Z. Hasan, N. Sarwar, I. Alam, M. Z. Hussain, A. A. Siddiqui, and A. Irshad, "Data Recovery and Backup management: A Cloud Computing Impact," in *2023 IEEE International Conference on Emerging Trends in Engineering, Sciences and Technology (ICES&T)*, Bahawalpur, Pakistan: IEEE, Jan. 2023, pp. 1–6. doi: 10.1109/ICEST56843.2023.10138852.

[4] A. Z. Abualkishik, A. A., and Y. Gulzar, "Disaster Recovery in Cloud Computing Systems: An Overview," *Int. J. Adv. Comput. Sci. Appl.*, vol. 11, no. 9, 2020, doi: 10.14569/IJACSA.2020.0110984.

[5] Izra Noor Zahara Aliya, Moh Azzam Priyanto, Rizka Fiddiyansyah, and Siti Mukaromah, "REKOMENDASI PENGGUNAAN DATABASE TERSENTRALISASI TERINTEGRASI USER ADMINISTRATION DAN BACKUP RECOVERY PADA UMKM," *Pros. Semin. Nas. Teknol. Dan Sist. Inf.*, vol. 3, no. 1, pp. 423–432, Nov. 2023, doi: 10.33005/sitasi.v3i1.347.

[6] R. Maher and O. A. Nasr, "DropStore: A Secure Backup System Using Multi-Cloud and Fog Computing," *IEEE Access*, vol. 9, pp. 71318–71327, 2021, doi: 10.1109/ACCESS.2021.3078887.

[7] E. Kamau, T. Myllynen, S. D. Mustapha, G. O. Babatunde, and A. A. Alabi, "A Conceptual Model for Real-Time Data Synchronization in Multi-Cloud Environments," *Int. J. Multidiscip. Res. Growth Eval.*, vol. 5, no. 1, pp. 1139–1150, 2024, doi: 10.54660/.IJMRGE.20247.5.1.1139-1150.

[8] P. Wang, C. Zhao, W. Liu, Z. Chen, and Z. Zhang, "Optimizing Data Placement for Cost Effective and High Available Multi-Cloud Storage," *Comput. Inform.*, vol. 39, no. 1–2, pp. 51–82, 2020, doi: 10.31577/cai_2020_1-2_51.

[9] M. Sravani and M. Krishna Murthy, "REDUCTION OF DATA LEAKAGE IN DISTRIBUTED CLOUD STORAGE SYSTEMS USING DISTRIBUTED CLOUD GUARD (DCG)," *Indian J. Comput. Sci. Eng.*, vol. 14, no. 3, pp. 444–450, June 2023, doi: 10.21817/indjcse/2023/v14i3/231403014.

[10] J. M. Aslam and K. M. Kumar, "Enhancing cloud data security: User-centric approaches and advanced mechanisms," *Sci. Temper*, vol. 15, no. 01, pp. 1784–1789, Mar. 2024, doi: 10.58414/SCIENTIFICTEMPER.2024.15.1.29.

[11] L. Lin, L. Pan, and S. Liu, "Backup or Not: An Online Cost Optimal Algorithm for Data Analysis Jobs Using Spot Instances," *IEEE Access*, vol. 8, pp. 144945–144956, 2020, doi: 10.1109/ACCESS.2020.3014978.

[12] Department of Information Technology, KLN College of Engineering, Madurai, India, G. Ramesh, J. Logeshwaran, Department of ECE, Sri Eshwar College of Engineering, Coimbatore, India, V. Aravindarajan, and Department of Information Technology, KLN College of Engineering, Madurai, India, "A Secured Database Monitoring Method to Improve Data Backup and Recovery Operations in Cloud Computing," *BOHR Int. J. Comput. Sci.*, vol. 2, no. 1, pp. 1–7, 2023, doi: 10.54646/bijcs.019.

[13] C. Qian, Y. Huang, X. Zhao, and T. Nakagawa, "Optimal Backup Interval for a Database System with Full and Periodic Incremental Backup," *J. Comput.*, vol. 5, no. 4, pp. 557–564, Apr. 2010, doi: 10.4304/jcp.5.4.557-564.
[14] A. A. M. Syed, "Disaster Recovery and Data Backup Optimization: Exploring Next-Gen Storage and Backup Strategies in Multi-Cloud Architectures," 2024.
[15] A. Venkatesh and M. S. Eastaff, "A Study of Data Storage Security Issues in Cloud Computing".
[16] N. S. Fatonah, B. Jokonowo, and S. Dwiasnati, "PEMANFAATAN GOOGLE DRIVE SEBAGAI MEDIA PENCATATAN INVENTARIS DI KELURAHAN PONDOK KACANG TIMUR," 2021.
[17] S. Amanpreet and B. Jyoti, "Strategies for Data Backup and Recovery in the Cloud," *Int. J. Perform. Eng.*, vol. 19, no. 11, p. 728, 2023, doi: 10.23940/ijpe.23.11.p3.728735.
[18] L. Pakravan Nejad, "Mitigating Data Loss and Its Impact on Modern Software Engaging: A Case Study Approach," *J. Appl. Intell. Syst. Inf. Sci.*, vol. 4, no. 2, Dec. 2023, doi: 10.22034/jaisis.2023.418932.1072.
[19] F. Pernando *et al.*, "SIMULASI BACKUP DATA DI GOOGLE DRIVE DENGAN RCLONE PADA UBUNTU VIRTUALBOX DI SMA YAPPENDA".
[20] T. D. Putra and A. Fadjriya, "Comparison Between Simple Round Robin and Intelligent Round Robin Algorithms in CPU Scheduling," *IJARCCE*, vol. 10, no. 4, Apr. 2021, doi: 10.17148/IJARCCE.2021.10413.
[21] A. Wibowo, A. Fathimah, D. Waskito, and R. A. Aprilianto, "Cron Job Implementation for Automated Data Processing and Transfer in Cloud Infrastructure," vol. 23, no. 1, 2025.
[22] H. Sanugommula, "A Comprehensive Study on Shell Scripting and Its Role in Automation and System Management," vol. 7, no. 5, 2019.
[23] "INTRODUCTION TO GOROUTINES IN GO."
[24] L. Widyawati, M. Azwar, H. Alafi, and I. Asif, "Analysis of Rclone Implementation for File Sharing Security on OneDrive Cloud Storage," *J. Bumigora Inf. Technol. BITe*, vol. 6, no. 1, pp. 95–104, July 2024, doi: 10.30812/bite.v6i1.4082.
[25] D. Bailleul, S. Stoeckel, and S. Arnaud-Haond, "RClone: a package to identify MultiLocus Clonal Lineages and handle clonal data sets in R .," *Methods Ecol. Evol.*, vol. 7, no. 8, pp. 966–970, Aug. 2016, doi: 10.1111/2041-210X.12550.
[26] F. Breitinger, F. Breitinger, X. Zhang, X. Zhang, D. Quick, and D. Quick, "A forensic analysis of rclone and rclone's prospects for digital forensic investigations of cloud storage," *Forensic Sci. Int. Digit. Investig.*, 2022, doi: 10.1016/j.fsidi.2022.301443.

LAMPIRAN

Code Instalasi

```
#!/bin/bash

set -e

# Colors
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
RED='\033[0;31m'
NC='\033[0m'

echo -e "${BLUE}"
echo "┌──────────────────────────────────────────────────┐"
echo "│         System Installation Wizard               │"
echo "└──────────────────────────────────────────────────┘"
echo -e "${NC}"

# ================================================
# Dependency Check & Install
# ================================================
echo -e "${BLUE}System Dependencies Check${NC}"

# Progress bar function
show_progress() {
    local duration=$1
    local message=$2
    local steps=20
    local step_duration=$(echo "scale=2; $duration / $steps" | bc)

    echo -n "$message "
    for i in $(seq 1 $steps); do
        echo -n "▓"
        sleep $step_duration
    done
    echo " Done"
}
```

```
# ============================================
# Go Installation (Official, >= 1.20)
# ============================================
check_go_version() {
    if command -v go &>/dev/null; then
        local version=$(go version | grep -oP 'go\K[0-9]+\.[0-9]+' |
head -1)
        local major=$(echo $version | cut -d. -f1)
        local minor=$(echo $version | cut -d. -f2)

        # Check if version >= 1.20
        if [ "$major" -ge 1 ] && [ "$minor" -ge 20 ]; then
            echo -e "${GREEN}✓ Go $version (compatible)${NC}"
            return 0
        else
            echo -e "${YELLOW}⚠ Go $version detected (requires >=
1.20)${NC}"
            return 1
        fi
    fi
    return 1
}

install_go_official() {
    echo -e "${BLUE}Installing Go (official)...${NC}"

    # Detect architecture
    local arch=$(uname -m)
    case $arch in
        x86_64) arch="amd64" ;;
        aarch64) arch="arm64" ;;
        armv6l) arch="armv6l" ;;
        *)
            echo -e "${RED}✗ Unsupported architecture: $arch${NC}"
            return 1
            ;;
    esac

    # Get latest stable version (1.23.x)
```

```
    local go_version="1.23.5"
    local go_file="go${go_version}.linux-${arch}.tar.gz"
    local go_url="https://go.dev/dl/${go_file}"

    echo "Downloading Go ${go_version}..."
    if ! wget -q --show-progress "$go_url" -O "/tmp/${go_file}"; then
        echo -e "${RED}✗ Failed to download Go${NC}"
        return 1
    fi

    echo "Installing..."
    # Remove old installation
    sudo rm -rf /usr/local/go

    # Extract new version
    sudo tar -C /usr/local -xzf "/tmp/${go_file}"
    rm "/tmp/${go_file}"

    # Add to PATH if not already there
    if ! grep -q "/usr/local/go/bin" ~/.bashrc; then
        echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
    fi

    # Add to current session
    export PATH=$PATH:/usr/local/go/bin

    # Verify installation
    if command -v go &>/dev/null; then
        local installed_version=$(go version | grep -oP 'go\K[0-9]+\.[0-
9]+\.[0-9]+')
        echo -e "${GREEN}✓ Go ${installed_version} installed${NC}"
        return 0
    else
        echo -e "${RED}✗ Go installation failed${NC}"
        return 1
    fi
}

# Check and install Go
if ! check_go_version; then
```

```
    if ! install_go_official; then
        echo -e "${YELLOW}Falling back to apt installation...${NC}"
        sudo apt update -qq && sudo apt install -y golang-go -qq
    fi
fi

# ============================================
# Node.js Installation (Official via nvm)
# ============================================
check_node_version() {
    if command -v node &>/dev/null; then
        local version=$(node -v | sed 's/v//' | cut -d. -f1)

        # Vite 5 requires Node.js >= 18
        if [ "$version" -ge 18 ]; then
            echo -e "${GREEN}✓ Node.js v$version (compatible with Vite
5)${NC}"
            return 0
        else
            echo -e "${YELLOW}⚠ Node.js v$version detected (Vite 5
requires >= 18)${NC}"
            return 1
        fi
    fi
    return 1
}

install_node_official() {
    echo -e "${BLUE}Installing Node.js via nvm (official)...${NC}"

    # Check if nvm is already installed
    if [ ! -d "$HOME/.nvm" ]; then
        echo "Installing nvm..."
        if ! curl -o- https://raw.githubusercontent.com/nvm-
sh/nvm/v0.40.0/install.sh | bash; then
            echo -e "${RED}✗ Failed to install nvm${NC}"
            return 1
        fi
    fi
```

```
    # Load nvm
    export NVM_DIR="$HOME/.nvm"
    [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"

    # Install Node.js LTS (v20.x)
    echo "Installing Node.js LTS..."
    if nvm install --lts; then
        nvm use --lts
        nvm alias default lts/*

        local installed_version=$(node -v)
        echo -e "${GREEN}✓ Node.js ${installed_version} installed${NC}"
        return 0
    else
        echo -e "${RED}✗ Node.js installation failed${NC}"
        return 1
    fi
}

update_node_version() {
    echo ""
    echo -e "${YELLOW}Vite 5 requires Node.js 18 or higher${NC}"
    read -p "Update Node.js to LTS version? (y/n): " -n 1 -r
    echo ""

    if [[ $REPLY =~ ^[Yy]$ ]]; then
        if ! install_node_official; then
            echo -e "${YELLOW}Falling back to NodeSource
installation...${NC}"
            echo -e "${YELLOW}This will require sudo password...${NC}"
            echo ""

            if curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo
-E bash -; then
                echo ""
                if sudo apt-get install -y nodejs; then
                    local new_version=$(node -v)
                    echo -e "${GREEN}✓ Node.js updated to
$new_version${NC}"
                    return 0
```

```
                fi
            fi

            echo -e "${RED}✗ Failed to update Node.js${NC}"
            return 1
        fi
    else
        echo -e "${YELLOW}⚠ Continuing with current Node.js version (may
cause issues)${NC}"
        return 1
    fi
}

# Check and install Node.js
if ! check_node_version; then
    # Check if Frontend uses Vite 5
    if [ -d "Frontend" ] && [ -f "Frontend/package.json" ]; then
        if grep -q '"vite".*"5\.' Frontend/package.json 2>/dev/null || \
           grep -q '"vite".*"\^5\.' Frontend/package.json 2>/dev/null;
then
            update_node_version
        else
            # No Vite 5, install without asking
            if ! command -v node &>/dev/null; then
                echo -e "${YELLOW}Installing Node.js...${NC}"
                install_node_official || sudo apt install -y nodejs npm
-qq
            fi
        fi
    else
        # No Frontend directory, install Node.js anyway
        if ! command -v node &>/dev/null; then
            echo -e "${YELLOW}Installing Node.js...${NC}"
            install_node_official || sudo apt install -y nodejs npm -qq
        fi
    fi
fi

# ============================================
# NPM Check
```

```
# ============================================
echo -n "Checking npm... "
if command -v npm &>/dev/null && npm --version &>/dev/null; then
    NPM_VERSION=$(npm --version 2>/dev/null)
    echo -e "${GREEN}✓ npm v${NPM_VERSION}${NC}"
else
    echo -e "${YELLOW}not found${NC}"
    echo -e "${YELLOW}Installing npm...${NC}"

    # Load nvm if available
    export NVM_DIR="$HOME/.nvm"
    [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"

    if command -v npm &>/dev/null; then
        NPM_VERSION=$(npm --version 2>/dev/null)
        echo -e "${GREEN}✓ npm v${NPM_VERSION}${NC}"
    else
        sudo apt update -qq && sudo apt install -y npm -qq
        if npm --version &>/dev/null; then
            NPM_VERSION=$(npm --version 2>/dev/null)
            echo -e "${GREEN}✓ npm v${NPM_VERSION} installed${NC}"
        else
            echo -e "${RED}✗ npm installation failed${NC}"
            exit 1
        fi
    fi
fi

# ============================================
# Other Dependencies
# ============================================
install_dep() {
    if ! command -v $1 &>/dev/null; then
        echo -e "${YELLOW}Installing $1...${NC}"
        (sudo apt update -qq && sudo apt install -y $2 -qq) &
        show_progress 2 "Downloading"
        wait
        echo -e "${GREEN}✓ $1 installed${NC}"
    else
        echo -e "${GREEN}✓ $1 available${NC}"
```

```
    fi
}

install_dep "mysql" "default-mysql-client"

# Rclone
if ! command -v rclone &>/dev/null; then
    echo -e "${YELLOW}Installing rclone...${NC}"
    (curl -fsSL https://rclone.org/install.sh | sudo bash > /dev/null
2>&1) &
    show_progress 4 "Downloading"
    wait
    echo -e "${GREEN}✓ rclone installed${NC}"
else
    echo -e "${GREEN}✓ rclone available${NC}"
fi

# ============================================
# MySQL Setup
# ============================================
echo -e "\n${BLUE}Database Server Setup${NC}"

check_mysql() {
    # Check if MySQL is running
    if systemctl is-active --quiet mysql 2>/dev/null || \
       systemctl is-active --quiet mysqld 2>/dev/null; then
        echo -e "${GREEN}✓ MySQL server running${NC}"
        return 0
    fi

    # Check if installed but not running
    if dpkg -l | grep -q mysql-server || dpkg -l | grep -q mariadb-
server; then
        echo -e "${YELLOW}Starting MySQL service...${NC}"
        sudo systemctl start mysql 2>/dev/null || sudo systemctl start
mysqld
        sleep 3
        return 0
    fi
```

```
    # MySQL not installed
    return 1
}

mysql_newly_installed=0
if check_mysql; then
    echo -e "${GREEN}✓ MySQL ready${NC}"
else
    # Install MySQL with user-defined credentials
    echo -e "${YELLOW}MySQL not found. Installing MySQL server...${NC}"
    echo ""
    echo -e "${BLUE}Configure MySQL Administrator Account${NC}"
    echo -e "${YELLOW}Set username and password for MySQL${NC}"

    # Get username from user
    read -p "MySQL username [root]: " MYSQL_NEW_USER
    MYSQL_NEW_USER=${MYSQL_NEW_USER:-root}

    # Get password from user
    while true; do
        read -sp "MySQL password for '$MYSQL_NEW_USER': " MYSQL_NEW_PASS
        echo ""

        if [ -z "$MYSQL_NEW_PASS" ]; then
            echo -e "${RED}Password cannot be empty${NC}"
            continue
        fi

        if [ ${#MYSQL_NEW_PASS} -lt 6 ]; then
            echo -e "${RED}Password must be at least 6 characters${NC}"
            continue
        fi

        read -sp "Confirm password: " MYSQL_NEW_PASS_CONFIRM
        echo ""

        if [ "$MYSQL_NEW_PASS" != "$MYSQL_NEW_PASS_CONFIRM" ]; then
            echo -e "${RED}Passwords do not match${NC}"
        else
            break
        fi
```

```
    done

    # Set debconf for non-interactive install
    echo "mysql-server mysql-server/root_password password
$MYSQL_NEW_PASS" | sudo debconf-set-selections
    echo "mysql-server mysql-server/root_password_again password
$MYSQL_NEW_PASS" | sudo debconf-set-selections

    echo ""
    echo -e "${YELLOW}Installing MySQL server...${NC}"
    sudo apt update -qq
    sudo DEBIAN_FRONTEND=noninteractive apt install -y mysql-server -qq

    sudo systemctl start mysql
    sudo systemctl enable mysql

    sleep 5
    echo -e "${GREEN}✓ MySQL server installed${NC}"

    # If user chose non-root username, create that user
    if [ "$MYSQL_NEW_USER" != "root" ]; then
        echo -e "${YELLOW}Creating user '$MYSQL_NEW_USER'...${NC}"
        mysql -u root -p"$MYSQL_NEW_PASS" -e "CREATE USER IF NOT EXISTS
'$MYSQL_NEW_USER'@'localhost' IDENTIFIED BY '$MYSQL_NEW_PASS';"
2>/dev/null
        mysql -u root -p"$MYSQL_NEW_PASS" -e "GRANT ALL PRIVILEGES ON
*.* TO '$MYSQL_NEW_USER'@'localhost' WITH GRANT OPTION;" 2>/dev/null
        mysql -u root -p"$MYSQL_NEW_PASS" -e "FLUSH PRIVILEGES;"
2>/dev/null
        echo -e "${GREEN}✓ User '$MYSQL_NEW_USER' created${NC}"
    fi

    DB_USER="$MYSQL_NEW_USER"
    DB_PASS="$MYSQL_NEW_PASS"
    mysql_newly_installed=1
fi

# ============================================
# MySQL Authentication
# ============================================
```

```
echo -e "\n${BLUE}MySQL User Credentials${NC}"

if [ $mysql_newly_installed -eq 1 ]; then
    echo -e "${GREEN}Using newly created account: $DB_USER${NC}"
else
    echo -e "${YELLOW}Enter your existing MySQL credentials${NC}"
    # Ask for existing MySQL credentials
    attempts=0
    max_attempts=3

    while [ $attempts -lt $max_attempts ]; do
        echo ""
        read -p "MySQL username [root]: " DB_USER
        DB_USER=${DB_USER:-root}

        read -sp "MySQL password: " DB_PASS
        echo ""

        if mysql -u "$DB_USER" -p"$DB_PASS" -e "SELECT 1;" 2>/dev/null;
then
            echo -e "${GREEN}✓ Authentication successful${NC}"
            break
        else
            attempts=$((attempts + 1))
            remaining=$((max_attempts - attempts))
            if [ $remaining -gt 0 ]; then
                echo -e "${RED}✗ Invalid credentials ($remaining
attempts left)${NC}"
            else
                echo -e "${RED}✗ Maximum attempts reached${NC}"
                exit 1
            fi
        fi
    done
fi

echo -e "${GREEN}✓ Will use MySQL user: $DB_USER${NC}"

# ============================================
# Database Configuration
```

```
# ============================================
echo -e "\n${BLUE}Database Configuration${NC}"

read -p "Database name [gbackup-db]: " DB_NAME
DB_NAME=${DB_NAME:-gbackup_db}

read -p "Database host [127.0.0.1]: " DB_HOST
DB_HOST=${DB_HOST:-127.0.0.1}

read -p "Database port [3306]: " DB_PORT
DB_PORT=${DB_PORT:-3306}

# ============================================
# Create Database
# ============================================
echo -e "\n${BLUE}Setting up database...${NC}"

# Create database if not exists
mysql -u "$DB_USER" -p"$DB_PASS" -e "CREATE DATABASE IF NOT EXISTS
\`$DB_NAME\`;" 2>/dev/null
echo -e "${GREEN}✓ Database '$DB_NAME' created${NC}"

# Test connection
if mysql -u "$DB_USER" -p"$DB_PASS" -D "$DB_NAME" -e "SELECT 1;"
2>/dev/null; then
    echo -e "${GREEN}✓ Database connection test passed${NC}"
else
    echo -e "${RED}✗ Database connection failed${NC}"
    exit 1
fi

# ============================================
# Rclone Configuration
# ============================================
echo -e "\n${BLUE}Storage Configuration${NC}"

# Auto-detect rclone config path based on current user
CURRENT_USER=$(whoami)
RCLONE_PATH="/home/$CURRENT_USER/.config/rclone/rclone.conf"
```

```
# Create rclone config directory if not exists
RCLONE_DIR=$(dirname "$RCLONE_PATH")
if [ ! -d "$RCLONE_DIR" ]; then
    mkdir -p "$RCLONE_DIR"
    echo -e "${YELLOW}Created rclone config directory${NC}"
fi

echo -e "${GREEN}✓ Rclone path: $RCLONE_PATH${NC}"

# ============================================
# Service Port Configuration
# ============================================
echo -e "\n${BLUE}Service Configuration${NC}"

# Function to check if port is available
is_port_available() {
    ! lsof -Pi :$1 -sTCP:LISTEN -t >/dev/null 2>&1
}

# Find available port
APP_PORT=""
DEFAULT_PORTS=(8080 3000 3001 8081 8082 8083 5173 5174 4000)

echo -n "Finding available port... "
for port in "${DEFAULT_PORTS[@]}"; do
    if is_port_available $port; then
        APP_PORT=$port
        break
    fi
done

# If all default ports are taken, find any available port in range 8000-
9000
if [ -z "$APP_PORT" ]; then
    for port in $(seq 8000 9000); do
        if is_port_available $port; then
            APP_PORT=$port
            break
        fi
    done
fi
```

```
# If still no port found, ask user
if [ -z "$APP_PORT" ]; then
    echo ""
    echo -e "${YELLOW}All default ports (8000-9000) are in use${NC}"
    while true; do
        read -p "Enter custom port (1024-65535): " APP_PORT

        # Validate port number
        if ! [[ "$APP_PORT" =~ ^[0-9]+$ ]]; then
            echo -e "${RED}✗ Invalid port number${NC}"
            continue
        fi

        if [ "$APP_PORT" -lt 1024 ] || [ "$APP_PORT" -gt 65535 ]; then
            echo -e "${RED}✗ Port must be between 1024-65535${NC}"
            continue
        fi

        if ! is_port_available $APP_PORT; then
            echo -e "${RED}✗ Port $APP_PORT is already in use${NC}"
            continue
        fi

        break
    done
else
    echo "Done"
fi

# Generate JWT secret
JWT_SECRET=$(openssl rand -base64 32)

echo -e "${GREEN}✓ Service port: $APP_PORT${NC}"
echo -e "${GREEN}✓ Security token generated${NC}"

# ============================================
# Create Environment File
# ============================================
echo -e "\n${BLUE}Creating configuration file...${NC}"
```

```
cat > Backend/.env << EOF
# Database Configuration
DB_USER=$DB_USER
DB_PASS=$DB_PASS
DB_HOST=$DB_HOST
DB_PORT=$DB_PORT
DB_NAME=$DB_NAME

# Security
JWT_SECRET_KEY=$JWT_SECRET

# Storage
RCLONE_CONFIG=$RCLONE_PATH

# Network
SERVER_HOST=0.0.0.0
APP_PORT=$APP_PORT
EOF

echo -e "${GREEN}✓ Configuration created${NC}"

# ============================================
# Build Backend Application
# ============================================
echo -e "\n${BLUE}Building application...${NC}"

cd Backend

# Fix go.mod version format if needed (remove patch version)
if grep -q 'go [0-9]\+\.[0-9]\+\.[0-9]\+' go.mod 2>/dev/null; then
    echo -e "${YELLOW}Fixing go.mod version format...${NC}"
    sed -i 's/go \([0-9]\+\.[0-9]\+\)\.[0-9]\+/go \1/' go.mod
    echo -e "${GREEN}✓ go.mod fixed${NC}"
fi

# Ensure dependencies are downloaded
echo -n "Downloading dependencies... "
(go mod download 2>&1 > /tmp/go_download.log) &
pid=$!
```

```
while kill -0 $pid 2>/dev/null; do
    echo -n "▓"
    sleep 0.3
done
wait $pid
if [ $? -ne 0 ]; then
    echo ""
    echo -e "${RED}✗ Failed to download dependencies${NC}"
    cat /tmp/go_download.log
    cd ..
    exit 1
fi
echo " Done"

go mod tidy > /dev/null 2>&1

# Build the application
echo -n "Compiling... "
(go build -o app ./cmd/main.go 2>&1 > /tmp/go_build.log) &
pid=$!
while kill -0 $pid 2>/dev/null; do
    echo -n "▓"
    sleep 0.2
done
wait $pid
if [ $? -ne 0 ]; then
    echo ""
    echo -e "${RED}✗ Build failed${NC}"
    echo -e "${YELLOW}Error details:${NC}"
    cat /tmp/go_build.log
    cd ..
    exit 1
fi
echo " Done"

echo -e "${GREEN}✓ Application built${NC}"
cd ..

# ============================================
# Install Frontend (if exists)
```

```
# ============================================
if [ -d "Frontend" ] && [ -f "Frontend/package.json" ]; then
    echo -e "${BLUE}Installing frontend...${NC}"

    # Load nvm if available
    export NVM_DIR="$HOME/.nvm"
    [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"

    # Double-check npm is working
    if ! npm --version &>/dev/null; then
        echo -e "${RED}✗ npm is not working${NC}"
        exit 1
    fi

    cd Frontend

    # Run npm install with progress
    echo -n "Installing packages... "
    (npm install --silent 2>&1 > /tmp/npm_install.log) &
    pid=$!
    while kill -0 $pid 2>/dev/null; do
        echo -n "▓"
        sleep 0.5
    done
    wait $pid
    if [ $? -ne 0 ]; then
        echo ""
        echo -e "${RED}✗ Frontend installation failed${NC}"
        cat /tmp/npm_install.log
        cd ..
        exit 1
    fi
    echo " Done"

    echo -e "${GREEN}✓ Frontend ready${NC}"
    cd ..
fi

# ============================================
# Installation Complete
```

```
# ============================================

# Get the correct network IP (not NAT/loopback)
get_network_ip() {
    # Get all IPs excluding loopback
    local all_ips=$(ip -4 addr show | grep "inet " | grep -v "127.0.0.1"
| \
                    awk '{print $2}' | cut -d/ -f1)

    # Get IPs excluding common NAT ranges
    local good_ips=$(echo "$all_ips" | grep -v "^10\.0\.2\." | grep -v
"^172\.17\." | grep -v "^169\.254\.")

    # Check if we have good IPs
    if [ ! -z "$good_ips" ]; then
        local ip_count=$(echo "$good_ips" | wc -l)

        # If only one good IP, use it
        if [ $ip_count -eq 1 ]; then
            echo "$good_ips"
            return 0
        fi

        # Multiple good IPs - use priority logic
        local default_iface=$(ip route | grep "^default" | awk '{print
$5}' | head -n1)
        if [ ! -z "$default_iface" ]; then
            local default_ip=$(ip addr show "$default_iface" | grep
"inet " | grep -v "127.0.0.1" | \
                               awk '{print $2}' | cut -d/ -f1 | head -n1)
            if [ ! -z "$default_ip" ] && [[ ! $default_ip =~ ^10\.0\.2\.
]]; then
                echo "$default_ip"
                return 0
            fi
        fi

        # Priority 2: Prefer 192.168.x.x or 10.x.x.x
        for ip in $good_ips; do
            if [[ $ip =~ ^192\.168\. ]] || [[ $ip =~ ^10\.[0-9]+\.[0-
9]+\. ]]; then
```

```
                echo "$ip"
                return 0
            fi
        done

        # Use first good IP
        echo "$good_ips" | head -n1
        return 0
    fi

    # No good IPs found, use NAT IP as fallback
    if [ ! -z "$all_ips" ]; then
        echo "$all_ips" | head -n1
        return 1
    fi

    # Absolute fallback
    echo "127.0.0.1"
    return 2
}

SERVER_IP=$(get_network_ip)
IP_STATUS=$?

# Show all available IPs
ALL_IPS=$(ip -4 addr show | grep "inet " | grep -v "127.0.0.1" | \
          awk '{print $2}' | cut -d/ -f1)
IP_COUNT=$(echo "$ALL_IPS" | wc -l)

echo -e "\n${GREEN}✓ System successfully installed${NC}"
echo ""

# Check if using NAT IP (status = 1)
if [ $IP_STATUS -eq 1 ]; then
    echo -e "${BLUE}Access URL:${NC}"
    echo "  http://$SERVER_IP:$APP_PORT"
    echo ""
    echo -e "${YELLOW}⚠ Network Notice:${NC}"
    echo "  Using NAT IP - may not be accessible from other devices"
    echo "  Fix: VirtualBox → Settings → Network → Bridged Adapter"
```

```
elif [ $IP_COUNT -gt 1 ]; then
    echo -e "${BLUE}Access URL (multiple networks):${NC}"
    while IFS= read -r ip; do
        if [ "$ip" = "$SERVER_IP" ]; then
            echo -e "  ${GREEN}➜ http://$ip:$APP_PORT${NC} (primary)"
        else
            echo -e "    http://$ip:$APP_PORT"
        fi
    done <<< "$ALL_IPS"
else
    echo -e "${BLUE}Access URL:${NC}"
    echo "  http://$SERVER_IP:$APP_PORT"
fi
echo ""

# Show instructions to reload shell
echo -e "${YELLOW}Note: If Go or Node.js commands are not found, please
run:${NC}"
echo -e "${YELLOW}  source ~/.bashrc${NC}"
```

Jadwal Pengujian

No	Aktivitas	Schedule															
		Agustus		September				Oktober					November				Desember
		w1	w2	w3	w4	w5	w6	w7	w8	w9	w10	w11	w12	w13	w14	w15	w16
1	Penentuan Topik PA																
2	Survey/Studi Pendahuluan																
3	Pembelian/Pengadaan Kebutuhan Penelitian																
4	Penulisan/Peyusunan Bab I																
5	Penulisan/Peyusunan Bab II																
6	Penulisan/Peyusunan Bab III																
7	Proposal ACC																
8	Seminar Proposal (Penyusunan TA 1)																
9	Revisi Proposal																
10	Pelaksanaan Penelitian TA																
11	Penulisan/Penyusunan Bab 4																
	Testing																
	Implementasi																
	Hasil dan Pembahasan																
12	Penulisan Bab 5																
13	Ujian Hasil (Prasidang)																
14	Revisi Prasidang (Penyusunan Laporan Akhir Sidang TA)																
15	Sidang																
16	Revisi/Cek Tata Tulis																
17	Pengumpulan Kebutuhan TA																